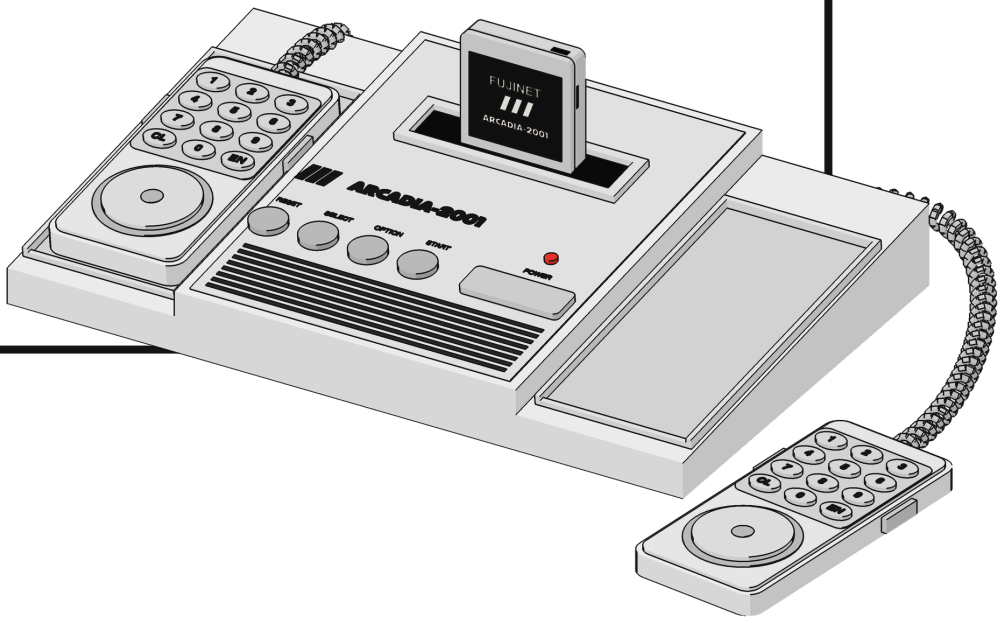
ARCADIA-2001

WITH FUJINET

PROGRAMMING THE NETWORK CARTRIDGE



PROGRAMMER'S HANDBOOK

MODEL NO. FN2001-PH

IN A HURRY?

1. Build the Macroassembler AS, and MAME with the FujiNet cartridge grafted in (Section 2).
2. Start the bench: `bench/up.sh` runs a fujinet-pc of the book's own and the two little servers the network programs talk to.
3. Build every program: `make listings`.
4. Run one: `emu/run.sh listings/asm/build/fujitest.bin`. Its screen shows the adapter's network name, firmware version and address.
5. Read Section 4, How the Mailbox Works, before you write a line: a console that can only read its cartridge talks to the network by reading, and everything else follows from that.

FujiNet Programmer's Handbook for the EMERSON ARCADIA-2001. First edition, 2026.

Typeset after the 1982 Emerson *ARCADIA-2001 Owner's Manual* and the Emerson game instruction booklets of the same year, as the FujiNet *Owner's Manual* for the Arcadia is. FujiNet is an independent project and is not affiliated with, endorsed by, or sponsored by Emerson Radio Corp. "Emerson" and "Arcadia 2001" are trademarks of their owners and are used here to identify the console this book is about. "M.U.L.E." is a trademark of its owner.

No FujiNet Arcadia cartridge has been built yet. Every program in this book was assembled from its listing and run on the FujiNet model of the cartridge in MAME, which compiles the cartridge's own firmware sources, against a fujinet-pc built from the same firmware; each one's screen was checked by a script. Where a real console may differ, the text says so.

Text, pictures and programs © 2026 the FujiNet Project. Text and pictures CC BY-SA 4.0; the programs GPL-3.0, as the firmware they come from.

CONTENTS

1 UNPACKING YOUR FUJINET CARTRIDGE. 1
2 INSTALL THE TOOLS. 3
3 THE MEMORY MAP. 6
4 HOW THE MAILBOX WORKS. 9
5 THE TEXT DISPLAY. 14
6 LIVING IN 288 BYTES. 16
7 THE 2650 LIBRARY. 18
8 PAGES, BLOCKS AND POINTERS. 21
9 HAND CONTROLLERS AND SOUND. 23
10 FIRST CONTACT. 27
11 THE NETWORK DEVICE. 30
12 NETCAT. 34
13 DIRECTORIES, MOUNT AND BOOT. 38
14 APP KEYS, THE CLOCK AND UTILITIES. 42
15 BIGGER THAN 8K: PROTOCOL 2. 45
16 THE 2637 UP CLOSE. 48
17 COMMAND REFERENCE: THE FUJI DEVICE. 51
18 COMMAND REFERENCE: THE NETWORK DEVICE. 59
19 COMMAND REFERENCE: DISK, PRINTER, CLOCK AND MODEM. 64
20 COMMAND REFERENCE: THE CARTRIDGE ITSELF. 68
21 BATTLESHIP. 72
22 5 CARD STUD. 76
23 CONFIG. 79
24 THINGS THAT COST REAL TIME. 83
25 TROUBLE SHOOTING CHECK LIST. 85
26 PARTS LIST. 87
A MAILBOX QUICK REFERENCE. 89
B ERROR AND STATUS CODES. 90
C PROGRAM LISTINGS. 91
D GAME AND CONFIG LISTINGS. 118

1. UNPACKING YOUR FUJINET CARTRIDGE

INTRODUCTION

The FujiNet cartridge puts the EMERSON ARCADIA-2001 on the network. In the console's cartridge slot it is a cartridge like any other; inside it are two small computers. The **game chip**, an RP2354A, answers every read the console's Signetics 2650 makes: it serves your program as a mask ROM would, and beside it a *mailbox* through which your program talks to the second computer. The **network chip**, an ESP32-S3, is the FujiNet itself, the same adapter that serves the Atari, the Apple II, the ColecoVision and a dozen other machines, with its WiFi, its memory card and its network devices.

This handbook is for the programmer. It shows how a program written in 2650 assembly language reaches the network through that mailbox, one read at a time, and it carries every command the FujiNet answers, each with a 2650 example. Assembly is the only language here because it is the only one there is: no compiler, BASIC or C, has ever targeted the Arcadia.

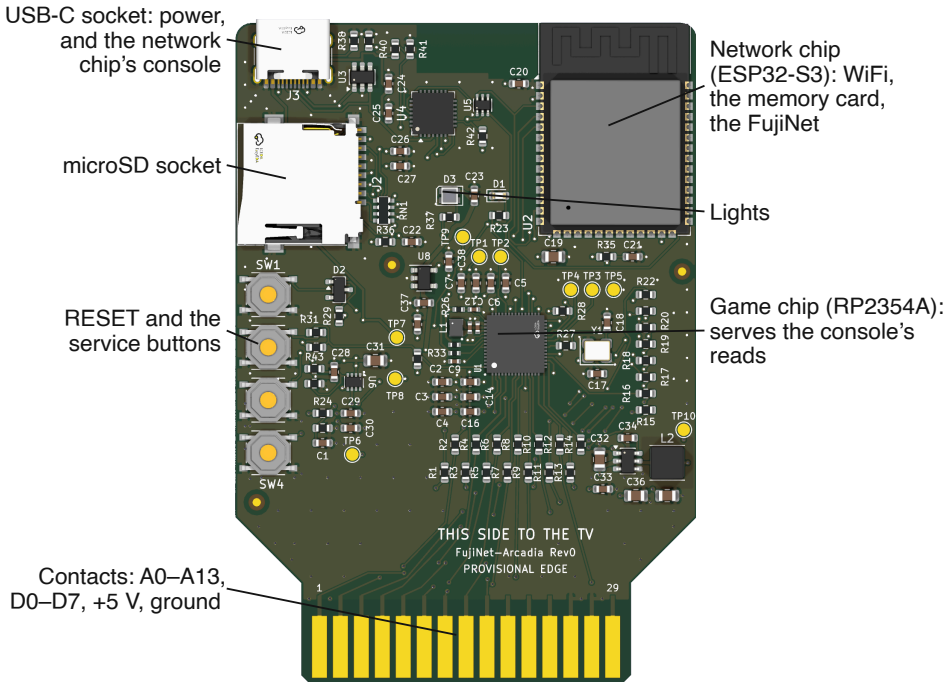


FIG. 1
THE FUJINET ARCADIA BOARD, REV0, FROM THE BACK

WHAT YOU SHOULD HAVE

– An ARCADIA-2001 and a FujiNet Arcadia cartridge – or, until there is one, the FujiNet model of the cartridge in MAME (Section 2), which is what every program in this book was run on.
– A computer with Python 3 and the Macroassembler AS, which assembles the 2650.
– A FujiNet to talk to: the cartridge's own network chip, or fujinet-pc, the FujiNet firmware built to run on a computer, which is what MAME's cartridge talks to.
– The listings that come with this handbook (`listings/`), which build every program it shows.

NOTE: No FujiNet Arcadia cartridge has been built yet. Everything in this handbook was run on the cartridge's model in MAME, which compiles the cartridge's own firmware sources, against a fujinet-pc built from the same firmware. Where a real console may differ – timing, mostly, and one bus fight (Section 15) – the text says so.

HOW THIS HANDBOOK IS ORGANISED

The machine, Sections 2 to 9: the tools; the memory map; the mailbox, which every later section leans on; the text display; the console's 288 bytes of RAM; the 2650 library the programs share; the 2650's 8K pages, which shape every program; and the hand controllers and the tone.

The programs, Sections 10 to 16, each built and run: first contact with the adapter, the network device, a netcat, directories and booting, app keys and the clock, programs bigger than the cartridge window, and the 2637 video chip up close.

The reference, Sections 17 to 20: every command the FujiNet answers, with the parameters it wants, the reply it sends and a 2650 example; then everything the cartridge itself does.

The games, Sections 21 to 23: how Battleship, 5 Card Stud and CONFIG are built. Then the things that cost real time, the Trouble Shooting Check List, the Parts List, and four appendices with the quick reference, the codes and the listings.

CONVENTIONS

- Numbers with a `$` are hexadecimal, as AS writes them: `$2B00`.
- **The console** is the ARCADIA-2001; **the cartridge** is the FujiNet cartridge in its slot; **the adapter** is the network chip, or the fujinet-pc standing in for it.
- A **cell** is a place on the screen, row × 16 + column: cell `$A3` is row 10, column 3.
- The 2650's registers are R0 to R3. The programs never select the second bank of R1 to R3.
- Every example in the text assembles: `tools/snipcheck.py` builds each one inside a program and checks the result (Section 26).

2. INSTALL THE TOOLS

A FujiNet program for the Arcadia is an 8,192-byte cartridge image: the two 4K blocks the console's slot shows, with the mailbox in the top of the second. Four tools make and run one: an assembler, three Python scripts, MAME with the FujiNet cartridge grafted in, and a FujiNet to talk to.

THE ASSEMBLER

Every program in this handbook is written for the Macroassembler AS by Alfred Arnold, which knows the 2650 as `CPU 2650`. Build it from its source (the `Macroassembler-AS/asl-releases` repository) or take a package; the handbook was checked with version 1.42, build 306.

1. Unpack AS and run its `make`. Keep `asl` and `p2bin` together; the build scripts look on your `PATH` first, then in `~/asl/`.
2. AS assembles a program to a `.p` file of address-tagged records. `p2bin` turns that into a flat binary, and the cartridge's `mkimage.py` folds the flat into the image (below).

IMPORTANT: Give a macro's arguments their own parentheses inside the macro: `(adr)>>8`, never `adr>>8`. AS substitutes the text, so an argument of `TABLE+2` becomes `TABLE+2>>8` and shifts the 2.

THE PYTHON TOOLS

Three scripts from the cartridge's firmware come with the listings, in `listings/runtime/tools/`, and every build runs them:

1) MKIMAGE.PY
   Folds the `$0000-$2FFF` flat that `p2bin` writes into the cartridge image: CPU `$0000-$0FFF` is image `0x0000`, CPU `$2000-$2FFF` is image `0x1000`. It refuses code in the `$1000-$1FFF` hole (the console's own RAM and video chip) and in the mailbox, `$2B00-$2FFF`.
2) CHECKROM.PY
   Checks the image: 8,192 bytes (or the protocol 2 sizes of Section 15), the `BCTA` header at `$0000`, the `RETC` guard at `$0003`, nothing in the mailbox but the claim, and the claim itself.
3) CHECKDEPTH.PY
   Bounds the depth of the program's subroutine calls: the 2650's return stack is eight deep and on the chip, and a ninth call overwrites the first return address without a word (Section 7).

The handbook adds a stricter fourth, `tools/depthcheck.py`, which also follows branches and fall-through into a routine's own labels; Section 7 shows what the firmware's script misses. They need Python 3 and nothing else.

MAME AND THE CARTRIDGE

The cartridge you will use is MAME's. The firmware carries a model of it, `pico/arcadia/emu/`, which compiles the cartridge's own C sources – the mailbox, the image store, the bus decode – into MAME's `arcadia` driver.

1. Check out MAME and the FujiNet firmware.
2. Graft the cartridge in: `pico/arcadia/emu/apply.sh ~/Workspace/mame`. It copies the device and the firmware's sources into `src/devices/bus/arcadia/`, adds the device to the bus, and the option `fujinet` to the cartridge slot.
3. Build MAME: `make -j$(nproc) REGENIE=1`. A build of the Arcadia alone is quicker: add `SUBTARGET=arcadia SOURCES=src/mame/arcadia/arcadia.cpp`.

The arcadia driver needs no BIOS: the console has none. The cartridge serves the image named on the command line, and FUJINET_TCP says where the FujiNet is:

```
FUJINET_TCP=127.0.0.1:9963 ./mame arcadia -cartslot fujinet -cart hello.bin
```

THE FUJINET

The network chip runs the FujiNet firmware's fujiversal-arcadia build, an RS-232 build (BUILD_RS232) that speaks to the game chip over USB. On a computer the same build runs as fujinet-pc, and MAME reaches it over *bus-over-IP* (BoIP), a TCP connection in place of the USB cable inside the cartridge.

1. Build it from the firmware: ./build.sh -p RS232. The result is in build/dist/.
2. Copy fujinet, run-fujinet and data/ to a folder of their own, with an SD/ folder beside them for the memory card.
3. Give it an fnconfig.ini with BoIP on: [BOIP], enabled=1, host=localhost and a port= nothing else on your computer uses.
4. Start it: ./run-fujinet -u http://127.0.0.1:8063 -c fnconfig.ini -s SD. The page at that address is the adapter's web control panel, as on any FujiNet.

IMPORTANT: Always set FUJINET_TCP when you start MAME. Unset, the cartridge tries 127.0.0.1:9995, and on a computer with other FujiNet rigs that port may be one of theirs. Only one MAME may talk to a fujinet-pc at a time.

THE HANDBOOK'S BENCH

The handbook keeps its whole bench in one place. bench/ports.env names its ports; bench/up.sh refuses any of them that something else holds, then starts fujinet-pc on BoIP 9963 (web page on 8063) with a fresh copy of the book's memory card, a small web server for the network chapter's test file (8790), and the line server the netcat talks to (8791). bench/up.sh games adds the Battleship and 5 Card Stud servers (8792, 8793), built from the FujiNet game servers with production mode off, so they never tell the public Lobby. bench/down.sh stops them all.

Command	Does
make listings	assembles every program into listings/asm/build/
make verify	runs each one in MAME against the bench and checks its screen
make shots	refreshes the screens in this book from the last runs
make check	checks the book against its sources (Section 26)

RUNNING A PROGRAM

emu/run.sh starts MAME with a program in the cartridge, in a window:

```
emu/run.sh listings/asm/build/netget.bin
```

With SNAP= set it runs with no window and emu/walk.lua drives it: STEPS= is a list of things to wait for and keys to press, WANT= the words that must be on the screen at the end. That is how emu/verify.sh checks every program.

Console	MAME's key	Note
Left keypad	the numeric pad	CLEAR is the pad's -, ENTER its +
Left disc	the arrows	read as a pot; Section 9
Action buttons	the pad's 2	one wire with the keypad's 2
Right keypad	Z X C, A S D, Q W E, V F R	1 2 3 on Z X C: the keypad upside down
START, SELECT, OPTION	1, 5, O	RESET is not wired in MAME's driver

NOTE: MAME's second disc takes R, F, D and G, which are also keys of the right keypad. Give one of them other keys in MAME's input menu (Tab) before you test a two-player program.

ON A REAL CONSOLE

When the cartridge exists, a program travels the way every FujiNet program does: copy the `.bin` to the FujiNet's memory card or a TNFS server, choose it in CONFIG, and CONFIG mounts it and hands the console over (Section 13). To come back to CONFIG, press the cartridge's own RESET, then the console's.

3. THE MEMORY MAP

The 2650 addresses 32K, `$0000–$7FFF`. The ARCADIA-2001 puts three things in it: the cartridge, the 2637 video chip with the console's RAM, and nothing.

WHAT THE CONSOLE SEES

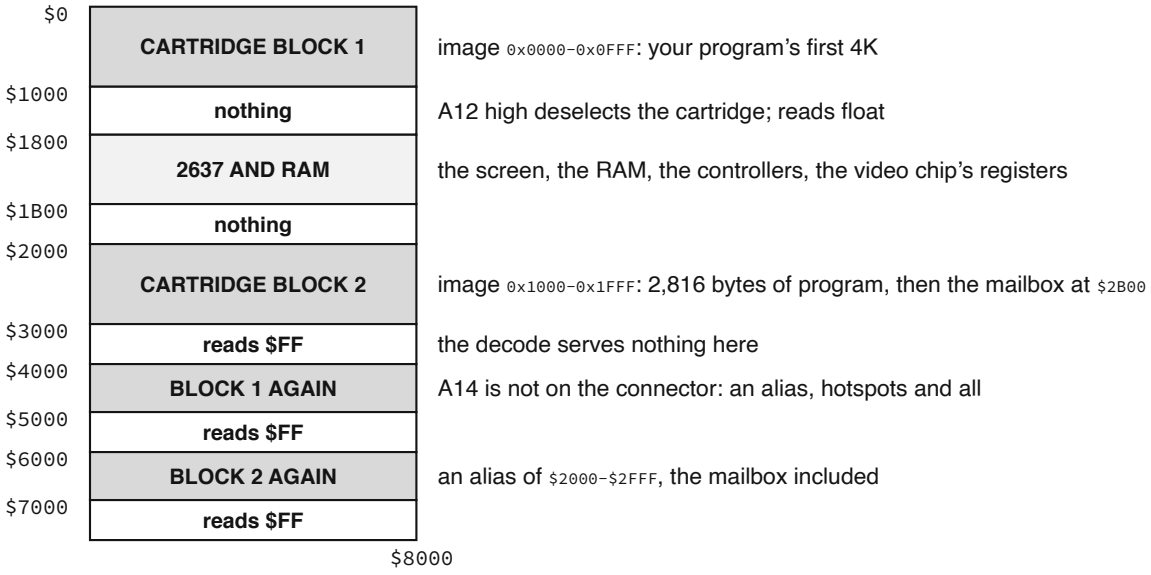


FIG. 2
THE 2650'S 32K, AS THE ARCADIA-2001 DECODES IT

The cartridge connector carries the address lines A0–A13, the data lines D0–D7, +5 V and ground, and nothing else: no write strobe, no clock, no reset. A12 is the cartridge's chip select, low for `$0000–$0FFF` and `$2000–$2FFF`; A13 picks which 4K block. Everything follows from that wiring:

– **The console can only read the cartridge.** There is no write line, so a store into cartridge space reaches nothing – and on a real console it is worse than nothing (Section 15). Every word your program says to the FujiNet, it says by reading (Section 4).
– **A program is at most 8K**, two 4K blocks with a hole between them. Section 15 shows how a program bigger than that pages through a window.
– **The two blocks are in different 8K pages of the 2650.** Block 1 is page 0 (`$0000–$1FFF`, shared with the video chip and the RAM); block 2 is page 1. A 2650 load or store reaches only its own page, so where a piece of code lives decides what it can touch directly (Section 8).

THE CARTRIDGE IMAGE

The image a FujiNet program is built into is exactly 8,192 bytes:

Image	Console	Holds
0x0000-0x0002	$0000	BCTA,UN START: the console starts at $0000; there is no BIOS
0x0003	$0003	$17, a RETC,UN: an interrupt guard, so a stray interrupt returns at once
0x0004-0x0FFF	$0004–$0FFF	your program (the runtime starts at $0020)
0x1000-0x1AFF	$2000–$2AFF	more program: 2,816 bytes
0x1B00-0x1BFF	$2B00–$2BFF	the mailbox's reply slice (Section 4)
0x1C00-0x1CFB	$2C00–$2CFB	the mailbox's status page
0x1CFC-0x1CFF	$2CFC–$2CFF	the claim: the four letters FUJI
0x1D00-0x1FFF	$2D00–$2FFF	the three hotspot pages

So a client has 4,096 + 2,816 = **6,912 bytes** of code and data. mkimage.py refuses a byte of it in the hole or in the mailbox, and checkrom.py refuses an image without the header, the guard or the claim.

IMPORTANT: The claim is a promise. "FUJI" at $2CFC says the image keeps $2B00–$2FFF free, so the cartridge leaves the mailbox alive when it boots the image. An image without it – any of the console's own 1982 games – kills the mailbox for the rest of the session, so that a game which happens to read a hotspot address can never start a transaction by accident.

THE 2637 AND THE CONSOLE'S RAM

The video chip, Signetics' 2637 Universal Video Interface (UVI), sits at $1800–$1AFF, and the console's RAM is threaded through it:

Address	Use	What it is
$1800–$18CF	r/w	the upper screen: 13 rows of 16 cells (Section 5)
$18D0–$18EF	r/w	32 bytes of RAM: the runtime's and the boot stub's
$18F0–$18F7	w	the four sprites' vertical and horizontal positions
$18F8–$18FB	r/w	4 bytes of RAM
$18FC	w	where the character display starts, as a complement
$18FD	w	bits 0–6 the tone's pitch, bit 7 the colour mode
$18FE	w	bits 0–2 loudness, 3 tone on, 4 noise on, 5–7 a scroll of 0–7 pixels left
$18FF	r	the character row being drawn, in bits 0–3
$1900–$1902	r	the left keypad, one column each (Section 9)
$1904–$1906	r	the right keypad
$1908	r	START, OPTION, SELECT
$1980–$19BF	w	the eight user-defined characters $38–$3F, 8 bytes each; the first four are the sprites' images
$19F8	w	bit 7 block-graphics mode, bit 6 the 26-row screen, 0–5 the alternate colours
$19F9	w	bit 7 character height, bit 6 the pot multiplexer, 3–5 and 0–2 the colours
$19FA–$19FB	w	the sprites' colours and heights
$19FC	r	sprite against character collisions (a 0 is a hit)
$19FD	r	sprite against sprite collisions
$19FE–$19FF	r	the right and left discs' pots, valid in the vertical blank
$1A00–$1ACF	r/w	the lower screen in the 26-row mode; otherwise RAM
$1AD0–$1AFF	r/w	48 bytes of RAM

In the 13-row mode every program in this book uses, `$1A00–$1AFF` is 256 bytes of plain RAM, and with `$18D0–$18EF` that is the console's working memory: **288 bytes** (Section 6). The 2650 has no RAM stack – its return addresses live on the chip – so all of it is yours.

FRAME TIME

The 2637 raises the 2650's SENSE input during the vertical blank, and `TPSU $80` tests it: CC is EQ while the blank lasts. There are no interrupts to use instead; every program in this book waits for the blank in a loop:

```
WAITVB: TPSU    $80             ; SENSE: 1 in the vertical blank
        BCFR,EQ WAITVB
```

The pots, the collision registers and a flicker-free screen change all want the blank, and a frame is 1/60 s. The 2650 runs at 3.58 MHz ÷ 4, about 895 kHz, three clocks to a processor cycle of 3.35 μs: some 5,000 cycles a frame. An instruction takes two to four cycles – a register move two, an absolute load four – so a frame holds 1,500 or so of them.

4. HOW THE MAILBOX WORKS

The ARCADIA-2001's cartridge slot was built for a mask ROM. It carries an address and gets back a byte, and that is all it does: there is no line on the connector a write could use. A program cannot store a byte into the cartridge.

So the cartridge listens to *where* the console reads. Every read puts an address on the connector, and the game chip sees all fourteen lines of it. A read of `$2F41` is a perfectly good way of saying `$41`: the high byte names what the byte is for, the low byte carries it. Going the other way is simpler still. The game chip paints its answers into the ROM it serves, and by the time the console reads them they are just ROM. The FujiNet mailbox is built on those two ideas.

WRITING BY READING

The mailbox has three pages of *hotspots*. A read from any of them sends its low byte to the cartridge; what it gives back the program ignores.

Page	A read of `$pp`*vv* sends vv as
`$2D00`	REGSEL: the number of the register the next byte goes to
`$2E00`	REGDATA: the value for the register REGSEL named
`$2F00`	one byte of the TX stream: the parameters, then the payload

Setting a register takes two reads, one of `$2D00` plus the register number, then one of `$2E00` plus the value. A REGDATA read with no REGSEL just before it is ignored, and a REGSEL is good for one REGDATA. The runtime's `FNREGWR` makes the pair:

2650 ASSEMBLY (AS) · FUJILIB.INC

```
; Write mailbox register: R1 = register number, R2 = value.
; Leaf. Clobbers R0.
FNREGWR:
        LODA,R0 *FSELP,R1       ; arm the register
        LODA,R0 *FDATP,R2       ; deliver the value
        RETC,UN
```

The register number and the value ride in the *index* registers, R1 and R2. That is the 2650 idiom the whole mailbox is written in: `LODA,R0 *FSELP,R1` reads the address the pointer word `FSELP` holds (`$2D00`), plus R1. The byte read lands in R0 and nobody looks at it.

NOTE: Why the pointer word? The runtime lives in block 1, page 0 of the 2650's address space, and the hotspots are in block 2, page 1. A 2650 load reaches only its own 8K page, but an indirect address carries all fifteen bits. Section 8 is about this rule; code in block 2 reads the hotspots directly, as CONFIG does.

THE REGISTERS

Name	No.	A value written
DEVICE	`$00`	the FujiNet device: `$70` the Fuji device, `$71` the first network channel, `$45` the clock
CMD	`$01`	the command, such as `$C4`
NPARAM	`$02`	how many parameters lead the TX stream; at most 8
DATA_RST	`$05`	any value: the TX stream starts over
RXSLICE	`$06`	which 256 bytes of the reply the window shows, 0–3
SEQ	`$10`	a new sequence number: **launch**
BOOTLOCK	`$11`	`$B5`: arm a staged image (Section 13)

The rest – the two-step BOOTSEL and the cartridge RAM registers of protocol 2 – are in Section 20.

THE TX STREAM

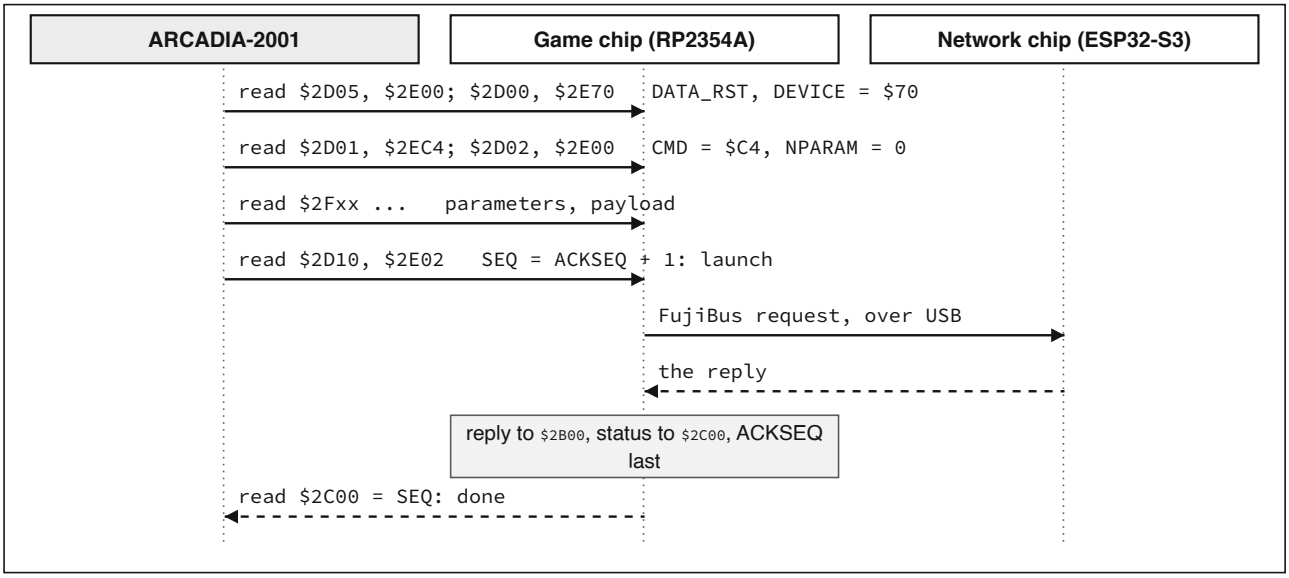
After the registers, the program streams what the command takes. Each parameter is its size, 1, 2 or 4, then its value, low byte first: a one-byte 7 goes as `$01 $07`, a two-byte 512 as `$02 $00 $02`. After the parameters comes the payload, the command's free-form bytes, such as a URL. The stream holds 320 bytes; the cartridge drops any more.

Routine	Streams
`FNPARB`	a one-byte parameter, the value in R2
`FNPARW`	a two-byte parameter, R2 the low byte, R3 the high (`booklib.inc`)
`FNTXSTR`	the NUL-terminated string at `(FSTRP)`, without its NUL
`FNPATH`	the same string, padded with NULs to exactly 256 bytes
`FNTXN`	R2 bytes from `(FSTRP)`, NULs and all (`booklib.inc`)

To append a byte *v* the program reads `$2F00` + *v*, so the byte to send must be in an index register: `LODA,R0 *FTXP,R1` appends R1.

THE TRANSACTION

A command is a short conversation. The program names the device and the command, says how many parameters follow, streams them, and writes a new sequence number. That last write launches the transaction. The game chip sends the request to the network chip over USB, waits for its reply, and paints it into the window it serves: the first 256 bytes at `$2B00`, a status page at `$2C00`. The very last byte it paints is ACKSEQ, the sequence number the program wrote: when ACKSEQ matches, the whole reply is in place.



THE STATUS PAGE

The cartridge paints the status page, `$2C00–$2C0F`, with every reply:

At	Name	Holds
`$2C00`	ACKSEQ	the sequence number answered; painted last
`$2C01`	STATUS	bit 0: the link to the network chip is up
`$2C02`	ERR	0 when the transaction went through (Appendix B)
`$2C03`	REPLY	`$06` ACK or `$15` NAK: the FujiNet's verdict
`$2C04`	RXLEN	how long the reply is, two bytes, low first; at most 1,024
`$2C06`	BOOT_STATE	the staged image: idle, coming, ready, failed (Section 13)
`$2C07`	BOOT_PCT	how much of it has come, 0–100
`$2C08`	BOOT_ERR	why staging failed
`$2C09`	MAGIC	`'F'`, `'N'`: a FujiNet cartridge is there
`$2C0B`	PROTO_VER	the mailbox's version: 2
`$2C0C`	SLICE_ECHO	which reply slice the window shows; painted last after a slice
`$2C0E`	TICK	a 60 Hz count, two bytes (Section 15)

`FNCHECK` reads the magic: R0 = 0 when the cartridge is a FujiNet. On an ordinary 8K EPROM those two bytes are whatever the image holds there, which is how `fujitest` tells you it is not on a FujiNet cartridge.

THE REPLY WINDOW

A reply can be up to 1,024 bytes, but the window at `$2B00` is 256: it shows one *slice* at a time. After a transaction it shows slice 0. To see bytes 256–511, write RXSLICE = 1 and wait for SLICE_ECHO to say 1 – the repaint takes a moment on the real cartridge, and SLICE_ECHO is painted after it. `FNSLICE` does both, with the slice in R2:

2650 ASSEMBLY (AS) · FUJILIB.INC

```
; Select reply slice R2 (0-3) and wait for FNSECHO, painted last, to echo it.
; Returns R0 = 0 ok, FETIMO on timeout (~1.5 s). Preserves R2.
; Calls one down. Clobbers R0,R1,R3.
FNSLICE:
        LODI,R1 FRRXSLC
        BSTA,UN FNREGWR
        LODI,R3 64              ; outer x64
FNSLO:  LODI,R1 0               ; inner x256
FNSLI:  LODA,R0 *FSECP
        COMZ    R2
        BCTR,EQ FNSLOK
        BDRR,R1 FNSLI
        BDRR,R3 FNSLO
        LODI,R0 FETIMO
        RETC,UN
FNSLOK: EORZ    R0
        RETC,UN
```

The reply and RXLEN change only when a transaction ends or a slice is selected, so a program may read them as often as it likes, and print straight out of the window. With 288 bytes of RAM, it usually must.

THE SEQUENCE NUMBER

The next sequence number is always the cartridge's ACKSEQ plus one, with 255 followed by 1: zero is never used. Never keep a counter of your own. The console's RESET restarts your program but not the cartridge, which keeps ACKSEQ, so a counter that starts at 1 again would write a number the cartridge has already answered – and writing that number does nothing at all. `FNCOMMIT` reads ACKSEQ every time:

2650 ASSEMBLY (AS) · FUJILIB.INC

```
FNCOMMIT:
        LODA,R0 *FACKP
        ADDI,R0 1
        BCFR,EQ FNCM1
        LODI,R0 1               ; 255 wraps past the reserved 0
FNCM1:  STRZ    R2              ; R2 = SEQ to write = ACKSEQ to expect
        LODI,R1 FRSEQ
        BSTA,UN FNREGWR
        LODI,R3 0               ; outer x256
```

HOW LONG TO WAIT

The cartridge gives the network chip 5 seconds to answer a command, 60 for MOUNT_IMAGE and COPY_FILE, and 90 for a network channel's OPEN, STATUS, READ and CLOSE. When its time runs out, ERR says so (2, a timeout) and ACKSEQ is painted as usual.

`FNCOMMIT` waits for ACKSEQ for about six seconds – 65,536 polls of some 90 *µ*s each – and then returns `FETIMO` (`$FF`). That is enough for an ordinary command and not for a slow one. The handbook's `FNGO` keeps polling the *same* sequence number for another 90 seconds when `FNCOMMIT` gives up, then checks ERR and REPLY:

2650 ASSEMBLY (AS) · BOOKLIB.INC

```
FNGO:   BSTA,UN FNCOMMIT        ; R2 = the SEQ it wrote
        COMI,R0 FETIMO
        BCFR,EQ FNGO4           ; answered, or failed, in time
        LODI,R0 15
        STRA,R0 BKTRY
FNGO1:  LODI,R3 0               ; one sweep: 64K polls of ~90 us
FNGO2:  LODI,R1 0
FNGO3:  NOP
```

```
        NOP
        NOP
        NOP
        NOP
        NOP
        LODA,R0 *FACKP
        COMZ    R2
        BCTR,EQ FNGO5           ; the reply is in
        BDRR,R1 FNGO3
        BDRR,R3 FNGO2
        LODA,R0 BKTRY
        SUBI,R0 1
        STRA,R0 BKTRY
        BCFR,EQ FNGO1
        LODI,R0 FETIMO
        RETC,UN
FNGO5:  LODA,R0 *FERRP
FNGO4:  COMI,R0 0
        BCFR,EQ FNGO7           ; the cart's error: return it
        LODA,R0 *FREPP
        COMI,R0 FCACK
        BCTR,EQ FNGO6
        LODI,R0 FCNAK
        RETC,UN
FNGO6:  EORZ    R0
FNGO7:  RETC,UN
```

IMPORTANT: Never launch again after a timeout of your own. The transaction is still running in the cartridge; a new sequence number would start a second one behind it, and the reply to the first would arrive looking like the answer to the second. Wait longer than the cartridge does, and the question never comes up.

NOTE: In MAME the whole transaction runs inside the read that writes SEQ: by the next instruction ACKSEQ already matches. That is why a program that waits too little works perfectly in MAME. Count on the times above, not on what the emulator shows.

THE RECIPE

1. `FNCHECK` once at start: R0 = 0 if a FujiNet cartridge answers.
2. `FNBEGIN device, command, nparam` (`booklib.inc`): DATA_RST, then the three registers.
3. Stream the parameters, then the payload.
4. `FNGO`: it launches with ACKSEQ + 1, waits as long as the cartridge may, and checks the reply. R0 = 0, CC EQ, on an ACK; otherwise ERR, `FETIMO`, or `FCNAK` (`$15`) when the FujiNet said no.
5. The reply is at `$2B00`, RXLEN bytes; `FNSLICE` for any past the first 256.

Here is first contact (Section 10) in those five steps:

2650 ASSEMBLY (AS)

```
        BSTA,UN FNCHECK         ; 1. a FujiNet cartridge?
        BCFA,EQ NOCART
        FNBEGIN FNDFUJI, FCACFGX, 0 ; 2. GET_ADAPTERCONFIG_EXTENDED
        BSTA,UN FNGO            ; 3. nothing to stream; 4. go
        BCFA,EQ FAIL
        SETSTR  FNRDATA         ; 5. the SSID is at $2B00
        LODI,R2 $40-1
        BSTA,UN DPRINT
```

5. THE TEXT DISPLAY

The 2637 draws its picture from 208 bytes of the console's RAM, $1800–$18CF: 13 rows of 16 cells, one byte a cell, row 0 first. Write a byte there and it is on the screen in the next frame. Nothing about the picture has to be set up besides a colour and a character height; the 2637 is drawing it from the moment the console is switched on.

ONE BYTE A CELL

colour bits 7–6	glyph, bits 5–0

The low six bits pick one of 64 glyphs; the top two pick the colour. The glyphs are the 2637's own and they are **not ASCII**:

Glyph	Is
$00	a space
$01–$0F	graphics: diagonals, a solid block ($03), half blocks, edges and corners ($04–$0B, Section 16)
$10–$19	the digits 0–9, so $10 + a nibble is a hex digit for 0–9
$1A–$33	the letters A–Z
$34–$37	. , + $
$38–$3F	the eight user-defined characters, drawn from $1980–$19BF

There is no lowercase, and no / - _ : ! ? ( ) or quotes. fujidisp.inc fills the gap with the user-defined characters: it loads / - _ : into $38–$3B, and a cursor chevron, a password bullet and a more-pages mark into $3C–$3E. Its XLAT table turns ASCII into glyphs: lowercase folds to capitals, and anything with no glyph becomes .. hello, the cartridge's first program, paints the whole set:



HELLO: THE 64 GLYPHS



FUJITEST

FIG. 3
THE RUNTIME'S PROGRAMS, WITH 8 × 8 CHARACTERS

IMPORTANT: The user-defined characters are also the four sprites' images (Section 16). A program that uses sprite 0 cannot also print fujidisp's slash.

HEIGHT AND COLOUR

Two registers set the whole screen, $19F9 and $19F8; $19F9 is the one that matters for text:

7 height	6 pot mux	5–3 character colour	2–0 screen colour

– **Bit 7 set** draws 8 × 8 characters: 13 rows fill only the top half of the picture. That is what DINIT sets ($87), and why the runtime's screens above sit high. **Bit 7 clear** draws them 8 × 16, and the 13 rows

fill the screen, as CONFIG and every game do. The handbook's programs use `BKINIT`, which calls `DINIT` and then writes `$07`.

– **Bit 6** is the pot multiplexer (Section 9). A program that reads both axes of a disc rewrites `$19F9` every frame, so it keeps its base value apart and ORs the multiplexer in.
– **The colours** are 0 white, 1 yellow, 2 cyan, 3 green, 4 magenta, 5 red, 6 blue, 7 black.

A cell's colour is three bits: its own two colour bits on top, and bit 3 of `$19F9` (the low bit of the character colour) under them. With `$19F9` = `$07`, colour bits `00` are white, `01` cyan, `10` magenta and `11` blue:

2650 ASSEMBLY (AS)

```
        BSTA,UN DXLAT           ; R0: 'G' -> its glyph
        IORI,R0 $80             ; colour bits 10: magenta
        STRA,R0 SCREEN+$A5      ; row 10, column 5
```

CAUTION: A cell of exactly `$C0` – colour bits `11` and a space – is not a blue space. It switches the rest of its row to block graphics, and `$40` switches it back. `fujidisp.inc` never paints colour bits `11` for that reason; any glyph but the space is safe in blue. Section 16 shows the block graphics on purpose.

PRINTING

`DPRINT` prints the NUL-terminated string at `(DSTRP)`, and `SETSTR` points `DSTRP` at a label. The cell goes in R2, **minus one**: `DPRINT` stores with the 2650's auto-increment, `STRA,R0 SCREEN,R2,+`, which adds one to R2 first. It returns R2 on the last cell written, so a second call carries on where the first stopped.

2650 ASSEMBLY (AS) · FUJIDISP.INC

```
; Print the string at (DSTRP) from screen offset R2 + 1 (R2 = target cell minus one for
; auto-increment; returns the last cell written). No line wrap. Calls one down (DXLAT).
; Clobbers R0,R1,R2,R3.
DPRINT: LODI,R3 $FF
DPRT1:  LODA,R0 *DSTRP,R3,+    ; str[++R3]; a load sets CC
        BCTR,EQ DPRT2
        BSTR,UN DXLAT
        STRA,R0 SCREEN,R2,+    ; screen[++R2]
        BCTR,UN DPRT1
DPRT2:  RETC,UN
```

`DSTRP` is a pointer and the load through it is indirect, so the string can be anywhere: in block 1, in block 2, or in the reply window at `$2B00`. `fujitest` prints the SSID with `SETSTR FNRDATA` and never copies a byte of it. The handbook's `PUTS cell, string` macro is `SETSTR`, the minus one and the call; `DHEX` prints R0 as two hex digits after R2, and `DREPLY` prints a count of reply bytes that has no NUL.

2650 ASSEMBLY (AS)

```
        PUTS    $00, TITLE      ; row 0: the title
        LODA,R0 *FERRP          ; the last transaction's ERR
        LODI,R2 $C0-1           ; row 12
        BSTA,UN DHEX
```

`DCLS` clears all 208 cells to spaces. There is no scrolling in the 2637; a program that scrolls copies rows itself, as NetCat does (Section 12).

CHANGING THE SCREEN WITHOUT TEARING

The 2637 reads a row from RAM as it draws it, so a change in the middle of a frame can show half old and half new for one frame. For text that is rarely worth worrying about. For a screen that changes every frame, make the changes in the vertical blank (`TPSU $80`), as `pads` and `sprites` do.

6. LIVING IN 288 BYTES

The ARCADIA-2001 has no RAM chip of its own. Its working memory is the part of the 2637's RAM the picture does not use: 32 bytes at `$18D0`, four at `$18F8`, and, in the 13-row mode, the 256 bytes of the lower screen at `$1A00`. That is 292 bytes in all, and since the runtime's figure leaves out the four at `$18F8`, this book calls it 288 as the firmware does.

It is less tight than it sounds. The 2650 keeps its return addresses on the chip, so there is no stack to feed. The program is in ROM. And the reply window is 256 bytes the program can read without owning: a FujiNet program in 288 bytes copies almost nothing.

THE LEDGER

Address	Bytes	Who
`$18D0–$18DD`	14	free: the runtime takes none of it (CONFIG's own variables live here)
`$18DE–$18DF`	2	`fujiboot`'s two scratch bytes
`$18E0–$18EA`	11	the boot stub, copied here just before a swap (Section 13)
`$18EB–$18EC`	2	`FSTRP`: the string `FNTXSTR` and `FNPATH` send
`$18ED–$18EE`	2	`DSTRP`: the string `DPRINT` prints
`$18EF`	1	free
`$18F8–$18FB`	4	free
`$1A00–$1A07`	8	`booklib.inc` and `netlib.inc`: FNGO's count, the failure code, the STATUS reply, DREPLY's position and count
`$1A08–$1AFF`	248	the program's own, from `BKVAR`

The boot stub's eleven bytes are needed only for the moment of a swap, so any program that does not boot another can use them; `fujibank` keeps its frame counter there. The same goes for anything the book's libraries own: a program that never calls `FNGO` may take `$1A00`.

NOTE: The lower screen is RAM only while the screen has 13 rows. Turn on the 26-row mode (bit 6 of `$19F8`) and `$1A00–$1ACF` is on the screen: every variable there is drawn as a character. Only `$1AD0–$1AFF` stays yours.

WHAT A WHOLE PROGRAM TAKES

CONFIG is the measure: it uses every one of the 292 bytes. Its cursor, counters and scratch fill `$18D0–$18EA` around the runtime's two pointers; four flags take `$18F8–$18FB`; and the lower screen holds, laid over one another because no two are needed at once, the path being browsed, the host name and the WiFi name and password being edited, then the filter, the copy's source, a stack of parent folders and the editor's state, up to `$1AFC`.

NetCat (Section 12), by contrast, fits in the first 41 bytes of the lower screen: the libraries' eight, nine variables at `BKVAR`, and a 17-byte line buffer at `BKVAR+16`.

READING INSTEAD OF COPYING

The reply window stays as it is until the next transaction ends or a slice is selected, so:

– **print from it**: `SETSTR FNRDATA+140` and `DPRINT` puts an IP address on the screen with no copy at all;
– **send from it**: `FNTXSTR` reads its string through a pointer, so `SETFSTR FNRDATA` streams a name the last reply held straight into the next command's TX stream;

– **test it in place**: `LODA,R0 *FRDP,R3` reads reply byte R3 wherever the window is.

What cannot stay in the window is anything needed *after* the next transaction: RXLEN, a count to READ, a byte of state. Copy those first.

IMPORTANT: The reply that matters is always the last one. `netlib.inc`'s `NSTAT` copies STATUS's four bytes into `NAVL..NDST` before anything else runs, because the READ that follows paints over them.

WHAT RESET KEEPS

The console's RESET starts the 2650 again at `$0000`. Nothing clears the RAM, and RESET does not reach the cartridge, which has no reset line. So after a RESET:

– the RAM holds what it held – initialise every variable you rely on;
– the cartridge keeps its ACKSEQ, so the next sequence number is not 1 (Section 4);
– a protocol 2 image keeps its page and its cartridge RAM (Section 15).

Switching the console off restarts everything, the cartridge included.

7. THE 2650 LIBRARY

Every program in this book is built on two files from the cartridge's firmware, `fujilib.inc` (the mailbox) and `fujidisp.inc` (the screen and the controller), used exactly as the firmware ships them, and on three of the handbook's own: `bookdefs.inc` (the commands), `booklib.inc` (what every program shares) and `netlib.inc` (the network device).

A PROGRAM'S SHAPE

2650 ASSEMBLY (AS) · CLOCK.ASM

```
         CPU     2650

         ORG     $0000
         BCTA    UN,START        ; cartridge header: jump past the
         DB      $17             ; RETC,UN interrupt guard at $0003

         ORG     $0020
         INCLUDE "fujidisp.inc"
         INCLUDE "fujilib.inc"
         INCLUDE "bookdefs.inc"
         INCLUDE "booklib.inc"

START:   EORZ    R0
         LPSU
         LODI,R0 $02             ; COM=1: logical compares
         LPSL
         BSTA,UN BKINIT
         PUTS    $00, TITLE
         BSTA,UN FNCHECK
         BCTR,EQ GO
         LODI,R0 FENOCRT
         BCTA,UN FAIL
```

– **The header** is the first four bytes: `BCTA,UN START` at `$0000`, where the 2650 starts, and the `RETC,UN` guard at `$0003`.
– **The includes come first.** AS resolves a label used before it is defined, but not a macro, so every file with a macro in it (`SETSTR`, `SETFSTR`, `DWBE`, `FNBEGIN`, `PUTS`) must be included before the code that uses it. `fujidisp.inc` goes first, then `fujilib.inc`.
– **The status word.** `EORZ R0` then `LPSU` clears the upper status byte; `LPSL` with `$02` sets COM, so every compare is *logical* (unsigned): `$C0` is greater than `$40`. The libraries count on it.
– **The display** comes up with `BKINIT` (or the runtime's own `DINIT` and `DCLS`), then `FNCHECK` asks whether there is a FujiNet at all.

THE REGISTERS

Reg.	By convention
R0	the data: every indexed load and store moves R0, and every routine returns its result there
R1, R2	the arguments: a register number and a value, a cell, a count
R3	a counter or an index

A routine's comment says which registers it clobbers; assume the rest are clobbered too unless it says otherwise. The second register bank (R1–R3 with RS set) is never selected.

IMPORTANT: On the 2650 a *load* sets CC (EQ, GT or LT, by sign), and so does every arithmetic and logical instruction and every compare. A *store* does not. After a subroutine returns, CC is whatever its last such instruction left: the library routines end with R0 loaded, so `BCFA,EQ FAIL` right after `FNGO` is safe. A routine of your own may not be; follow it with `COMI,R0 0` when in doubt.

FUJILIB.INC: THE MAILBOX

Routine	Does	Clobbers
FNREGWR	register R1 = value R2	R0
FNSTART	DATA_RST, and DEVICE = $70: begin a Fuji command	R0–R2
FNPARB	append a one-byte parameter, value R2	R0, R1
FNTXSTR	append the string at (FSTRP), at most 255, no NUL	R0, R1, R3
FNPATH	append it padded with NULs to 256 bytes	R0, R1, R3
FNCOMMIT	SEQ = ACKSEQ + 1, wait about 6 s: R0 = ERR, or FETIMO	R1–R3
FNSLICE	show reply slice R2, wait for SLICE_ECHO	R0, R1, R3
FNCHECK	R0 = 0 when the F, N magic is there	R0

SETFSTR label points FSTRP at a string. At the end of the file are the pointer words page-0 code reaches the mailbox through – FSELP, FDATP and FTXP for the hotspots, FRDP for the reply, FACKP, FERRP, FREPP, FRXLP and the rest for the status page – each a DWBE, big-endian, as the 2650 reads an indirect address (Section 8).

FUJIDISP.INC: THE SCREEN AND THE CONTROLLER

Routine	Does	Clobbers
DINIT	13-row text, 8 × 8 characters, white on black, sound off; loads the eight UDCs	R0, R1, R3
DCLS	all 208 cells to spaces	R0, R3
DXLAT	ASCII in R0 to a glyph in R0	R0, R1
DPRINT	the string at (DSTRP) from cell R2 + 1	R0–R3
DHEX	R0 as two hex digits from cell R2 + 1	R0–R2
SCAN	in the vertical blank: the first control held, as a code (Section 9)	R0, R1
GETKEY	wait for a fresh press: its code	R0, R1

BOOKLIB.INC AND NETLIB.INC: THE HANDBOOK'S

Name	Does	Clobbers
FNBEGIN	macro: DATA_RST, then DEVICE, CMD and NPARAM	R0–R2
PUTS	macro: print a string from a cell	R0–R3
BKINIT	DINIT, then 8 × 16 characters, then DCLS	R0, R1, R3
FNPARW	append a two-byte parameter, R2 low, R3 high	R0, R1
FNTXN	append R2 bytes from (FSTRP)	R0–R3
FNGO	commit, wait up to 96 s, check ERR and REPLY: R0 = 0 on an ACK	R1–R3
WAITF	wait R3 frames	R3
DREPLY	print DRCNT reply bytes from DRPOS at cell R2 + 1	R0–R3
FAIL	branch here with a code: FAILED and the code on row 12	–
NOPEN	OPEN the devicespec at (FSTRP), mode R2	R0–R3
NSTAT	STATUS, its four bytes kept in NAVL..NDST	R0–R3
NREAD	READ R2 (low), R3 (high) bytes	R0–R3
NWRITE	WRITE R2 bytes from (FSTRP)	R0–R3
NCLOSE	CLOSE	R0–R3

EIGHT CALLS DEEP

The 2650 keeps its return addresses in an eight-entry stack on the chip. `BSTA` pushes one and `RETC` pops it; there is no RAM stack and no way to see how full this one is. A ninth call overwrites the oldest entry without a word, and the routine at the bottom of the chain then returns to the wrong place.

So the depth of every program is counted when it is built. The firmware's `checkdepth.py` builds a graph from the `BSTA`s, one node per label, and reports the deepest chain. That is a lower bound, for two reasons:

– **It does not follow falling through.** `FNCOMMIT` makes its `BSTA,UN FNREGWR` after its own label `FNCM1`; falling into `FNCM1` is not a call, so the graph loses the edge.
– **It does not follow branches.** A routine that ends with `BCTA,UN FNGO`, or a `BCFA,EQ FAIL` from inside a subroutine, runs the target at the depth the branch was taken; the graph counts the target from nothing.

The handbook's `tools/depthcheck.py` reads the program as AS does – includes in place, macros expanded – and gives each label its calls (one deeper), its branches (the same depth) and its fall-through (the same depth). It counts a chain as the firmware's script does, the top level included, so a depth of 8 is seven return addresses. For `fujiboot` the firmware's script says 3 and the stricter count says 4: `CHKACK` branches to `FAIL`, and `FAIL` calls `DPRINT`, which calls `DXLAT`. NetCat, the deepest program here, is 6. Battleship and 5 Card Stud, whose READMEs audit their deepest chains by hand because the firmware's script undercounts them, come out at the hand audits' figures. Both scripts run on every build; the stricter one decides.

A jump from inside a subroutine back to a top-level loop that never returns – CONFIG does it on every error – leaves return addresses on the stack. On the 2650 that is harmless: the stack is a ring of eight, and the old entries are overwritten. `depthcheck.py` notes each such jump rather than calling it recursion.

BUILDING

`listings/asm/build.sh` does for every program what the firmware's `build.sh` does for the cartridge's own, and its images are byte for byte the firmware's:

2650 ASSEMBLY (AS) · BUILD.SH

```
    (cd "$src" && "$ASL" -q -L -i "$r:$r/../runtime:$r/build/gen" \
        -o "$r/$obj.p" -olist "$r/$obj.lst" "$name.asm")
    pages=$(pages_of "$name")
    "$P2BIN" -q "$obj.p" "$obj.flat" \
        -r "\$0000-\$$(printf '%04X' $((0x3000 + pages * 0x800 - 1)))" -l 0
    if [ "$pages" -gt 0 ]; then
        python3 "$TOOLS/mkimage.py" --pages "$pages" "$obj.flat" "build/$name.bin"
    else
        python3 "$TOOLS/mkimage.py" "$obj.flat" "build/$name.bin"
    fi
    printf 'FUJI' | dd of="build/$name.bin" bs=1 seek=$((0x1CFC)) \
        conv=notrunc status=none
    python3 "$TOOLS/checkrom.py" "build/$name.bin"
```

1. `asl` assembles the program, from its own folder, with `-i` naming the runtime and the generated `bookcfg.inc` (the bench's URLs).
2. `p2bin` writes the flat `$0000-$2FFF`; `-l 0` fills the gaps with zeros.
3. `mkimage.py` folds the two blocks into the 8,192-byte image.
4. `dd` stamps the claim, `FUJI`, at image `0x1CFC`.
5. `checkrom.py`, `checkdepth.py` and `depthcheck.py` check the result.

8. PAGES, BLOCKS AND POINTERS

The 2650 divides its 32K into four *pages* of 8K, and most of its instructions cannot see past the page they are in. On the ARCADIA-2001 that rule meets the cartridge's two blocks, and between them they decide where every routine and every table goes.

WHAT REACHES WHAT

Addressing	Example	Reaches
absolute, a load, store or ALU instruction	`LODA,R0 $1800`	only the instruction's own 8K page: 13 address bits
absolute, a branch or call	`BCTA,UN $2000`	anywhere: 15 bits
relative	`BCTR,EQ LOOP`	64 bytes back to 63 ahead
indirect, `*`	`LODA,R0 *PTR`	anywhere: the 15-bit address in the two bytes at PTR, high byte first – but PTR itself must be in the instruction's page
indexed	`LODA,R0 TAB,R3`	the page, at TAB + R3; R0 moves the data
indirect indexed	`LODA,R0 *PTR,R3`	anywhere: the address at PTR, plus R3
zero page	`ZBSR *$10`	`$0000-$003F` and `$1FC0-$1FFF`, from any page

Block 1 (`$0000-$0FFF`) is in page 0 with the 2637 and the RAM (`$1800-$1AFF`). Block 2 (`$2000-$2FFF`) is in page 1 with the mailbox. So:

– **code in block 1** reaches the screen, the controllers and the RAM directly, and the mailbox only through a pointer;
– **code in block 2** reaches the mailbox directly, and the screen and the RAM only through a pointer.

AS knows the rule and stops a build that breaks it:

```
START:  LODA,R0 $2C00           ; in block 1
> > > pg.asm(6): error: target not on same page
```

POINTER WORDS

An indirect address is two bytes, *high byte first*. AS's `DW` writes words low byte first, the other way round, so every pointer the 2650 dereferences is made with `fujilib.inc`'s `DWBE` macro:

2650 ASSEMBLY (AS) · FUJILIB.INC

```
DWBE    MACRO   val             ; big-endian word: what indirection reads
        DB      ((val)>>8)&$FF, (val)&$FF
        ENDM
```

CAUTION: A pointer written with `DW` assembles without a word and reads the wrong address. `DW $2C00` stores `$00 $2C`, and the 2650 goes to `$002C`.

The runtime lives in block 1, so it keeps a pointer word in block 1 for every mailbox address it uses, and reaches each through it: `LODA,R0 *FACKP` reads ACKSEQ, and `LODA,R0 *FTXP,R1` reads `$2F00` + R1, appending R1 to the TX stream.

The pointer word itself is found the ordinary way, so it must be in the instruction's own page: code in block 2 cannot use `FACKP`, which is in block 1. Going the other way, code in block 2 needs pointers into page 0, kept in block 2. 5 Card Stud and Battleship put their network code in block 2, where `LODA,R0`

FNREPLY reads the reply byte directly, and reach their RAM variables through pointer words such as STRA,R0 *PAVA0.

ZERO-PAGE VECTORS

ZBSR and ZBRR are the 2650's short calls and branches: two bytes, relative not to the instruction but to address zero, so they reach the same 64 bytes from every page. With * they are indirect, through a pointer word in those bytes. CONFIG keeps 30 such words, from $0004 to $003F – the bytes between the header and the runtime that every other program leaves empty – and calls any routine, from either block, in two bytes:

```
ZMSTART: DWBE   MSTART          ; at $0004 on: one word a routine
...
        ZBSR    *ZMSTART        ; 2 bytes, from anywhere
```

Against a three-byte BSTA that saved CONFIG about 180 bytes, which is how the whole CONFIG fits the 6,912. depthcheck.py cannot follow an indirect call, and says so; a program built this way counts its depth by hand.

SPLITTING A PROGRAM BETWEEN THE BLOCKS

Block 1 has 4,096 bytes and block 2 has 2,816. A program that outgrows block 1 moves to block 2, in this order of cheapness:

1. **strings**, read through DSTRP (any page), if nothing reads them with a direct indexed load;
2. **tables**, reached through a pointer word with an indirect indexed load;
3. **whole modules** that talk to the mailbox more than to the screen: the network code, the boot code – which in block 2 can read the mailbox directly and get smaller as they move.

CONFIG splits that way: block 1 holds the screen, the RAM and the controller; block 2 the mailbox, WiFi, boot, the tables and the strings.

FIVE TRAPS

Every one of these cost an evening during the FujiNet ports.

EORZ works on R0. EORZ R1 sets R0 to R0 XOR R1; it does not clear R1. To clear any register but R0 use LODI,R1 0. (EORZ R0, which clears R0, is the one form that does what it looks like.)

Indexed means R0. LODA,R1 TAB,R2 will not assemble: in an indexed instruction the register field names the index, and the data is R0.

A store does not set CC. SUBI,R0 1 then STRA,R0 N then BCFR,EQ tests the subtraction, which is usually what you want – but not if you meant to test the stored byte loaded from elsewhere.

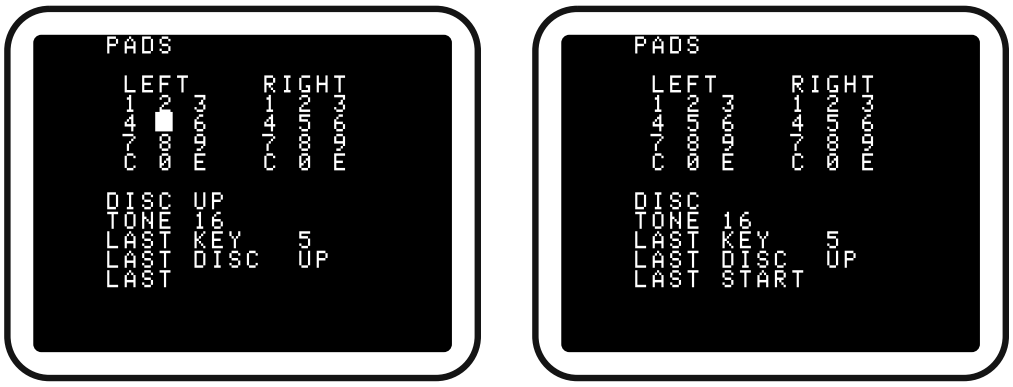
CC after a call is the callee's last. Battleship's random number routine ended on COMZ R1, so a BCTR,EQ after the call tested that compare, not the number; every ship was placed the same way round. Follow such a call with COMI,R0 0.

Read the branch twice. BCTA branches when CC *is* the condition, BCFA when it is *not*. While this book was written, NetGet's test for "the count is not zero" was a BCFA,EQ where a BCTA,EQ belonged, so it threw every good reading away and timed out with the file waiting:

```
        IORA,R0 NAVL            ; and not 0?
        BCTA,EQ AGAIN           ; zero: read STATUS again
```

9. HAND CONTROLLERS AND SOUND

Each of the ARCADIA-2001's two hand controllers has a 12-key keyboard, a control disc and an Action button on each side. The console has START, SELECT and OPTION (and RESET, which restarts the 2650 and cannot be read). The 2637 makes a tone or a noise. `pads` shows all of it on one screen.



5 AND THE DISC HELD — AFTER A KEY, THE DISC AND START

FIG. 4
PADS

THE KEYBOARDS

Each keyboard is read as three columns, one address each, active high, in the low four bits:

Left	Right	bit 3	bit 2	bit 1	bit 0
`$1900`	`$1904`	1	4	7	CLEAR
`$1901`	`$1905`	2, Action	5	8	0
`$1902`	`$1906`	3	6	9	ENTER

IMPORTANT: The Action buttons are the keypad's 2. Both are bit 3 of `$1901` – one wire – so no program can tell them apart. CONFIG and every FujiNet game treat that bit as "select", and never ask the player to type a 2 that means a 2.

`pads` reads all 24 keys from a table of four bytes a key: the column's offset from `$1900`, the key's bit, its cell on the screen, and its label. A held key is drawn as a solid block:

2650 ASSEMBLY (AS) · PADS.ASM

```
KEYS:   EORZ    R0
        STRA,R0 ANYK
        LODI,R3 0
KEY1:   LODA,R0 KTAB,R3         ; the column
        STRZ    R1
        LODA,R0 $1900,R1
        ANDA,R0 KTAB+1,R3       ; this key's bit
        STRZ    R2              ; R2: down or not
        LODA,R0 KTAB+2,R3       ; its cell
        STRZ    R1
        LODA,R0 KTAB+3,R3       ; its label
        COMI,R2 0
        BCTR,EQ KEY2            ; up: the label
        STRA,R0 LASTK           ; down: remember it, show a block
        STRA,R0 ANYK
        LODI,R0 GBLOCK
KEY2:   STRA,R0 SCREEN,R1
        ADDI,R3 4
        COMI,R3 24*4
        BCFR,EQ KEY1
        LODA,R0 LASTK           ; the last key, on row 10
        STRA,R0 SCREEN+$AB
        RETC,UN
```

THE CONSOLE'S BUTTONS

`$1908` holds START in bit 0, OPTION in bit 1 and SELECT in bit 2, active high. RESET is not there: it resets the 2650.

```
        LODA,R0 $1908
        TMI,R0  $01             ; START held?
        BCTA,EQ START1
```

`TMI` tests under a mask: CC is EQ when *every* bit of the mask is 1.

THE DISCS

A disc is not four switches. It is two potentiometers, one an axis, and the 2637 turns them into numbers – but only during the vertical blank. Read outside it, a pot gives `$FF`.

Address	Holds
`$19FF`	the left disc: vertical with bit 6 of `$19F9` clear, horizontal with it set
`$19FE`	the right disc, the same way

Up and left read near `$00`, down and right near `$FF`, centred somewhere in the middle (MAME gives `$6F` for the left disc). Anything under `$40` is one way and anything over `$C0` the other.

The multiplexer, bit 6 of `$19F9`, shares its register with the character height and both colours, so a program writes the whole register every time: its base value with the bit, or without it. It reads one axis a frame, keeps both, and decides from the kept pair – deciding from the axis just read would see "centred" on every other frame:

2650 ASSEMBLY (AS) · PADS.ASM

```
MAIN:   TPSU    $80             ; wait for the vertical blank
        BCFR,EQ MAIN
        LODA,R0 FRAME           ; the pots: valid only now
        ANDI,R0 1
        BCFR,EQ MHORZ
        LODA,R0 $19FF           ; mux clear: the vertical axes
        STRA,R0 DV1
        LODA,R0 $19FE
        STRA,R0 DV2
        LODI,R0 BGBASE|$40      ; the next frame reads horizontal
        STRA,R0 BGCOL
        BCTR,UN MKEYS
MHORZ:  LODA,R0 $19FF           ; mux set: the horizontal axes
        STRA,R0 DH1
        LODA,R0 $19FE
        STRA,R0 DH2
        LODI,R0 BGBASE          ; the next frame reads vertical
        STRA,R0 BGCOL
```

NOTE: Read outside the blank, the left disc's vertical axis is `$FF`, which looks like "down" held for ever. A menu that scrolls by itself is the symptom. `fujidisp.inc`'s `SCAN` waits for the blank before it reads.

SCAN AND GETKEY

For a menu, `fujidisp.inc` has the controller already decoded. `SCAN` waits for the blank and returns the first control it finds held, as a code; `GETKEY` waits for every control to be let go, then for a press, so a held key steps once:

Code		Held
`KUP`	1	the disc up
`KDOWN`	2	the disc down, or 8
`KPGUP`	3	4
`KPGDN`	4	6
`KSEL`	5	an Action button (or 2), or ENTER
`KBACK`	6	CLEAR

THE TONE

The 2637 has one tone generator and one noise generator, and two registers:

Register	Bits
`$18FD`	0–6: the pitch, *n*; bit 7 is the colour mode, keep it 0
`$18FE`	0–2: loudness; 3: the tone on; 4: the noise on; 5–7: the screen's scroll, keep them 0

The tone's frequency is 15,734 Hz ÷ (2 (*n* + 1)): *n* = 15 is about 490 Hz, *n* = 127 about 61 Hz. `pads` gives each key a pitch while it is held:

2650 ASSEMBLY (AS) · PADS.ASM

```
TONE:   LODA,R0 ANYK
        BCTR,EQ TONE1
        ANDI,R0 $1F             ; 0-9, C, E: higher keys, higher notes
        RRL,R0
        STRZ    R1
        LODI,R0 $40
        SUBZ    R1
        STRA,R0 PITCH
        LODI,R0 $0F             ; on, loudest
        BCTR,UN TONE2
TONE1:  EORZ    R0              ; off
TONE2:  STRA,R0 VOLUME
        LODA,R0 PITCH
        LODI,R2 $95-1           ; row 9: the pitch
        BSTA,UN DHEX
        RETC,UN
```

There are no envelopes and no second voice. A beep is a tone on for a few frames; CONFIG's key click is two frames.

10. FIRST CONTACT

`fujitest` is the cartridge's own first program, and the shortest that talks to the FujiNet: one command, GET_ADAPTERCONFIG_EXTENDED (`$C4`), and its answer on the screen. Burned to an ordinary 8K EPROM it is also the cartridge's first hardware test: there it shows NO FUJINET CART and the two bytes it found where the magic should be.



FUJITEST, AGAINST THE BENCH'S FUJINET-PC

FIG. 5

IS THERE A FUJINET?

2650 ASSEMBLY (AS) · FUJITEST.ASM

```
        BSTA,UN FNCHECK
        BCTR,EQ FOUND           ; R0=0: magic present

; ---- no cartridge magic: the EPROM path -------------------------------
        SETSTR  NCSTR
        LODI,R2 $20-1           ; row 2
        BSTA,UN DPRINT
        LODA,R0 *FMAGFP         ; the two bytes living where the magic
        LODI,R2 $30-1           ; should be, row 3
        BSTA,UN DHEX
        LODA,R0 *FMAGNP
        BSTA,UN DHEX
HALT1:  BCTA,UN HALT1
```

`FNCHECK` compares `$2C09` and `$2C0A` with `F` and `N`. On a FujiNet cartridge the game chip paints them at power-on; on an EPROM they are whatever the EPROM holds there – zero, for an image built by `mkimage.py`.

THE QUESTION

2650 ASSEMBLY (AS) · FUJITEST.ASM

```
FOUND:  BSTA,UN FNSTART
        LODI,R1 FRCMD
        LODI,R2 FCACFGX
        BSTA,UN FNREGWR
        LODI,R1 FRNPARM
        LODI,R2 0
        BSTA,UN FNREGWR
        BSTA,UN FNCOMMIT
        COMI,R0 0
        BCTR,EQ GOTCFG

        STRZ    R3              ; hold the error code
        SETSTR  ERSTR
        LODI,R2 $20-1           ; row 2: ERR nn
        BSTA,UN DPRINT
        LODZ    R3
        BSTA,UN DHEX
```

`FNSTART` rewinds the TX stream and sets DEVICE to `$70`; the two `FNREGWR` calls set CMD and NPARAM. There is nothing to stream. `FNCOMMIT` launches and waits, and R0 comes back 0. (`fujitest` checks ERR but not REPLY: this command is never refused. The handbook's `FNGO` checks both.)

THE ANSWER

GET_ADAPTERCONFIG_EXTENDED answers with 240 bytes, and the program reads three fields of it straight out of the window:

ssid	hostname	ip, gw, mask, dns	mac, bssid	version	IP text	gw, mask, dns text	mac, bssid text

at 0, 33, 97, 113, 125, 140, 156 and 204; every text field ends in a NUL

2650 ASSEMBLY (AS) · FUJITEST.ASM

```
GOTCFG: LODI,R2 0
        BSTA,UN FNSLICE

        SETSTR  FNRDATA         ; SSID: flows from row 4 into row 5
        LODI,R2 $40-1
        BSTA,UN DPRINT

        SETSTR  FNRDATA+125     ; row 7: firmware version
        LODI,R2 $70-1
        BSTA,UN DPRINT

        SETSTR  FNRDATA+140     ; row 9: IP (sLocalIP is dotted-decimal
        LODI,R2 $90-1           ; text)
        BSTA,UN DPRINT
IPDONE: BCTA,UN IPDONE
```

`FNSLICE` with R2 = 0 makes sure the window shows the first 256 bytes; all 240 are in it. `SETSTR FNRDATA+125` points `DPRINT` into the reply window, and the firmware version goes to the screen without one byte of it passing through the console's RAM.

The binary fields are there too: at 97, the IP address as four bytes. The text fields are easier to print, and every program in this book uses them.

NOTE: fujinet-pc is always "on WiFi", whatever the computer it runs on is connected to, and calls the network `Dummy Cafe`. The real cartridge reports the network it has joined.

RUN IT TWICE

Press the console's RESET and `fujitest` runs again, and its transaction gets a new sequence number: the cartridge kept the last one, and `FNCOMMIT` reads it every time. That is the reset rule of Section 4, and the bring-up's test for it: a program that kept its own counter would write 1 again, the cartridge would answer nothing, and the screen would never change.

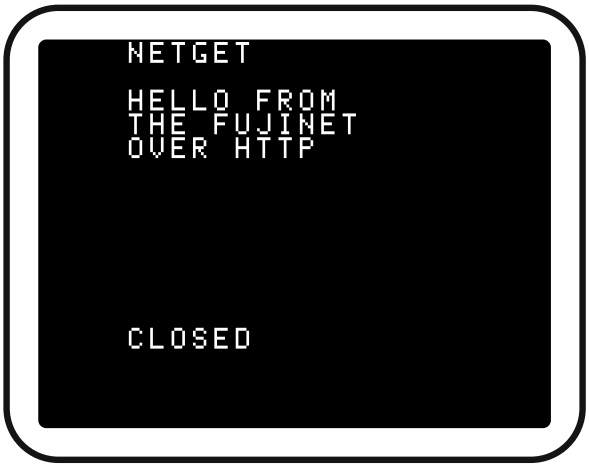
11. THE NETWORK DEVICE

The FujiNet has eight network channels, devices `$71` to `$78`. Each holds one connection: an HTTP request, a TCP socket, a file on a TNFS or FTP server, a UDP conversation. The devicespec says which, and the channel does the rest; an Arcadia program only opens, asks, reads, writes and closes.

THE LIFECYCLE

1. **OPEN** (`$4F`): two one-byte parameters, the mode and the translation, then the devicespec as the payload: `N:` and a URL.
2. **STATUS** (`$53`): two one-byte parameters of 0. Four bytes come back: how many bytes are waiting (two, low first), whether the connection is up, and the channel's status, 1 when all is well.
3. **READ** (`$52`): one two-byte parameter, the count. The bytes arrive in the reply window, RXLEN of them.
4. **WRITE** (`$57`): one two-byte parameter, the count, then that many bytes as the payload.
5. **CLOSE** (`$43`): no parameters.

The cartridge gives a network channel 90 seconds for each of OPEN, STATUS, READ and CLOSE, and `FNGO` waits that long (Section 4). `netlib.inc` wraps the five; each returns R0 = 0 on an ACK.



NETGET: THREE LINES OVER HTTP, FROM THE BENCH'S WEB SERVER

FIG. 6

OPENING

`netget` fetches a small text file over HTTP and shows it. The URL is the bench's; `build.sh` writes it into `build/gen/bookcfg.inc`, so the program carries no address of its own:

2650 ASSEMBLY (AS) · NETGET.ASM

```
OPEN:   SETFSTR URL
        LODI,R2 NMREAD
        BSTA,UN NOPEN
        BCFA,EQ FAIL
```

2650 ASSEMBLY (AS) · NETLIB.INC

```
NOPEN:  STRA,R2 NDST            ; the mode, until the parameters go
        FNBEGIN DVNET, NCOPEN, 2
        LODA,R2 NDST
        BSTA,UN FNPARB          ; mode
        LODI,R2 NTRNONE
        BSTA,UN FNPARB          ; translation
        BSTA,UN FNTXSTR         ; the devicespec: the payload
        BSTA,UN FNGO
        RETC,UN
```

Mode 4 is a read: over HTTP, a GET. Mode 12 is a read and a write, which is what a TCP connection wants (Section 12). Translation 0 leaves the bytes alone; 1, 2 and 3 turn the line endings into CR, LF or CR LF. The devicespec goes as the payload at its own length.

HOW MUCH IS THERE?

The adapter answers OPEN as soon as it can, and an HTTP request is not even sent until the first STATUS. STATUS says how many bytes are waiting *so far*, so `netget` asks until two answers agree and are not zero, three frames apart, thirty times at most:

2650 ASSEMBLY (AS) · NETGET.ASM

```
STAT1:  BSTA,UN NSTAT
        BCFA,EQ FAIL
        LODA,R0 NAVL            ; the same as the reading before?
        COMA,R0 PREVL
        BCFR,EQ AGAIN
        LODA,R0 NAVH
        COMA,R0 PREVH
        BCFR,EQ AGAIN
        IORA,R0 NAVL            ; and not 0?
        BCTA,EQ AGAIN
        BCTA,UN READ
AGAIN:  LODA,R0 NAVL            ; this reading is the one before
        STRA,R0 PREVL
        LODA,R0 NAVH
        STRA,R0 PREVH
        LODA,R0 TRIES
        ADDI,R0 1
        STRA,R0 TRIES
        COMI,R0 NTRIES
        BCFR,EQ WAIT
        LODI,R0 FETMOUT         ; it never settled
        BCTA,UN FAIL
```

NOTE: In MAME, unthrottled, those thirty readings take a few milliseconds of real time. Against a server across the internet they would run out before it answered; against the bench's own web server they do not. A program for a real console asks for longer, or watches the tick (Section 15) rather than counting readings.

READING

READ takes the bytes waiting, at most 192 here, a screen's worth. The window shows 256 at a time, and a reply can be 1,024 (Section 4):

2650 ASSEMBLY (AS) · NETGET.ASM

```
READ:   LODA,R2 NAVL
        LODA,R0 NAVH
        BCFR,EQ RDMAX           ; 256 or more
        LODZ    R2
        COMI,R0 NMAX
        BCTR,LT RDGO            ; under NMAX
RDMAX:  LODI,R2 NMAX
RDGO:   LODI,R3 0
        BSTA,UN NREAD
        BCFA,EQ FAIL
```

Then the program walks the reply: a CR is skipped, an LF starts the next row, anything else goes through `DXLAT` to the screen:

2650 ASSEMBLY (AS) · NETGET.ASM

```
SHOW1:  ADDI,R3 1
        LODZ    R3
        COMA,R0 *FRXLP          ; the reply's length (under 256 here)
        BCTA,EQ CLOSE
        LODA,R0 *FRDP,R3        ; reply[R3]
        COMI,R0 $0D
        BCTR,EQ SHOW1           ; CR: skip it
        COMI,R0 $0A
        BCTR,EQ SHOW2           ; LF: the next row
        BSTA,UN DXLAT           ; ASCII to a glyph (clobbers R1)
        LODA,R2 CELL
        STRA,R0 SCREEN,R2,+     ; screen[++R2]
        STRA,R2 CELL
        BCTR,UN SHOW1
```

CLOSING

2650 ASSEMBLY (AS) · NETGET.ASM

```
CLOSE:  BSTA,UN NCLOSE
        BCFA,EQ FAIL
        PUTS    $C0, DONE
HANG:   BCTR,UN HANG
```

Close every channel you open, on the way out of a failure too: the adapter keeps a connection open otherwise, even after the console is switched off. A program that opens the same channel again closes it first; 5 Card Stud puts the CLOSE of one request at the start of the next.

JSON

The adapter can parse what it fetched and answer one value at a time, so a program never holds a whole JSON document. Set the parser on the open channel, parse, then query by path; the value comes back through STATUS and READ like any other bytes:

2650 ASSEMBLY (AS)

```
        FNBEGIN DVNET, NCPARSR, 2 ; SET_PARSER: JSON
        LODI,R2 0
        BSTA,UN FNPARB          ; ignored, but it must be there
        LODI,R2 NPJSON
        BSTA,UN FNPARB          ; the parser
        BSTA,UN FNGO
        BCFA,EQ FAIL
        FNBEGIN DVNET, NCPARSE, 0 ; PARSE the whole reply
        BSTA,UN FNGO
        BCFA,EQ FAIL
        FNBEGIN DVNET, NCQUERY, 0 ; QUERY one value
        SETFSTR QPATH           ; "/0/name"
        BSTA,UN FNTXSTR
        BSTA,UN FNGO            ; then NSTAT and NREAD
        BCFA,EQ FAIL
```

IMPORTANT: Read the whole of each query's value before the next QUERY. A value left half read makes the next queries come back empty.

12 NETCAT

HOST SD **FILE /arcbook/netcat.bin**

I. GENERAL

NetCat is a terminal for a TCP connection: what the far end sends scrolls up the screen, and what you type goes back a line at a time. It is the handbook's longest program and the one that uses everything before it: the network device, the controller, the tone of the keys, and the RAM arithmetic of Section 6.

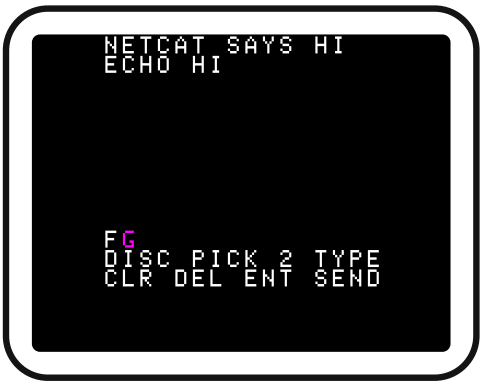
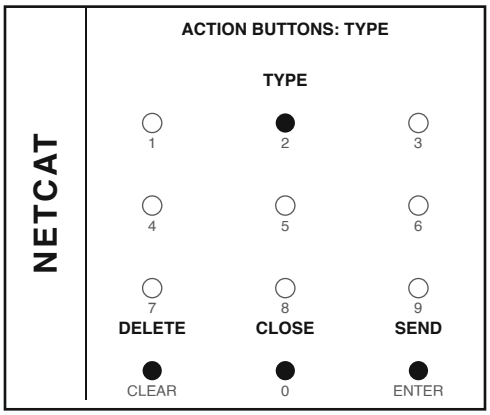


FIG. 7
NETCAT, A LINE SENT AND THE NEXT BEGUN

1. NetCat opens `N:TCP://` and the bench's line server, which greets it with **NETCAT SAYS HI**.
2. Turn the disc up or down: the character at the end of the line, in magenta, steps through the alphabet, the digits and `. , + $ / - _ :`. Hold it and it runs.
3. Press the Action button to type that character. CLEAR takes the last one back.
4. ENTER sends the line with CR LF. The server answers **ECHO** and the line.
5. 0 closes the connection: **CLOSED**.

II. THE KEYBOARD



The ARCADIA-2001 has twelve keys and no keyboard, and the 2 is the Action buttons' wire, so NetCat does not type from the keys at all. It takes its characters from a wheel, as an arcade machine takes a high-score name.

The wheel is a string in ROM, `WHEEL`, and its position a byte, `WIDX`. The disc is read once a frame, in the blank; a turn moves `WIDX` at once, then again every eighth frame from the sixteenth while the disc is held, so one press is one character and a held disc runs.

2650 ASSEMBLY (AS) · NETCAT.ASM

```
INPUT:  LODA,R0 $19FF           ; the disc, vertical (the mux is clear)
        COMI,R0 $40
        BCTR,LT INUP
        COMI,R0 $C0
        BCTR,GT INDN
        EORZ    R0              ; centred: the next turn is at once
        STRA,R0 HELD
        BCTA,UN INKEYS
INUP:   LODI,R1 WLEN-1          ; up: the character before
        BCTR,UN INTURN
INDN:   LODI,R1 1               ; down: the one after
INTURN: LODA,R0 HELD            ; at once; held, from the 16th frame
        ADDI,R0 1               ; on, every 8th
        STRA,R0 HELD
        COMI,R0 1
        BCTR,EQ INT1
        COMI,R0 16
        BCTA,LT INKEYS
        ANDI,R0 7
        BCFA,EQ INKEYS
INT1:   LODA,R0 WIDX            ; WIDX = (WIDX + R1) mod WLEN
        ADDZ    R1
        COMI,R0 WLEN
        BCTR,LT INT2
        SUBI,R0 WLEN
INT2:   STRA,R0 WIDX
        BSTA,UN LSHOW
```

The keys are taken on their edges: the program keeps last frame's keys in `PREVK`, and a key counts only in the frame it goes down. The four bits that matter are gathered into one byte first:

2650 ASSEMBLY (AS) · NETCAT.ASM

```
INKEYS: LODA,R0 $1901           ; b3: Action (and 2), b0: 0
        ANDI,R0 $09
        STRZ    R1
        LODA,R0 $1900           ; b0: CLEAR
        ANDI,R0 $01
        RRL,R0                  ; to b1
        IORZ    R1
        STRZ    R1
        LODA,R0 $1902           ; b0: ENTER
        ANDI,R0 $01
        RRL,R0
        RRL,R0                  ; to b2
        IORZ    R1
        STRZ    R1              ; R1: the keys now
        LODA,R0 PREVK
        EORI,R0 $FF
        ANDZ    R1              ; R0: the keys just pressed
        STRA,R1 PREVK
        TMI,R0  $08
        BCTA,EQ TYPE
        TMI,R0  $02
        BCTA,EQ DELETE
        TMI,R0  $04
        BCTA,EQ SEND
        TMI,R0  $01
        BCTA,EQ QUIT
        RETC,UN
```

III. THE CONNECTION

The devicespec is `N:TCP://127.0.0.1:8791/`, from `build.sh`, opened in mode 12, read and write. After that NetCat never waits on the network. Every sixth frame it asks STATUS; if bytes are waiting it READs them, up to 255, and prints them; then it goes back to the controller:

2650 ASSEMBLY (AS) · NETCAT.ASM

```
RECV:   BSTA,UN NSTAT
        BCFA,EQ FAIL
        LODA,R0 NAVH
        BCFR,EQ RECV1           ; 256 or more: take 255
        LODA,R2 NAVL
        BCTR,EQ RECV9           ; nothing
        BCTR,UN RECV2
RECV1:  LODI,R2 255
RECV2:  LODI,R3 0
        BSTA,UN NREAD
        BCFA,EQ FAIL
        EORZ    R0
        STRA,R0 RIDX
RECV3:  LODA,R0 RIDX
        COMA,R0 *FRXLP          ; the reply's length (under 256)
        BCTR,EQ RECV9
        STRZ    R3
        LODA,R0 *FRDP,R3        ; reply[RIDX]
        BSTA,UN TPUTC
        LODA,R0 RIDX
        ADDI,R0 1
        STRA,R0 RIDX
        BCTR,UN RECV3
RECV9:  RETC,UN
```

A line goes out with one WRITE: the characters, then CR and LF, from the line buffer in RAM through FSTRP:

2650 ASSEMBLY (AS) · NETCAT.ASM

```
SEND:   LODA,R1 LLEN            ; the line, CR, LF
        LODI,R0 $0D
        STRA,R0 LBUF,R1
        ADDI,R1 1
        LODI,R0 $0A
        STRA,R0 LBUF,R1
        ADDI,R1 1
        LODZ    R1              ; R2: the count
        STRZ    R2
        SETFSTR LBUF
        BSTA,UN NWRITE
        BCFA,EQ FAIL
        EORZ    R0
        STRA,R0 LLEN
        BSTA,UN LSHOW
        RETC,UN
```

IV. THE TERMINAL

Rows 0–9 are the terminal, with a cursor of its own (TROW, TCOL). TPUTC ignores CR, starts a new line at LF, and wraps at the sixteenth column. Off the bottom, TSCRL copies rows 1–9 up a row and clears row 9: the 2637 cannot scroll, but the screen is RAM, and the copy takes under a frame.

2650 ASSEMBLY (AS) · NETCAT.ASM

```
TSCRL:  LODI,R3 0
TSC1:   LODA,R0 SCREEN+16,R3
        STRA,R0 SCREEN,R3
        ADDI,R3 1
        COMI,R3 16*(TROWS-1)
        BCFR,EQ TSC1
TSC2:   EORZ    R0
        STRA,R0 SCREEN,R3
        ADDI,R3 1
        COMI,R3 16*TROWS
        BCFR,EQ TSC2
        RETC,UN
```

V. LIVING IN RAM

Where	Bytes	What
$1A00–$1A07	8	the libraries: FNGO, FAIL, the STATUS reply, DREPLY
$1A08–$1A10	9	TROW, TCOL, TCH, WIDX, LLEN, HELD, PREVK, POLL, RIDX
$1A18–$1A28	17	LBUF: 15 characters, CR, LF

The received bytes are never copied: RECV prints each one straight out of the reply window, keeping only its index, RIDX.

VI. TESTING IT

emu/verify.sh drives NetCat as a player would: it waits for the greeting, turns the disc down seven times to H, types, turns once more to I, types, presses ENTER and waits for **ECHO HI**. The line server writes what it heard to bench/run/ncserver.log.txt.

IMPORTANT: NetCat must be tested in real time (THROTTLE=1). The server waits half a second before its greeting, and an unthrottled MAME runs a minute of console time in that half second; the test would give up before the greeting came.

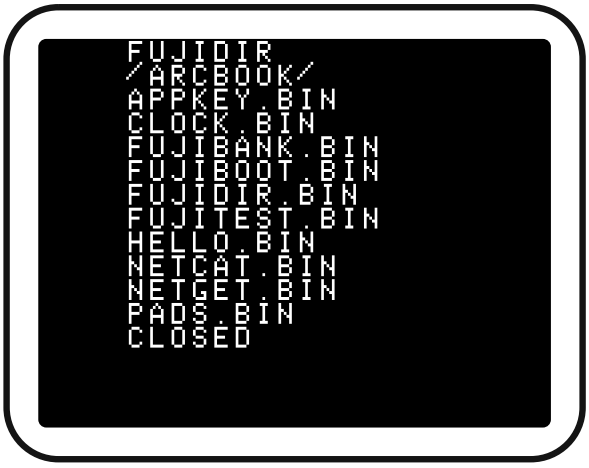
NetCat has no way back to CONFIG: no ARCADIA-2001 program does. 0 closes the connection, and the cartridge's RESET, then the console's, bring CONFIG back.

13. DIRECTORIES, MOUNT AND BOOT

A FujiNet host is a place files live: the memory card, a TNFS server, an HTTP or SMB share. The adapter holds eight of them in *host slots*, numbered from 0, and the commands below take a slot number. This section lists a directory, then does what CONFIG does when you choose a game: mounts it, streams it into the cartridge, and hands the console over.

LISTING A DIRECTORY

1. **MOUNT_HOST** (`$F9`), one byte: the host slot. For the memory card this is a formality; for a TNFS server it makes the connection.
2. **OPEN_DIRECTORY** (`$F7`), one byte, the slot; then the path as the payload, padded with NULs to exactly **256 bytes**. A shorter payload is refused. A second string after the path's NUL, inside the 256, is a filter such as `*.BIN`.
3. **READ_DIR_ENTRY** (`$F6`), two bytes: the longest name to return, and 0. The reply is the name, NUL-terminated, with a `/` on the end of a folder. After the last entry the reply is two bytes of `$7F`.
4. **CLOSE_DIRECTORY** (`$F5`), no parameters.



FUJIDIR: THE FIRST TEN OF THE BOOK'S OWN PROGRAMS

FIG. 8

2650 ASSEMBLY (AS) · FUJIDIR.ASM

```
NEXT:   FNBEGIN FNDFUJI, FCRDIR, 2
        LODI,R2 MAXLEN
        BSTA,UN FNPARB          ; the longest name we take
        LODI,R2 0
        BSTA,UN FNPARB          ; no extra fields (no icons)
        BSTA,UN FNGO
        BCFA,EQ FAIL
        LODA,R0 *FRDP           ; $7F,$7F: the end
        COMI,R0 $7F
        BCFR,EQ SHOW
        LODI,R1 1
        LODA,R0 *FRDP,R1
        COMI,R0 $7F
        BCTA,EQ DONE
```

IMPORTANT: Ask for 36 characters at least. The adapter adds a folder's `/` only when it fits within the length asked for, so a short maximum can turn a long folder name into what looks like a file.

SET_DIRECTORY_POSITION ($E4) moves to an entry by number, which is how CONFIG pages through a long folder without reading every entry each time.

MOUNTING AN IMAGE

The boot is three commands and a wait:

1. **MOUNT_HOST** ($F9): the slot the file is on.
2. **SET_DEVICE_FULLPATH** ($E2): three bytes – the device slot (0), the host slot and the mode (1, read) – then the path, padded to 256 bytes.
3. **MOUNT_IMAGE** ($F8): two bytes, the device slot and the mode. Before it answers, the network chip streams the image to the cartridge over the USB link's DBC device, and the game chip stages it in the half of its memory it is not serving from.
4. Wait until BOOT_STATE ($2C06) says READY.

2650 ASSEMBLY (AS) · FUJIBOOT.ASM

```
; ---- SET_DEVICE_FULLPATH(dev, host, mode, path) ------------------------
        BSTA,UN FNSTART
        LODI,R1 FRCMD
        LODI,R2 FCSDFP
        BSTA,UN FNREGWR
        LODI,R1 FRNPARM
        LODI,R2 3
        BSTA,UN FNREGWR
        LODI,R2 DEVSLOT
        BSTA,UN FNPARB
        LODI,R2 BOOTHST
        BSTA,UN FNPARB
        LODI,R2 FMREAD
        BSTA,UN FNPARB
        SETFSTR BOOTPTH
        BSTA,UN FNPATH          ; path, NUL-padded to 256
        BSTA,UN FNCOMMIT
        LODI,R1 2               ; err class 2 = path
        BSTA,UN CHKACK
```

Value	BOOT_STATE	Means
0	IDLE	nothing staged
1	XFER	the image is coming: BOOT_PCT, $2C07, says how much
2	READY	staged and checked: the swap may be armed
$80	FAILED	BOOT_ERR, $2C08, says why: 1 too big, 2 cut short, 3 not a size the cartridge can map, 4 the store busy

The image may be up to 8,192 bytes: a smaller one (a 2K or 4K game) is padded with $FF. A bigger one must have the shape of protocol 2, exactly 8,192 + k × 2,048 bytes and the claim (Section 15); anything else fails with BOOT_ERR 3.

2650 ASSEMBLY (AS) · FUJIBOOT.ASM

```
; ---- wait for the staged image (progress bar on row 3) -----------------
WAITRD: LODA,R0 *FBSTP
        COMI,R0 FBREADY
        BCTR,EQ READY
        LODA,R0 *FBSTP
        COMI,R0 FBFAIL
        BCFR,LT WSHOW           ; < FBFAIL: still working
        LODI,R1 4               ; stage failure
        LODA,R0 *FBERP
        STRZ    R2
        BCTA,UN FAIL
WSHOW:  LODA,R0 *FBPCP          ; 0-100 -> a 10-cell bar on row 3
        BSTA,UN BAR
        BCTR,UN WAITRD
```

NOTE: In MAME the image arrives all at once, inside MOUNT_IMAGE, and the progress bar jumps from 0 to 100. On the cartridge it fills as the bytes come.

THE SWAP

The console cannot be told to “jump to the new cartridge”: as far as it knows the cartridge never changes. So the program arms the cartridge, leaves the cartridge’s address space altogether, and from RAM reads the one address that makes the game chip change images:

1. Write BOOTLOCK (`$11`) = `$B5`. Only a READY image can be armed.
2. Copy the 11-byte stub to RAM at `$18E0` and jump to it.
3. The stub clears the status word, reads `$2DFE` through a pointer at `$18E9`, and the game chip flips to the staged image before the next read.
4. The stub jumps to `$0000`, the new image’s header.

2650 ASSEMBLY (AS) · FUJIBOOT.ASM

```
READY:  SETSTR  BOOTG
        LODI,R2 $30-1
        BSTA,UN DPRINT
        LODI,R1 FRBOOTL         ; arm the swap
        LODI,R2 FNBLMAG
        BSTA,UN FNREGWR
        LODI,R3 STUBLEN-1       ; copy the stub into RAM
STCPY:  LODA,R0 STUBSRC,R3
        STRA,R0 STUB,R3
        BDRR,R3 STCPY
        LODA,R0 STUBSRC
        STRA,R0 STUB
        BCTA,UN STUB

; Run from $18E0, so the FNSWAP pointer must land at STBPTA ($18E9): 9 bytes precede it.
STUBSRC:
        EORZ    R0              ; +0
        LPSU                    ; +1  PSU = 0
        LPSL                    ; +2  PSL = 0 (power-on-like for the image)
        LODA,R0 *STBPTA         ; +3  read $2DFE indirect -> swap
        BCTA,UN $0000           ; +6  the new image's header
        DWBE    FNSWAP          ; +9  = $18E9 at runtime
STUBLEN EQU     $-STUBSRC
```

The stub’s bytes are `20 92 93 0C 98 E9 1F 00 00 2D FE`. It must run from RAM: the instruction that reads `$2DFE` would otherwise be fetched from the very image it replaces.

AFTER THE SWAP

What the new image finds depends on its claim (Section 3):

– **An image with the claim** – a FujiNet program – finds the mailbox alive, and the status page and reply window as the program before left them: the same ACKSEQ, F and N, the version. It can begin a transaction at once.
– **An image without it** – any of the console's own games – kills the mailbox for the rest of the session. The hotspots become plain ROM addresses, so the game can read anything it likes.

To leave either and come back to CONFIG, press the cartridge's RESET (it restarts both of its chips, and the game chip serves CONFIG again), then the console's.

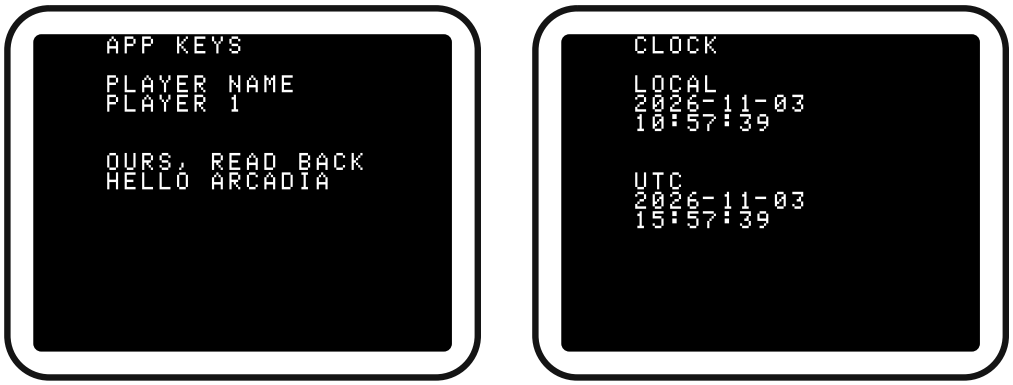


FUJIBOOT, AFTER BOOTING HELLO.BIN

FIG. 9
THE SWAP: FUJIBOOT'S SCREEN IS REPLACED BY THE PROGRAM IT BOOTED

14. APP KEYS, THE CLOCK AND UTILITIES

An ARCADIA-2001 forgets everything when it is switched off. The FujiNet does not: it keeps small *app keys* on its memory card for any program that asks, and it knows the time.



APPKEY CLOCK

FIG. 10
THE PLAYER'S NAME, A KEY OF OUR OWN, AND THE TIME TWICE OVER

APP KEYS

An app key is up to 64 bytes, named by three numbers: a *creator*, two bytes, an *app* and a *key* number. On the memory card it is the file `/FujiNet/CCCCAAKK.key`. Creator 1, app 1, key 0 is special: it holds the player's name, which the FujiNet games on most machines read, so a name typed once is known everywhere. (The Arcadia's Battleship and 5 Card Stud do not yet: they ask for a name each time, and offer ARCADIA.) `appkey` reads that name, then writes a key of the handbook's own – creator `$2650`, app `$FF`, key 0 – and reads it back:

2650 ASSEMBLY (AS) · APPKEY.ASM

```
GO:     PUTS    $20, TNAME      ; the player's name
        SETFSTR KNAME
        BSTA,UN AKOPEN
        BSTA,UN AKREAD
        LODI,R2 $30-1
        BSTA,UN AKSHOW
        PUTS    $60, TOURS      ; ours: write it...
        SETFSTR KOURW
        BSTA,UN AKOPEN
        BSTA,UN AKWRIT
        SETFSTR KOURR           ; ...and read it back
        BSTA,UN AKOPEN
        BSTA,UN AKREAD
        LODI,R2 $70-1
        BSTA,UN AKSHOW
HANG:   BCTR,UN HANG
```

OPENING A KEY

OPEN_APPKEY chooses the key the next read or write uses. It has no parameters: the key goes as a six-byte payload, the creator's low byte first, then its high byte, the app, the key, the mode (0 read, 1 write) and a reserved 0. `FNTXN` streams the six bytes as they are, zeros included:

2650 ASSEMBLY (AS) · APPKEY.ASM

```
AKOPEN: FNBEGIN FNDFUJI, FCAKOP, 0
        LODI,R2 6
        BSTA,UN FNTXN
        BSTA,UN FNGO
        BCFA,EQ FAIL
        RETC,UN
```

2650 ASSEMBLY (AS) · APPKEY.ASM

```
KNAME:  DB      AKCREAT&$FF, AKCREAT>>8, AKAPP, AKKEY, AKMREAD, 0
KOURW:  DB      OURCRE&$FF, OURCRE>>8, OURAPP, OURKEY, AKMWRIT, 0
KOURR:  DB      OURCRE&$FF, OURCRE>>8, OURAPP, OURKEY, AKMREAD, 0
```

IMPORTANT: Send the reserved byte. Without it the adapter waits for a sixth byte that never comes, until the transaction times out.

READING AND WRITING

READ_APPKEY answers with a two-byte length and then the value; `AKSHOW` prints the value from byte 2 of the reply window. WRITE_APPKEY takes the length as one parameter and the bytes as the payload. A write ends the selection: open the key again before the next read.

2650 ASSEMBLY (AS) · APPKEY.ASM

```
AKWRIT: FNBEGIN FNDFUJI, FCAKWR, 1
        LODI,R2 VALLEN
        BSTA,UN FNPARB
        SETFSTR VALUE
        LODI,R2 VALLEN
        BSTA,UN FNTXN
        BSTA,UN FNGO
        BCFA,EQ FAIL
        RETC,UN
```

THE CLOCK

The FujiNet's clock is a device of its own, `$45`. GET_ISO_LOCAL (`$49`, an `I`) answers with the local time as text, `2026-11-03T10:45:31` and the zone's offset; GET_ISO_UTC (`$5A`, a `Z`) the same in UTC. `clock` prints the date and the time on two rows, straight out of the reply window, with `DREPLY`:

2650 ASSEMBLY (AS) · CLOCK.ASM

```
ASK:    STRA,R1 BKVAR           ; hold them across the transaction
        STRA,R2 BKVAR+1
        LODI,R1 FRDATRS
        LODI,R2 0
        BSTA,UN FNREGWR
        LODI,R1 FRDEVIC
        LODI,R2 DVCLOCK
        BSTA,UN FNREGWR
        LODI,R1 FRCMD
        LODA,R2 BKVAR
        BSTA,UN FNREGWR
        LODI,R1 FRNPARM
        LODI,R2 0
        BSTA,UN FNREGWR
        BSTA,UN FNGO
        BCFA,EQ FAIL
        EORZ    R0              ; YYYY-MM-DD: reply[0..9]
        STRA,R0 DRPOS
        LODI,R0 10
        STRA,R0 DRCNT
        LODA,R2 BKVAR+1
        BSTA,UN DREPLY
        LODI,R0 11              ; HH:MM:SS: reply[11..18]
        STRA,R0 DRPOS
```

```
        LODI,R0 8
        STRA,R0 DRCNT
        LODA,R2 BKVAR+1
        ADDI,R2 $10
        BSTA,UN DREPLY
        RETC,UN
```

The clock command is in R1 at run time, so `ASK` cannot use the `FNBEGIN` macro, whose arguments are fixed when the program is assembled; it makes the four register writes itself. A fujinet-pc knows no zone until it is given one, and its local time is UTC, so `clock` sets one first, with command `$74` (a `t`) and a POSIX zone as the payload:

2650 ASSEMBLY (AS) · CLOCK.ASM

```
GO:     FNBEGIN DVCLOCK, CKSETTA, 0
        SETFSTR TZONE           ; the zone: the payload
        BSTA,UN FNTXSTR
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

NOTE: `$99`, which the firmware calls SETTZ, sets the *alternate* zone: the one a clock read uses when it is given a parameter of 1. `$74` sets the adapter's own, and the adapter saves it.

GETTIME (`$93`) answers in binary instead: six bytes – day, month, year (two digits), hour, minute, second.

UTILITIES

The Fuji device has a few services no ARCADIA-2001 could afford to do itself:

Command	Gives
GENERATE_GUID `$BB`	a fresh unique id, 36 characters
HASH `$C8-$C2`	MD5, SHA-1, SHA-256 or SHA-512 of the bytes you feed it
BASE64 `$D0-$C9`	base64, either way

RANDOM_NUMBER (`$D3`) is in the firmware's list but not answered on this build; a program's randomness comes from a GUID or a hash. A hash is three transactions: feed the bytes with HASH_INPUT, as often as needed; choose the algorithm with HASH_COMPUTE; read the result with HASH_OUTPUT, as text or in binary.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCHSHI, 1 ; HASH_INPUT: feed it
        LODI,R2 5
        LODI,R3 0
        BSTA,UN FNPARW          ; five bytes
        SETFSTR HDATA           ; "HELLO"
        BSTA,UN FNTXSTR
        BSTA,UN FNGO
        BCFA,EQ FAIL
        FNBEGIN FNDFUJI, FCHSHC, 1 ; HASH_COMPUTE: SHA-256
        LODI,R2 HASH256
        BSTA,UN FNPARB
        BSTA,UN FNGO
        BCFA,EQ FAIL
        FNBEGIN FNDFUJI, FCHSHO, 1 ; HASH_OUTPUT: as hex text
        LODI,R2 1
        BSTA,UN FNPARB
        BSTA,UN FNGO            ; 64 characters at $2B00
        BCFA,EQ FAIL
```

Section 17 has all of them.

15. BIGGER THAN 8K: PROTOCOL 2

The connector never shows the console more than 8K, and a program's share of that is 6,912 bytes. M.U.L.E. needed about 48K. So the mailbox has a second version, protocol 2 (`$2C0B` reads 2), with three things a big program needs: *pages* of code seen through a window, a kilobyte of *cartridge RAM*, and a *tick* that counts the frames. On an ordinary 8K image all three are inert, so nothing in Sections 2 to 14 changes.

THE APPBANK IMAGE

A protocol 2 image is exactly 8,192 + *k* × 2,048 bytes, *k* from 1 to 31 – at most 71,680 – and it carries the claim. Its first 8K are laid out as an ordinary image; the 2K pages after them are seen, one at a time, through the *window* at `$0800-$0FFF`:

Image	Console	Is
`0x0000-0x05FF`	`$0000-$05FF`	fixed: always there
`0x0600-0x07FF`	`$0600-$07FF`	cartridge RAM, the low half (RAM-L)
`0x0800-0x0FFF`	`$0800-$0FFF`	the window; page 0 lives here
`0x1000-0x18FF`	`$2000-$28FF`	fixed
`0x1900-0x1AFF`	`$2900-$2AFF`	cartridge RAM, the high half (RAM-H)
`0x1B00-0x1FFF`	`$2B00-$2FFF`	the mailbox, as on any image
`0x2000` + (*n*–1) × `0x800`	`$0800-$0FFF`	page *n*, when selected

So the fixed code is 1,536 bytes in block 1 and 2,304 in block 2, and every page adds 2,048. M.U.L.E. is 51,200 bytes: the base and 21 pages.

SELECTING A PAGE

One read of `$2D80` + *n* selects page *n* (*n* under 32). The game chip decodes it where it serves the bus, so the very next fetch already sees the new page: the read must be made from fixed code, never from the window it changes. A page that does not exist reads `$FF`.

```
        LODI,R1 2               ; page 2
        LODA,R0 *FBNKP,R1       ; FBNKP: DWBE FNBANK ($2D80)
        BSTA,UN $0800           ; call into it
```

The swap of Section 13 maps page 0. A console RESET changes neither the page nor the cartridge RAM, so a program's start code selects page 0 itself before it calls into the window, as `fujibank` does in its first lines.

CARTRIDGE RAM

The 1K of cartridge RAM is the image's own bytes at `$0600-$07FF` and `$2900-$2AFF`, made writable. **Reading** it is reading ROM: code in block 1 reads `$0600` directly, code in block 2 reads `$2900`. **Writing** it is, as always, reading:

1. Set the pointer: REGSEL `$14`–`$17`, then REGDATA *v*. The pointer is `0x000`, `0x100`, `0x200` or `0x300` plus *v*, in the RAM's own 10-bit space: `0x000-0x1FF` is RAM-L, `0x200-0x3FF` is RAM-H.
2. REGSEL `$18`, POKE, then one REGDATA for each byte: each stores its value at the pointer and moves it on, wrapping at `0x400`.
3. End the burst with a bare REGSEL `$14`.

So *n* bytes cost *n* + 4 reads. These reads are taken by the game chip's bus code and never reach the mailbox proper, whose REGSEL is good for one REGDATA. The closing REGSEL matters: after it, a stray REGDATA can move the pointer but never store.

2650 ASSEMBLY (AS) · FUJIBANK.ASM

```
; ---- 3. RAM-L and RAM-H ------------------------------------------------
        LODI,R1 FRRAMP0         ; pointer = 0x000 ($0600)
        LODI,R2 0
        BSTA,UN FNREGWR
        LODI,R1 FRRAMPK         ; stream A1 A2 A3 A4
        LODA,R0 *FSELP,R1
        LODI,R1 $A1
RAML1:  LODA,R0 *FDATP,R1
        ADDI,R1 1
        COMI,R1 $A5
        BCFR,EQ RAML1
        LODI,R1 FRRAMP3         ; pointer = 0x3FC ($2AFC)
        LODI,R2 $FC
        BSTA,UN FNREGWR
        LODI,R1 FRRAMPK
        LODA,R0 *FSELP,R1
        LODI,R1 $B1
        LODA,R0 *FDATP,R1
        LODI,R1 $B2
        LODA,R0 *FDATP,R1
        LODI,R1 FRRAMP0         ; close the burst
        LODA,R0 *FSELP,R1
```

CAUTION: Never *store* into the cartridge's space, cartridge RAM included. The connector has no read/write line, so the cartridge drives the data lines whenever A12 is low, and a STRA into $0000–$0FFF or $2000–$2FFF makes the 2650 and the game chip drive them at once: a bus fight. MAME ignores the store; a real console may not survive it.

THE TICK

$2C0E (low) and $2C0F (high) count 60 Hz frames from the cartridge's power-up and keep counting through transactions: a clock the console never had. Read the high byte, the low, then the high again, and try again if the two highs differ:

2650 ASSEMBLY (AS) · FUJIBANK.ASM

```
; Read the tick tear-free: R1 = lo, R2 = hi. Leaf. Clobbers R0.
TICKRD: LODA,R0 *FTKHP
        STRZ    R2
        LODA,R0 *FTKLP
        STRZ    R1
        LODA,R0 *FTKHP
        COMZ    R2
        BCFR,EQ TICKRD
        RETC,UN
```

NOTE: In MAME the tick is emulated time: it stands still across a transaction, which runs inside one read, and runs fast in an unthrottled test. On the cartridge it is the game chip's own timer.

FUJIBANK

fujibank is the cartridge's own test of all of this: an 8K base and three pages. It reads the id byte at $0FFF after selecting each of pages 0 to 4 (4 does not exist), calls into page 2, pokes four bytes into RAM-L and two into RAM-H and reads them back, checks that strays after the closing REGSEL stored nothing, and times the tick over 60 frames. The firmware's emu/bankprobe.lua reads the results from RAM and judges them; it is one of emu/verify.sh's eleven.

The pages are assembled at `$3000` and up in the flat, each with `PHASE $0800` so its labels have the window's addresses, and `mkimage.py --pages 3` appends them to the folded 8K:

2650 ASSEMBLY (AS) · FUJIBANK.ASM

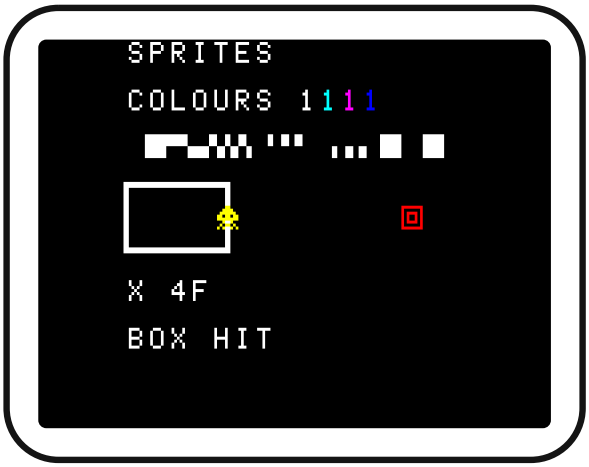
```
BANKPG  MACRO   n
        ORG     $3000+((n)-1)*$800
        PHASE   $0800
        LODI,R0 $C0+(n)
        RETC,UN
        DS      $0FFF-$
        DB      n
        DEPHASE
        ENDM
```

M.U.L.E.

The FujiNet M.U.L.E. client for the Arcadia is the reason protocol 2 exists, and the measure of it. It runs its game screens in the 26-row mode, where the console has only 84 bytes of RAM left, so its state lives in the cartridge RAM: the 550-byte game state from the server in RAM-L, names and the server's address in RAM-H. A small fixed layer, in the 1,536 and 2,304 fixed bytes, holds what every page needs; the boot, the network core and each screen – the map, the town, the auction – are pages of their own, called through the window.

16. THE 2637 UP CLOSE

The FujiNet programs keep the 2637 simple: 13 rows of white text. The chip can do more, and the games use it: colour, block graphics, a 26-row screen, eight characters of your own and four sprites that know when they touch something. `sprites` shows most of it on one screen.



SPRITES: THE COLOUR BITS, A BLOCK-GRAPHICS ROW, A BOX OF BUILT-IN GLYPHS, AND TWO SPRITES, MET

FIG. 11

COLOUR

The 2637 has eight colours: 0 white, 1 yellow, 2 cyan, 3 green, 4 magenta, 5 red, 6 blue, 7 black. `$19F9` holds the screen colour (bits 0–2) and the character colour (bits 3–5); a cell's two colour bits pick among four character colours, the character colour's low bit underneath them. Row 2 of `sprites` is the same digit with colour bits `00`, `01`, `10` and `11`:

2650 ASSEMBLY (AS) · SPRITES.ASM

```
COLROW: DB      $1C,$28,$25,$28,$2E,$2B,$2C,$00 ; COLOURS
        DB      $11,$51,$91,$D1                  ; 1 in 00 01 10 11
        DB      $00,$00,$00,$00
```

Bit 7 of `$18FD` turns on the *multicolour* mode, in which each cell also chooses its background: colour bit 6 picks the character colour of `$19F9` or of `$19F8`, colour bit 7 the screen colour of one or the other. `$19F8` bits 0–5 are those alternate colours. (Bit 7 of `$18FD` is also the tone's register: write the pitch with the mode bit you want.)

BLOCK GRAPHICS

A cell of exactly `$C0` turns the rest of its row into *block graphics*: each cell after it is a 3 × 2 grid of blocks, its glyph bits saying which are lit, the low three the top row. `$40` turns the row back to characters, and the next row starts as characters anyway. Bit 7 of `$19F8` makes the whole screen block graphics.

2650 ASSEMBLY (AS) · SPRITES.ASM

```
BLKROW: DB      $C0,$3F,$07,$38,$15,$2A,$01,$02
        DB      $04,$08,$10,$20,$3F,$00,$3F,$00
```

Alien Invaders draws its shields this way. It is also why a text program must never paint a blue space: `$C0` is colour bits `11` and glyph 0 (Section 5).

THE BUILT-IN FRAME

Glyphs `$04` to `$0B` are edges and corners, a complete kit for drawing a box without spending a user-defined character on it:

`$04`	top edge	`$08`	top right corner
`$05`	right edge	`$09`	top left corner
`$06`	bottom edge	`$0A`	bottom left corner
`$07`	left edge	`$0B`	bottom right corner

2650 ASSEMBLY (AS) · SPRITES.ASM

```
BOXTOP: DB      $09,$04,$04,$04,$08     ; the built-in corners and edges
BOXMID: DB      $07,$00,$00,$00,$05
BOXBOT: DB      $0A,$06,$06,$06,$0B
```

CHARACTERS OF YOUR OWN

The eight user-defined characters, glyphs `$38` to `$3F`, are drawn from `$1980-$19BF`, eight bytes each, the top row first and bit 7 on the left. Write them at any time; the next frame draws the new shapes everywhere they are on the screen. `fujidisp.inc` loads its own eight at start; a program may load its own instead – CONFIG changes one while its editor runs, to preview the character under the cursor.

SPRITES

The 2637 has four sprites, 8 pixels wide, drawn over everything. Their *images* are the first four user-defined characters: sprite 0 is glyph `$38`, `$1980-$1987`, and so on. A program that uses a sprite gives up that character.

Register	Holds
`$18F0`, `$18F1`	sprite 0's vertical and horizontal position; sprites 1–3 follow in pairs to `$18F7`
`$19FB`	bits 3–5 sprite 0's colour, 0–2 sprite 1's; bit 7 sprite 0 8 × 8 (1) or 8 × 16 (0), bit 6 sprite 1's
`$19FA`	the same for sprites 2 and 3

The positions count oddly. The horizontal register is 43 plus the pixel column, so a sprite sits on text column *c* at 43 + 8 *c*. The vertical register counts *up*: 256 less the line, so with 8 × 16 characters row *r* is at 256 – 16 *r*. `sprites` parks sprite 1 on row 7, column 13, and runs sprite 0 to and fro along the row:

2650 ASSEMBLY (AS) · SPRITES.ASM

```
        LODI,R0 $0D             ; 8x16 both; sprite 0 yellow, 1 red
        STRA,R0 SPCOL
        LODI,R0 YROW7
        STRA,R0 S0Y
        STRA,R0 S1Y
        LODI,R0 43+8*13         ; sprite 1: column 13
        STRA,R0 S1X
        LODI,R0 XLEFT
        STRA,R0 XPOS
        LODI,R0 1
        STRA,R0 STEP
```

COLLISIONS

As it draws a frame the 2637 notes which sprites touched a lit pixel of the characters and which touched each other. A bit reads **0** for a collision:

Reg.	Bit 0 … 5
$19FC	sprites 0, 1, 2, 3 against the characters (bits 0–3)
$19FD	sprites 0 and 1, 0 and 2, 0 and 3, 1 and 2, 1 and 3, 2 and 3

Read them in the vertical blank, for the frame just drawn:

2650 ASSEMBLY (AS) · SPRITES.ASM

```
MAIN:   TPSU    $80             ; the vertical blank
        BCFR,EQ MAIN
        LODA,R0 XPOS
        STRA,R0 S0X
        LODI,R2 $A2-1           ; row 10: the horizontal register
        BSTA,UN DHEX
        LODA,R0 SSHIT           ; bit 0 low: sprites 0 and 1 met
        ANDI,R0 $01
        BCFR,EQ MAIN1
        PUTS    $B0, TSPR
        BCTR,UN MAIN2
MAIN1:  PUTS    $B0, TBLANK
MAIN2:  LODA,R0 SPHIT           ; bit 0 low: sprite 0 met a character
        ANDI,R0 $01
        BCFR,EQ MAIN3
        PUTS    $C0, TCHR
        BCTR,UN MAIN4
MAIN3:  PUTS    $C0, TBLANK
```

`emu/verify.sh` waits until `sprites` has printed both SPRITE HIT and BOX HIT, so both registers are tested on every run.

THE 26-ROW SCREEN

Bit 6 of `$19F8` adds a lower screen: rows 13 to 25, from `$1A00-$1ACF`. With 8 × 8 characters the 26 rows fill the picture, twice the text of the 13-row screen. The cost is the RAM: the lower screen is the 208 bytes Section 6 gave to the program, and only 84 bytes are left – `$18D0-$18EF`, `$18F8-$18FB` and `$1AD0-$1AFF`. M.U.L.E. runs in this mode and keeps its state in the cartridge RAM of Section 15.

SCROLLING AND POSITION

Bits 5–7 of `$18FE` shift the whole screen 0 to 7 pixels left, and `$18FC` sets how far down the character display starts, as a complement. `$18FF` reads the character row the 2637 is drawing, which a game can use to change things part way down the frame. None of the FujiNet programs do.

17. COMMAND REFERENCE: THE FUJI DEVICE

Every command device `$70` answers, checked against the firmware the network chip runs: the RS-232 build's `rs232Fuji.cpp` and the four mixins every FujiNet shares (app keys, base64, hashing, QR codes), at the revision in the Parts List. On the cards, *nparam* is the value for the NPARAM register, and each parameter goes into the TX stream as its size and then its value: a *byte* is 1 and the byte, a *word* 2 and two bytes, low first. The reply is in the window at `$2B00`, 256 bytes at a time.

The examples use the runtime, `bookdefs.inc` and `booklib.inc`, and labels of the program's own such as `DONE` and the strings they name. Every one is assembled by `tools/snipcheck.py`.

WIFI AND THE ADAPTER

GET_ADAPTERCONFIG_EXTENDED / GET_ADAPTERCONFIG
$C4 / $E8

DEVICE $70
NPARAM 0
PARAMS none
PAYLOAD none
REPLY `$C4`: 240 bytes, laid out in Section 10. `$E8`: the first 140, the addresses in binary only

The first-contact command, and the one every status screen wants. It works before the WiFi is up, and its text fields print straight out of the window.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCACFGX, 0
        BSTA,UN FNGO
        BCFA,EQ FAIL
        SETSTR  FNRDATA+140     ; the IP address, as text
        LODI,R2 $90-1
        BSTA,UN DPRINT
```

SCAN_NETWORKS
$FD

DEVICE $70
NPARAM 0
PARAMS none
PAYLOAD none
REPLY 1 byte: how many networks were found

Starts a fresh scan and answers when it is done, which takes seconds: inside the cartridge's five, usually. Follow it with GET_SCAN_RESULT once for each network.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCSCAN, 0
        BSTA,UN FNGO
        BCFA,EQ FAIL
        LODA,R0 *FRDP           ; how many
        STRA,R0 NNETS
```

GET_SCAN_RESULT
$FC

DEVICE $70
NPARAM 1
PARAMS byte: which result, from 0
PAYLOAD none
REPLY 34 bytes: the SSID (33, NUL-terminated), then the signal strength, a signed dBm

The strength is negative: closer to zero is stronger.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCSCANR, 1
        LODA,R2 WHICH           ; which network
        BSTA,UN FNPARB
        BSTA,UN FNGO
        BCFA,EQ FAIL
        LODI,R1 33
        LODA,R0 *FRDP,R1        ; strength: negative dBm
```

SET_SSID
$FB

DEVICE $70
NPARAM 1
PARAMS byte: any value; **required but ignored**
PAYLOAD **exactly 97 bytes**: the SSID (33) then the password (64), each NUL-padded
REPLY ACK

Joins a network and saves it as the default. Two sharp edges: the parameter must be sent although nothing reads it, and the payload must be all 97 bytes; a short one fails. CONFIG builds the 97 bytes in the lower screen's RAM.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCSSSID, 1
        LODI,R2 0
        BSTA,UN FNPARB          ; must be there
        SETFSTR SSIDPW          ; ssid[33], password[64]
        LODI,R2 97
        BSTA,UN FNTXN
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

GET_SSID
$FE

DEVICE $70
NPARAM 0
PARAMS none
PAYLOAD none
REPLY 97 bytes: the SSID (33) then the password (64)

The network the adapter is set to join, the password in clear.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCGSSID, 0
        BSTA,UN FNGO            ; SSID at $2B00, password at $2B21
        BCFA,EQ FAIL
```

GET_WIFISTATUS / GET_WIFI_ENABLED
$FA / $EA

DEVICE $70
NPARAM 0
PARAMS none
PAYLOAD none
REPLY `$FA`: 1 byte, 3 when connected; `$EA`: 1 byte, 1 if the radio is on

What to poll while a join is under way. CONFIG polls it once a second.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCWIFIS, 0
        BSTA,UN FNGO
        BCFA,EQ FAIL
        LODA,R0 *FRDP
        COMI,R0 3               ; connected?
        BCTA,EQ DONE
```

RESET
$FF

DEVICE $70
NPARAM 0
PARAMS none
PAYLOAD none
REPLY ACK, then the adapter restarts

Restarts the network chip. The game chip, its mailbox and your program carry on; the link bit of STATUS, `$2C01`, drops and comes back, and the cartridge waits up to three seconds for it before the next transaction.

STATUS / DEVICE_READY
$53 / $00

DEVICE $70
NPARAM 1 / 0
PARAMS STATUS: byte, the kind of status wanted (1: mount times)
PAYLOAD none
REPLY STATUS 1: four bytes a device slot, a time, 0 if nothing is mounted; DEVICE_READY: the bus test's answer

STATUS needs its parameter on this build. DEVICE_READY is a round trip with no other effect: it proves the mailbox, the game chip, the USB link and the adapter at once.

HOSTS, SLOTS AND MOUNTING

READ_HOST_SLOTS
$F4

DEVICE $70
NPARAM 0
PARAMS none
PAYLOAD none
REPLY 256 bytes: eight host names of 32 bytes – the whole window

Slot 0 is the memory card on a standard fujinet-pc. A name longer than 16 characters runs on to the next row: CONFIG cuts it.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCRHSL, 0
        BSTA,UN FNGO
        BCFA,EQ FAIL
        SETSTR  FNRDATA+32      ; host 1's name
        LODI,R2 $30-1           ; row 3
        BSTA,UN DPRINT
```

WRITE_HOST_SLOTS
$F3

DEVICE $70
NPARAM 0
PARAMS none
PAYLOAD 256 bytes: all eight slots, 32 bytes each
REPLY ACK

All or nothing: there is no command to write one slot. Read the table with READ_HOST_SLOTS and stream it back with the one slot changed. The reply window holds still until the next launch, so the stream can be read straight out of it – `SETFSTR FNRDATA` and `FNTXN` – with the changed slot sent from RAM in its place. CONFIG renames a host this way.

MOUNT_HOST
$F9

DEVICE $70
NPARAM 1
PARAMS byte: the host slot, from 0
PAYLOAD none
REPLY ACK; NAK if the host cannot be reached

Connects the host: SD, TNFS, SMB and the rest. Needed before OPEN_DIRECTORY or a mount. UNMOUNT_HOST (`$E6`) is not answered on this build.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCMHOST, 1
        LODI,R2 0               ; host slot 0
        BSTA,UN FNPARB
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

READ_DEVICE_SLOTS / WRITE_DEVICE_SLOTS $F2 / $F1

DEVICE $70
NPARAM 0
PARAMS none
PAYLOAD none
REPLY $F2: the device-slot table: host slot, mode and file name for each slot; $F1 takes the same table as its payload

What is set up in each of the FujiNet's virtual drives. The Arcadia boots images rather than reading drives, so CONFIG does not show them, but they answer.

SET_DEVICE_FULLPATH $E2

DEVICE $70
NPARAM 3
PARAMS bytes: device slot, host slot, mode (1 to read)
PAYLOAD **256 bytes**: the path, NUL-padded
REPLY ACK

Names the file for a device slot: the first half of booting.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCSDFP, 3
        LODI,R2 0               ; device slot
        BSTA,UN FNPARB
        LODI,R2 0               ; host slot
        BSTA,UN FNPARB
        LODI,R2 FMREAD
        BSTA,UN FNPARB
        SETFSTR FILE
        BSTA,UN FNPATH          ; padded to 256
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

MOUNT_IMAGE / UNMOUNT_IMAGE $F8 / $E9

DEVICE $70
NPARAM 2 / 1
PARAMS $F8: device slot, mode (1 to read); $E9: device slot
PAYLOAD none
REPLY ACK when the mount is done, the image staged in the cartridge

The second half of booting. Before it answers, the FujiNet pushes the file into the cartridge, and BOOT_STATE and BOOT_PCT report the progress; the cartridge allows sixty seconds. Section 13 has the rest.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCMIMG, 2
        LODI,R2 0               ; device slot
        BSTA,UN FNPARB
        LODI,R2 FMREAD
        BSTA,UN FNPARB
        BSTA,UN FNGO            ; up to 60 s
        BCFA,EQ FAIL
PUSH:   LODA,R0 *FBSTP          ; the image, staged?
        COMI,R0 FBREADY
        BCFR,EQ PUSH
```

MOUNT_ALL / SET_BOOT_MODE / CONFIG_BOOT $D7 / $D6 / $D9

DEVICE $70
NPARAM 0 / 1 / 1
PARAMS SET_BOOT_MODE: byte, the mode; CONFIG_BOOT: byte, on or off
PAYLOAD none
REPLY ACK

MOUNT_ALL mounts every slot that has a file; the other two choose what the adapter serves at power-on on computers that boot from its drives. On the Arcadia the cartridge decides what boots, and CONFIG does not send them.

DIRECTORIES

OPEN_DIRECTORY $F7

DEVICE $70
NPARAM 1
PARAMS byte: the host slot
PAYLOAD **256 bytes**: the path, a NUL, then an optional filter such as `*.BIN`, the rest NULs
REPLY ACK

Opens a directory on a mounted host. The adapter applies the filter, so a program never sees what it does not want.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCODIR, 1
        LODI,R2 0               ; host slot
        BSTA,UN FNPARB
        SETFSTR PATH
        BSTA,UN FNPATH
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

READ_DIR_ENTRY $F6

DEVICE $70
NPARAM 2
PARAMS bytes: the longest name wanted (**36 or more**), flags (0)
PAYLOAD none
REPLY one name, NUL-terminated, a `/` at the end of a folder's; **`$7F`, `$7F` when there are no more**

One name a call; the adapter moves on by itself. The folder's `/` is added only within the length asked for. At a length of exactly 31 the firmware puts icon bytes in front of the name. Stop at the end marker: an SD host goes on answering `..` past it.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCRDIR, 2
        LODI,R2 36
        BSTA,UN FNPARB
        LODI,R2 0
        BSTA,UN FNPARB
        BSTA,UN FNGO
        BCFA,EQ FAIL
        LODA,R0 *FRDP
        COMI,R0 $7F             ; the end?
        BCTA,EQ DONE
```

SET_DIRECTORY_POSITION / GET_DIRECTORY_POSITION $E4 / $E5

DEVICE $70
NPARAM 1 / 0
PARAMS `$E4`: **one word**, the entry number
PAYLOAD none
REPLY `$E5`: a word, the current entry number; `$E4`: ACK

Seek and tell for the open directory: paging in one command. The position is one two-byte parameter, not two one-byte ones.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCSDPS, 1
        LODI,R2 8               ; entry 8: page two
        LODI,R3 0
        BSTA,UN FNPARW
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

CLOSE_DIRECTORY $F5

DEVICE $70
NPARAM 0
PARAMS none
PAYLOAD none
REPLY ACK

Close what you open: the adapter has room for few open directories.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCCDIR, 0
        BSTA,UN FNGO
```

FILES, PREFIXES AND PATHS

COPY_FILE
$D8

DEVICE $70
NPARAM 2
PARAMS bytes: source host, destination host, **counting from 1**
PAYLOAD `source|destination`; at its own length, or NUL-padded
REPLY ACK when the copy is done; the cartridge allows sixty seconds

Host to host, without a byte passing through the console. Its hosts count from 1, unlike every other slot number; 0 is refused. When the destination ends in `/`, the adapter adds the source's file name. CONFIG's copy screen sends it at its own length.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCCOPY, 2
        LODI,R2 1               ; host 0, counted from 1
        BSTA,UN FNPARB
        LODI,R2 1
        BSTA,UN FNPARB
        SETFSTR SPEC            ; source|destination
        BSTA,UN FNTXSTR
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

SET_HOST_PREFIX / GET_HOST_PREFIX
$E1 / $E0

DEVICE $70
NPARAM 1
PARAMS byte: the host slot; SET carries the prefix as its payload
PAYLOAD none
REPLY GET: the prefix

A working directory for each host, put in front of relative paths.

GET_DEVICE_FULLPATH
$DA

DEVICE $70
NPARAM 1
PARAMS byte: the device slot
PAYLOAD none
REPLY the path SET_DEVICE_FULLPATH stored

What a device slot holds, as one string.

APP KEYS

OPEN_APPKEY
$DC

DEVICE $70
NPARAM 0
PARAMS none
PAYLOAD 6 bytes: creator (word), app, key, mode (0 read, 1 write), and a reserved 0
REPLY ACK; NAK with no memory card, a creator of 0 or a bad mode

Chooses the key the next read or write uses. The payload is one packed structure; leave off the reserved byte and the adapter waits for it until the transaction times out. On the memory card the key is the file `/FujiNet/CCCCAAKK.key`; it holds 64 bytes at most.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCAKOP, 0
        SETFSTR KEYSEL          ; $50,$26,$FF,0,0,0:
        LODI,R2 6               ; creator $2650, app $FF,
        BSTA,UN FNTXN           ; key 0, read, reserved
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

READ_APPKEY / WRITE_APPKEY / CLOSE_APPKEY
$DD / $DE / $DB

DEVICE	$70
NPARAM	0 / 1 / 0
PARAMS	WRITE: byte, the length
PAYLOAD	WRITE: the value, at most 64 bytes
REPLY	READ: a word, the length, then the value; the others: ACK

A write ends the selection, so each write needs its own OPEN in write mode. Section 14 has both directions.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCAKRD, 0
        BSTA,UN FNGO
        BCFA,EQ FAIL
        LODA,R0 *FRDP           ; the length
        BCTA,EQ DONE            ; empty
        STRA,R0 DRCNT
        LODI,R0 2               ; the value: from byte 2
        STRA,R0 DRPOS
        LODI,R2 $30-1
        BSTA,UN DREPLY
```

UTILITIES

GENERATE_GUID
$BB

DEVICE	$70
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	36 characters and a NUL

A fresh, printable, unique id: 36 characters, more than two rows of the screen (DPRINT does not stop at the edge; the next row is the next 16 cells). RANDOM_NUMBER ($D3) is not answered on this build; the adapter's randomness reaches a program through a GUID or a hash.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCGUID, 0
        BSTA,UN FNGO
        BCFA,EQ FAIL
        SETSTR  FNRDATA         ; rows 4 to 6: it wraps
        LODI,R2 $40-1
        BSTA,UN DPRINT
```

HASH: INPUT, COMPUTE, LENGTH, OUTPUT
$C8 / $C7 / $C6 / $C5

DEVICE	$70
NPARAM	1
PARAMS	INPUT: word, the byte count; COMPUTE: byte, the algorithm (0 MD5, 1 SHA-1, 3 SHA-256, 4 SHA-512); LENGTH and OUTPUT: byte, 1 for hex text, 0 for binary
PAYLOAD	INPUT: the bytes
REPLY	LENGTH: the size; OUTPUT: the hash

Four steps on the adapter: feed bytes with INPUT, as often as needed; COMPUTE; ask the LENGTH; read the OUTPUT. Section 14 has the whole sequence. HASH_COMPUTE_NO_CLEAR ($C3) keeps the input for another algorithm, and HASH_CLEAR ($C2) empties it.

2650 ASSEMBLY (AS)

```
        FNBEGIN FNDFUJI, FCHSHL, 1
        LODI,R2 1               ; as text
        BSTA,UN FNPARB
        BSTA,UN FNGO
        BCFA,EQ FAIL
        LODA,R0 *FRDP           ; 64 for SHA-256
```

BASE64: ENCODE AND DECODE

$D0–$CD / $CC–$C9

DEVICE	$70
NPARAM	varies
PARAMS	INPUT: word, the byte count; OUTPUT: word, how many bytes to read
PAYLOAD	INPUT: the bytes
REPLY	LENGTH: 4 bytes, the size; OUTPUT: the result

The same four steps as the hash. Encoding is INPUT $D0, COMPUTE $CF, LENGTH $CE, OUTPUT $CD; decoding is $CC, $CB, $CA, $C9.

QR CODE: INPUT, ENCODE, LENGTH, OUTPUT

$BC / $BD / $BE / $BF

DEVICE	$70
NPARAM	varies
PARAMS	INPUT: word, the count; ENCODE: version (0 to choose), error correction (0–3), shorten; LENGTH: the output mode (0 bytes, 2 bitmap); OUTPUT: word, the count
PAYLOAD	INPUT: the text
REPLY	LENGTH: 4 bytes, the size; OUTPUT: the modules

The adapter draws a QR code's matrix; a program only has to plot it. A version 1 code is 21 × 21 modules: in block graphics, 3 × 2 blocks a cell, seven cells by eleven rows – most of the screen.

ANSWERED WITH A NAK ON THIS BUILD

These are in fujiCommandID.h but fall through every table the RS-232 build dispatches, so they earn a NAK: ENABLE_UDPSTREAM ($F0), SET_BAUDRATE ($EB), NEW_DISK ($E7), UNMOUNT_HOST ($E6), SET_HSIO_INDEX ($E3), SET_SIO_EXTERNAL_CLOCK ($DF), ENABLE_DEVICE and DISABLE_DEVICE ($D5, $D4), RANDOM_NUMBER ($D3), GET_TIME ($D2; use the clock device), DEVICE_ENABLE_STATUS ($D1), GET_HEAP ($C1), QRCODE_CLEAR ($BA), GET_DEVICE1..10_FULLPATH ($A0–$A9), UPDATE_FIRMWARE ($90) and HSIO_INDEX ($3F). Most of them tune other computers' buses.

18. COMMAND REFERENCE: THE NETWORK DEVICE

Every command devices `$71`–`$78` answer, checked against the network device every FujiNet shares, `NDevice.cpp`. Most of the codes are the ASCII letter of their name, `'O'`pen, `'R'`ead, `'W'`rite, a habit from the Atari 8-bits. What a devicespec can say, and how each protocol behaves, belongs to *The FujiNet Network Protocol Handbook*; these are the cards. A READ's answer must fit the 1,024-byte reply, seen 256 bytes at a time, and a WRITE's bytes the 320-byte TX stream with the parameter in front of them.

The cartridge gives each OPEN, STATUS, READ and CLOSE 90 seconds, and `FNGO` waits that long. The examples use channel 1, `DVNET` (`$71`); `netlib.inc` has all five of the lifecycle as routines.

THE LIFECYCLE

NET_OPEN
$4F ┐'O'

DEVICE $71
NPARAM 2
PARAMS bytes: mode (4 read, 8 write, 12 read and write, 13 POST), translation (0 none, 1 CR, 2 LF, 3 CR LF)
PAYLOAD the devicespec, `N:` and the URL
REPLY ACK once the connection is set up

On HTTP, modes 4 and 12 are both GET; 12 also lets a program set headers (NET_SET_HTTP_MODE). The request itself waits for the first STATUS or READ. Other HTTP modes: 14 PUT with headers, 5 and 9 DELETE. On TCP, mode 12 is a socket both ways, as NetCat uses it. Opening a channel that is already open closes the old connection first.

2650 ASSEMBLY (AS)

```
        FNBEGIN DVNET, NCOPEN, 2
        LODI,R2 NMRDWR          ; 12
        BSTA,UN FNPARB
        LODI,R2 NTRNONE
        BSTA,UN FNPARB
        SETFSTR URL             ; as long as it is
        BSTA,UN FNTXSTR
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

NET_STATUS
$53 ┐'S'

DEVICE $71
NPARAM 2
PARAMS two bytes of 0
PAYLOAD none
REPLY 4 bytes: bytes waiting (word), connected, status (1 good, 136 end of file)

The heartbeat. A terminal watches *connected*; a program that fetches and shows watches the status for the end of the file; every program reads the count, until two readings agree (Section 11). Copy the four bytes before the next transaction, as `NSTAT` does.

2650 ASSEMBLY (AS)

```
        FNBEGIN DVNET, NCSTAT, 2
        LODI,R2 0
        BSTA,UN FNPARB
        LODI,R2 0
        BSTA,UN FNPARB
        BSTA,UN FNGO
        BCFA,EQ FAIL
        LODA,R0 *FRDP           ; bytes waiting, low...
        LODI,R1 1
        IORA,R0 *FRDP,R1        ; ...or high
        BCTA,EQ DONE            ; nothing yet
```

NET_READ
$52 'R'

DEVICE $71
NPARAM 1
PARAMS **one word**: how many bytes, no more than are waiting and no more than 1,024
PAYLOAD none
REPLY that many bytes

A short or failed read is a NAK. Use the bytes, or copy them, before the next transaction paints over the window; past the first 256, `FNSLICE`.

2650 ASSEMBLY (AS)

```
        FNBEGIN DVNET, NCREAD, 1
        LODI,R2 128             ; 128 bytes
        LODI,R3 0
        BSTA,UN FNPARW
        BSTA,UN FNGO            ; RXLEN bytes at $2B00
        BCFA,EQ FAIL
```

NET_WRITE
$57 'W'

DEVICE $71
NPARAM 1
PARAMS word: how many bytes
PAYLOAD that many bytes
REPLY ACK

The count goes twice: as the parameter, and as the length of the payload. The parameter's three bytes leave 317 for the data in one transaction.

2650 ASSEMBLY (AS)

```
        FNBEGIN DVNET, NCWRITE, 1
        LODI,R2 5
        LODI,R3 0
        BSTA,UN FNPARW          ; the count...
        SETFSTR DATA            ; ...and the bytes
        LODI,R2 5
        BSTA,UN FNTXN
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

NET_CLOSE
$43 'C'

DEVICE $71
NPARAM 0
PARAMS none
PAYLOAD none
REPLY ACK

Close every connection you open, on the way out of a failure too; the adapter keeps it open otherwise, even after the console is switched off.

2650 ASSEMBLY (AS)

```
        FNBEGIN DVNET, NCCLOSE, 0
        BSTA,UN FNGO
```

PARSERS, PARAMETERS AND LINE ENDINGS

NET_SET_PARSER / NET_PARSE / NET_QUERY
$FC / $50 'P' / $51 'Q'

DEVICE $71
NPARAM 2 / 0 / 0
PARAMS SET_PARSER: byte (any), then the parser: 0 none, 1 JSON, 2 HTML, 3 XML
PAYLOAD QUERY: the path, such as `/0/name`
REPLY QUERY: ACK; then STATUS and READ the value as text

Switch an open connection to one of the adapter's parsers (the parser is the **second** parameter), parse the body, then ask for one value by its path. Section 11 has the three in a row. Read each value to its end before the next QUERY.

NET_SET_PARAMETER $FB

DEVICE $71
NPARAM 2
PARAMS bytes: which (0 the query flags, 1 the line ending), the value
PAYLOAD none
REPLY ACK

How the parser answers a query: flags remap characters and choose the output form; the line ending is the byte that ends each value.

NET_SET_HTTP_MODE $4D 'M'

DEVICE $71
NPARAM 2
PARAMS byte (any), then the mode: 0 body, 1 collect headers, 2 get headers, 3 set headers, 4 POST data
PAYLOAD none
REPLY ACK; NAK on a connection that is not HTTP

HTTP only. In mode 3 each WRITE sets a request header; in mode 4 a WRITE supplies the body of a POST; in mode 2 a READ returns the headers collected in mode 1. Mode 0 is the page itself. Open with mode 12 to use it.

2650 ASSEMBLY (AS)

```
        FNBEGIN DVNET, NCHTTPM, 2
        LODI,R2 0
        BSTA,UN FNPARB
        LODI,R2 3               ; WRITE sets headers
        BSTA,UN FNPARB
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

NET_SEEK / NET_TELL $25 '%' / $26 '&'

DEVICE $71
NPARAM 1 / 0
PARAMS SEEK: **one four-byte parameter**, the offset
PAYLOAD none
REPLY TELL: 4 bytes, the position

Random access where the protocol has it: TNFS and other files, HTTP ranges. The libraries have no four-byte helper: append a 4, then the four bytes, low first, each with `LODA,R0 *FTXP,R1`.

NET_TRANSLATION / NET_SET_EOL / NET_SET_INT_RATE $54 'T' / $4C 'L' / $5A 'Z'

DEVICE $71
NPARAM 2 / 0 / 1
PARAMS TRANSLATION: byte (any), then the code; SET_INT_RATE: the rate
PAYLOAD SET_EOL: the line ending, as a string; empty restores the default
REPLY ACK

Line-ending translation and terminators after OPEN. SET_INT_RATE paces an interrupt line on computers that have one; the cartridge has no such line, so the command is accepted and does nothing. Polling ACKSEQ is the only way an Arcadia program learns a transaction is done.

FILES, SOCKETS AND CREDENTIALS

RENAME / DELETE / LOCK / UNLOCK / MKDIR / RMDIR
$20 / $21 / $23 / $24 / $2A / $2B

DEVICE	$71
NPARAM	2
PARAMS	bytes: mode (4), and 0
PAYLOAD	the devicespec; RENAME: `N:…/old,new`
REPLY	ACK

File operations on the protocols that have files: TNFS, FTP, SMB and the memory card. Each opens a connection just for itself and closes it again.

NET_CHDIR / NET_GETCWD
$2C / $30

DEVICE	$71
NPARAM	0
PARAMS	none
PAYLOAD	CHDIR: the path: absolute, relative, .., or a whole new `SCHEME://host/`
REPLY	GETCWD: the prefix

A working directory for the channel, put in front of every devicespec that does not name its own host.

NET_ACCEPT / NET_CLOSE_CLIENT
$41 'A' / $63 'c'

DEVICE	$71
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	ACK

For a TCP server: a channel opened on `N:TCP://:port/` listens; ACCEPT takes the waiting client, and CLOSE_CLIENT hangs up on that client while the channel keeps listening. An ARCADIA-2001 can be the bulletin board.

NET_GET_REMOTE / NET_SET_DESTINATION / NET_USERNAME / NET_PASSWORD
$72 'r' / $44 'D' / $FD / $FE

DEVICE	$71
NPARAM	0
PARAMS	none
PAYLOAD	SET_DESTINATION: `host:port`; USERNAME and PASSWORD: the credential
REPLY	GET_REMOTE: who sent the last UDP datagram

UDP's "who said that" and "talk to them instead", and credentials for protocols that log in (FTP, SMB and others). Send the credentials before the OPEN.

ANSWERED WITH A NAK ON THIS BUILD

In `fujiCommandID.h` but not in the network device's table: NET_GET_DSTATS_VALUE (`$FF`), NET_SET_UNIT (`$FA`), NET_SET_HSIO_INDEX (`$E3`), NET_QUERY_ALT and NET_PARSE_ALT (`$81`, `$80`), NET_GET_ERROR (`$45`; STATUS's fourth byte says what went wrong), and NET_HSIO_INDEX (`$3F`).

19. COMMAND REFERENCE: DISK, PRINTER, CLOCK AND MODEM

The mailbox is a transport, not a translator: whatever byte goes into the DEVICE register goes onto the wire. Beyond the Fuji and network devices, the FujiNet answers four more for an ARCADIA-2001: disk drives, a printer, a clock and a Hayes-style modem. The clock is exercised by a program in this handbook, and its replies were measured on the bench. The others are reachable, transcribed from the firmware's handlers, and offered as such.

DISK: DEVICES `$31–$38`

Eight virtual drives, `$31` plus the drive number, each serving the image mounted in its device slot. On the Arcadia a mount also pushes the image into the cartridge; reading it as a disk as well is possible, and a way for a program to read a file a sector at a time.

DISK_READ / DISK_WRITE / DISK_PUT `$52 / $57 / $50`

DEVICE `$31`
NPARAM `1`
PARAMS the sector number, as one parameter (two or four bytes)
PAYLOAD WRITE and PUT: one sector
REPLY READ: one sector; WRITE and PUT: ACK

With nothing mounted, READ answers NAK. WRITE verifies what it wrote; PUT does not.

2650 ASSEMBLY (AS)

```
        FNBEGIN DVDISK, $52, 1  ; DISK_READ, drive 1
        LODI,R2 1               ; sector 1
        LODI,R3 0
        BSTA,UN FNPARW
        BSTA,UN FNGO            ; the sector at $2B00
        BCFA,EQ FAIL
```

DISK_STATUS `$53`

DEVICE `$31`
NPARAM `0 or 1`
PARAMS optional byte: the kind of status
PAYLOAD none
REPLY 4 bytes: drive status, controller status, format timeout, 0

The Atari-style status block, from whatever image is mounted.

DISK_FORMAT / FORMAT_MEDIUM / PERCOM_READ / PERCOM_WRITE $21 / $22 / $4E / $4F

DEVICE	$31
NPARAM	0
PARAMS	none
PAYLOAD	none
REPLY	FORMAT: a map of bad sectors; PERCOM_READ: a 12-byte geometry block

Atari-drive commands the RS-232 build keeps; on an image they do little.

PRINTER: DEVICE $40

PRINTER_WRITE / PRINTER_PUT / PRINTER_STATUS $57 / $50 / $53

DEVICE	$40
NPARAM	0 / 0 / 1
PARAMS	STATUS: byte, required but ignored
PAYLOAD	WRITE and PUT: the bytes to print, fewer than 255
REPLY	WRITE and PUT: ACK; STATUS: 4 bytes

The page goes to a file on the adapter, in the printer it is set to imitate, and comes out through its web page. A printer switched off in the adapter's settings does not answer at all, and the cartridge reports a timeout.

2650 ASSEMBLY (AS)

```
        FNBEGIN DVPRN, $57, 0   ; PRINTER_WRITE
        SETFSTR LINE            ; the text, then a CR
        BSTA,UN FNTXSTR
        LODI,R1 $0D
        LODA,R0 *FTXP,R1
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

CLOCK: DEVICE $45

On by default. A read with a parameter of 1 uses the alternate time zone instead of the adapter's own. The lengths below are what the bench's fujinet-pc answered.

GETTIME / GET_ISO_LOCAL / GET_ISO_UTC
$93 / $49 'I' / $5A 'Z'

DEVICE $45
NPARAM 0 or 1
PARAMS optional byte: 1 for the alternate zone
PAYLOAD none
REPLY GETTIME: 6 bytes: day, month, year (two digits), hour, minute, second. The ISO forms: `2026-11-03T09:57:25-0500` and a NUL, 25 bytes

Section 14 prints the ISO forms. GET_ISO_UTC ignores the alternate zone.

2650 ASSEMBLY (AS)

```
        FNBEGIN DVCLOCK, CKGTIME, 0
        BSTA,UN FNGO
        BCFA,EQ FAIL
        LODI,R1 3
        LODA,R0 *FRDP,R1        ; the hour
        STRA,R0 HOUR
```

THE OTHER FORMATS
$54 $50 $53 $4D $47 $4C $9A

DEVICE $45
NPARAM 0 or 1
PARAMS none
PAYLOAD none
REPLY `$54 'T'`: 7 bytes, century, year, month, day, hour, minute, second; `$50 'P'`: 4 bytes, ProDOS; `$53 'S'`: 19 characters, Apple III SOS; `$4D 'M'`: the 7 bytes and hundredths; `$47 'G'`: the zone's name, NUL-terminated; `$4C 'L'`: one byte, its length; `$9A`: GETTIME in the alternate zone

Every format the FujiNet's clock knows, and the zone. Each also has a lower-case twin, `$69 'i'`, `$7A 'z'`, `$70 'p'`, `$73 's'` and `$61 'a'` (the Atari's own six bytes, also `$41`), that reads in the alternate zone. `$54` doubles as a write command on buses that have one; on this one it only reads.

SETTZ_ALT / SETTZ
$74 't' / $99

DEVICE $45
NPARAM 0
PARAMS none
PAYLOAD a POSIX time-zone string, such as `EST5EDT`
REPLY ACK

`$74` sets the zone the adapter uses for local time, and saves it; `$99` sets the **alternate** zone that a read with a parameter of 1 uses. A string that is not a valid zone is dropped without a word.

2650 ASSEMBLY (AS)

```
        FNBEGIN DVCLOCK, CKSETTA, 0 ; the adapter's zone
        SETFSTR TZONE
        BSTA,UN FNTXSTR
        BSTA,UN FNGO
        BCFA,EQ FAIL
```

MODEM: DEVICE $50

A Hayes-style modem that dials TCP instead of telephone lines: `ATDT host:port` opens a connection, and `ATNET1` runs it through telnet negotiation. Ten packet commands set it up and poll it; the characters themselves flow through WRITE and STATUS.

MODEM_STATUS / MODEM_WRITE / MODEM_STREAM $53 / $57 / $58

DEVICE	$50
NPARAM	1 / 0 / 0
PARAMS	STATUS: byte, required but ignored
PAYLOAD	WRITE: an AT command line, or data once connected
REPLY	STATUS: 2 bytes, the second the handshake lines ($CD connected with data waiting, $CC connected, $00 idle); WRITE: ACK; STREAM: a 9-byte baud table

The other seven set the modem up: CONTROL $41, CONFIGURE $42, SET_DUMP $44, LISTEN $4C, UNLISTEN $4D, BAUDRATELOCK $4E, AUTOANSWER $4F; and $3F, $40, $21 and $26 answer the Atari's handler loader and polls. Switched off in the adapter's settings, the modem does not answer.

20. COMMAND REFERENCE: THE CARTRIDGE ITSELF

Everything the game chip does on its own account, by the names in the firmware's `pico/arcadia/firmware/include/fuji_mailbox.h` and the shared `pico/common/include/fuji_mailbox_proto.h`, which every FujiNet cartridge uses. `fujilib.inc` mirrors them by hand under shorter names, given in the last column; `tools/defcheck.py` and `tools/cartcheck.py` hold this section and the library to the headers.

THE IMAGE

Name	Value	Is	fujilib
`FN_WINDOW_SIZE`	`0x2000`	the image, 8,192 bytes: block 1 at `$0000`, block 2 at `$2000`	
`FN_ROM_TOP`	`0x1B00`	the client's code and data end below this: 6,912 bytes	
`FN_R_CLAIM`	`0x1CFC`	`FUJI` here keeps the mailbox alive after a boot (`$2CFC`)	
`FN_R_CLAIM_SIG`	`"FUJI"`	the claim; `FN_R_CLAIM_LEN` is 4	

An image without the claim kills hotspot decode for the session: from then on, the mailbox's pages are plain ROM. `checkrom.py` also wants `$1F` (a `BCTA,UN`) at `0x0000`, `$17` (`RETC,UN`) at `0x0003`, and zeros in `0x1B00-0x1FFF` but for the claim.

HOTSPOTS: CONSOLE TO CARTRIDGE

Name	Console	A read of it, plus *n*	fujilib
`FN_H_REGSEL`	`$2D00`	*n* under `$80`: arm register *n*	`FNRSEL`
`FN_H_REGDATA`	`$2E00`	the armed register = *n*, and disarm	`FNRDAT`
`FN_H_DATA`	`$2F00`	append *n* to the TX stream	`FNDATA`
`FN_HOT_SWAP`	`$2DFE`	serve the staged image; only once armed	`FNSWAP`
`FN_HOT_BANK`	`$2D80`	*n* under 32: the window shows page *n* (to `FN_HOT_BANK_LAST`, `$2D9F`)	`FNBANK`

`FN_H_PAGE_MASK` (`0x1F00`) is how the decode tells the REGSEL page from the others. REGSEL offsets `$80-$FF` are one-shot operations, not registers: the swap and the bank selects.

The gates against stray reads. A REGDATA with no REGSEL just before it does nothing (but `$2D80`+*n* always selects); a transaction launches only on a SEQ that is not 0 and not the last one answered; the swap fires only after BOOTLOCK; an image without the claim kills the mailbox; and a hotspot acts once for each stable state of the address lines, so the game chip counts a read once however long the 2650 holds it.

REGISTERS

Name	No.	A value written	fujilib
FN_REG_DEVICE	$00	the FujiBus device	FRDEVIC
FN_REG_CMD	$01	the command	FRCMD
FN_REG_NPARAM	$02	the number of parameters, at most 8	FRNPARM
FN_REG_DATA_RST	$05	any value: rewind the TX stream	FRDATRS
FN_REG_RXSLICE	$06	the reply slice the window shows, 0–3	FRRXSLC
FN_REG_SEQ	$10	not 0 and not ACKSEQ: launch	FRSEQ
FN_REG_BOOTLOCK	$11	FN_BOOTLOCK_MAGIC ($B5): arm the staged image	FRBOOTL
FN_REG_BOOTSEL_1	$12	FN_BOOTSEL_MAGIC1 ($B5), then…	FRBSEL1
FN_REG_BOOTSEL_2	$13	…FN_BOOTSEL_MAGIC2 ($4A): restart the game chip into its USB loader, for new firmware	FRBSEL2
FN_REG_PLATFORM	$14	the first register a cartridge may define for itself	
FN_REG_RAMPTR0	$14	protocol 2: the cartridge RAM pointer = 0x000 + value; $15, $16 and FN_REG_RAMPTR3 ($17) for 0x100, 0x200, 0x300	FRRAMP0
FN_REG_RAMPOKE	$18	protocol 2: each REGDATA after it stores and advances	FRRAMPK

BOOTSEL takes two register writes, each with its own magic, so that no single stray read can trigger it.

THE TX STREAM

Name	Value	Is
FN_TX_MAX	320	the stream's length: NPARAM × (size, value, low first), then the payload
FN_STREAM_ROM	0	the DBC push of the image itself
FN_STREAM_CFG	1	the push of a .cfg sibling, whose mapper= line chooses a mapper on carts that have them; the Arcadia has none

THE STATUS PAGE: CARTRIDGE TO CONSOLE

Name	Console	Holds	fujilib
FN_R_DATA	$2B00	the reply, FN_R_SLICE_LEN (256) bytes of it; FN_R_NSLICES (4) slices	FNRDATA
FN_R_BASE	$2C00	the first painted status byte	
FN_R_ACKSEQ	$2C00	the SEQ answered: painted last	FNACKSQ
FN_R_STATUS	$2C01	bit 0 FN_R_STATUS_LINK, the USB link is up; bit 1 FN_R_STATUS_BUSY	FNSTATS
FN_R_ERR	$2C02	the transaction's result (Appendix B)	FNERR
FN_R_REPLY_CMD	$2C03	$06 ACK or $15 NAK	FNREPLY
FN_R_RXLEN_LO	$2C04	the reply's length, with FN_R_RXLEN_HI ($2C05)	FNRXLL
FN_R_BOOT_STATE	$2C06	the staged image (below)	FNBSTAT
FN_R_BOOT_PCT	$2C07	0–100, for a progress bar	FNBPCT
FN_R_BOOT_ERR	$2C08	why staging failed	FNBERR
FN_R_MAGIC0	$2C09	'F', then FN_R_MAGIC1 ($2C0A) 'N'	FNMAGF
FN_R_PROTO_VER	$2C0B	FN_PROTO_VER: 2	FNPVER
FN_R_SLICE_ECHO	$2C0C	the slice the window shows, painted after it	FNSECHO
–	$2C0D	reserved: reads 0	
FN_R_TICK_LO	$2C0E	a 60 Hz count, with FN_R_TICK_HI ($2C0F)	FNTIKL

Name	Console	Holds	fujilib
FN_R_PAINT_END	$2CFC	the painted page ends here, at the claim	

BOOT STATES AND ERRORS

Name	Value	Means	fujilib
FN_BOOT_IDLE	0	nothing staged	FBIDLE
FN_BOOT_XFER	1	an image is coming	FBXFER
FN_BOOT_READY	2	staged and checked: BOOTLOCK may arm it	FBREADY
FN_BOOT_FAILED	$80	staging failed: see BOOT_ERR	FBFAIL
FN_BOOT_ERR_TOOBIG	1	larger than the cartridge can hold (71,680)	
FN_BOOT_ERR_TRUNCATED	2	fewer bytes came than were promised	
FN_BOOT_ERR_NOMAP	3	over 8K but not 8K + *k* × 2K, or with no claim	
FN_BOOT_ERR_STOREBUSY	4	the only store it would fit is being served from	

The game chip keeps two image stores. A push always lands in the one it is not serving, and staging an image with the claim copies the live status page and reply into it, so the program it boots finds F, N, the version and ACKSEQ in place.

PROTOCOL 2

Name	Value	Is
FN_APP_WIN	0x0800	the window, console $0800–$0FFF
FN_APP_PAGE	0x0800	a page: 2,048 bytes
FN_APP_NPAGES	32	page 0, in the base, and up to 31 more
FN_APP_MAX	0x11800	the largest image: 71,680 bytes
FN_RAM_L	0x0600	cartridge RAM, the low half, at console $0600
FN_RAM_H	0x1900	the high half, at console $2900
FN_RAM_HALF	0x0200	each half, 512 bytes; FN_RAM_SIZE 0x0400 in all

Page *n* is image 0x2000 + (*n* – 1) × 0x800; a page that is not there reads $FF; the swap maps page 0; a console RESET changes neither the page nor the RAM. Section 15 has the whole of it.

TIMEOUTS

Transaction	Allowed	Notes
MOUNT_IMAGE, COPY_FILE	60 s	
a network channel's OPEN, STATUS, READ, CLOSE	90 s	
everything else	5 s	after waiting up to 3 s for the USB link, if it is down

When one runs out, ERR is 2 (FN_ERR_TIMEOUT) and ACKSEQ is painted as for any other answer.

ON THE BOARD

The game chip serves the bus from its second core: it watches A0–A13 and answers combinationally, without the read strobe the connector does not have, and counts a hotspot once per change of the full address, after two samples 100 ns apart agree. The network chip is on the far side of a USB link inside

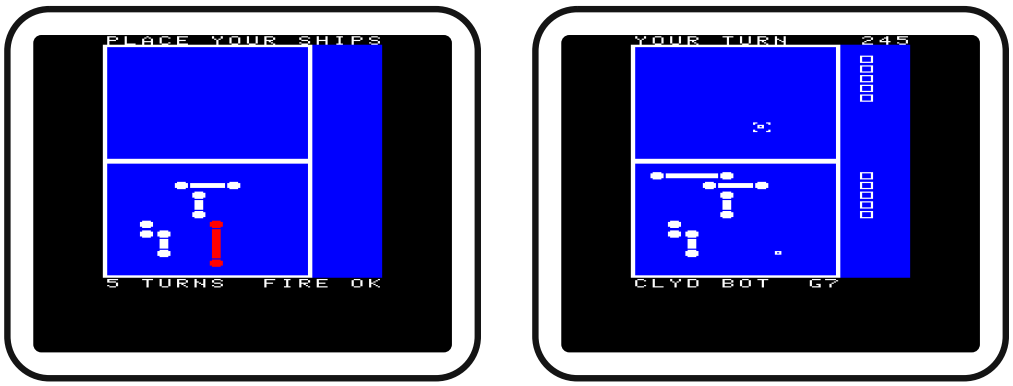
the cartridge, speaking FujiBus; the cartridge’s own RESET restarts both. The pin map and the power path belong to the hardware repository’s Rev0 design, which the Parts List names.

21 BATTLESHIP

HOST APPS.IRATA.ONLINE **FILE** ARCADIA/!FUJINET/Battleship.bin

I. GENERAL

Battleship is the FujiNet's game for two to four fleets on one ocean, played against people or the server's robot admirals. The Arcadia client is 2650 assembly, `fujinet-battleship/arcadia/` at the revision in the Parts List: an 8K image, protocol 1, no banking. Its game logic is the Astrocade client's, cut again for this hardware. This section is about how it is built; the Owner's Manual tells how to play it.



PLACING THE SHIPS — A SHOT FIRED AND ANSWERED

FIG. 12
BATTLESHIP, AGAINST THE BENCH'S SERVER AND ITS ROBOT ADMIRAL

II. THE SCREEN

Battleship runs the 26-row mode with 8 × 8 characters, and the multicolour mode (bit 7 of `$18FD`): `$19F8` = `$46` and `$19F9` = `$AF`, so a cell's two colour bits choose white or red ink on blue or black. A framed 10 × 10 board is exactly 12 rows, so two boards, a status row and a footer fill the 26 with nothing left over – and the split between the two screen halves falls exactly between the boards. The frames are the 2637's built-in glyphs `$04`–`$0B`, which leaves all eight user-defined characters for the ships, the hits and the misses; `$38` is also the targeting reticle's image, sprite 0.

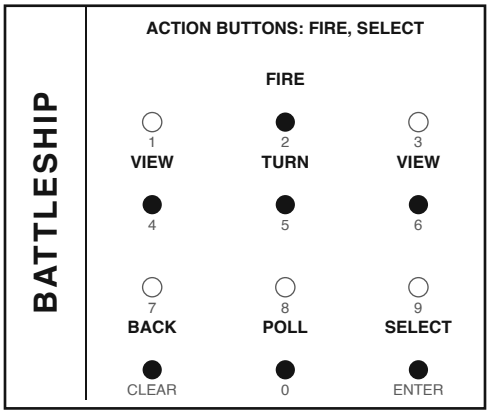
One routine writes every coloured cell, and it knows the trap of Section 5:

2650 ASSEMBLY (AS) · DISP.INC

```
DPUTC:  IORA,R0 DCOLOR
        COMI,R0 $C0             ; space with attrs 11 = block-gfx trap
        BCFR,EQ DPUT1
        LODI,R0 $80             ; same look (blank white cell), no trap
DPUT1:  STRA,R0 *SCRP,R2,+
        RETC,UN
```

A space with colour bits `11` would turn the row into block graphics, so `DPUTC` writes `$80` instead, which looks the same here.

III. THE CONTROLS



The disc moves the reticle in both directions, which most Arcadia programs do not attempt: both axes of a disc arrive at `$19FF`, one at a time, by the multiplexer of Section 9. `KEYRAW` flips the multiplexer once a frame, samples only the axis it selected the frame before (so the 2637 has had a whole frame to digitise it), and decodes the direction from the *cached* pair – so a held direction reads the same every frame, which the edge detection and the auto-repeat need.

2650 ASSEMBLY (AS) · INPUT.INC

```
KEYRAW: LODA,R0 AXSEL           ; arm the mux for the axis we will sample
        BCTR,EQ KRVERT
        LODA,R0 BGCBAS
        IORI,R0 $40             ; mux SET: horizontal
        BCTR,UN KRARM
KRVERT: LODA,R0 BGCBAS          ; mux CLEAR: vertical
KRARM:  STRA,R0 BGCOL
        BSTR,UN WAITVB          ; a whole frame with the mux settled
        LODA,R0 JOYP1
        STRZ    R1
        LODA,R0 AXSEL
        BCTR,EQ KRSTV
        LODZ    R1
        STRA,R0 AXH
        LODI,R0 0               ; alternate for next frame
        BCTR,UN KRSTD
KRSTV:  LODZ    R1
        STRA,R0 AXV
        LODI,R0 1
KRSTD:  STRA,R0 AXSEL
; Decode from the CACHE, not from this frame's sample: vertical wins
; over horizontal so a diagonal push resolves to one axis.
        LODA,R0 AXV
        COMI,R0 JOYLO
        BCTR,LT KRUP
        COMI,R0 JOYHI
        BCTR,GT KRDN
        LODA,R0 AXH
        COMI,R0 JOYLO
        BCTR,LT KRLF
        COMI,R0 JOYHI
        BCTR,GT KRRT
```

The base value of `$19F9` lives in `BGCBAS`, set when a screen's mode is, because the same register holds the character height and the palette: leave it unset and the first frame writes 0 over the colours.

IV. THE NETWORK

Block 2 shares the 2650's page 1 with the mailbox, so the whole network stack lives there and every hotspot read is a direct indexed load; block 1 holds what touches the screen. `net.inc` carries five rules the FujiNet family learned on the Odyssey 2 and the Intellivision, at its head:

2650 ASSEMBLY (AS) · NET.INC

```
; A 2650 transcription of astrocade/net.inc's proven sequence (which is
; o2/net.inc's), over mailbox.inc. The rules it carries forward, learned
; the hard way on the Odyssey 2 and Intellivision:
;
;  * The CLOSE belongs at the START of the next request, never after the
;    READ: every transaction repaints the whole reply window, a CLOSE
;    reply is empty, and every screen renders straight out of the window
;    between polls.
;  * FNRXLL/FNRXLH belong to the most recent transaction, so the reply
;    length is captured immediately after the READ.
;  * The ESP32 reports STATUS as soon as SOME of the response has
;    arrived; poll until two consecutive readings agree (bounded) rather
;    than guessing a delay.
;  * The 4th STATUS reply byte is nDevStatus_t and checking it is not
;    optional: the HTTP GET is deferred until the first STATUS, and an
;    HTTP error page still has a readable body.
;  * Validate the reply length against what playerCount implies before
;    believing a byte of it.
;
```

`APICALL` is one round trip: close the previous request, open this one, STATUS until two readings agree, READ, check the length:

2650 ASSEMBLY (AS) · NET.INC

```
APICALL: STRA,R0 *PVURL
         BSTA,UN NCLOSE          ; the PREVIOUS request's connection
         BSTA,UN NOPEN
         BCFA,EQ APRET           ; open failed: report it (already closed)
         EORZ    R0
         STRA,R0 *PPRA0
         STRA,R0 *PPRA1
         LODI,R0 20              ; settle-loop bound
         STRA,R0 *PVTMP
APSET:   BSTA,UN NSTATUS
         BCFA,EQ APFAIL
         LODA,R0 *PAVA0
         BCFR,EQ APNZ
         LODA,R0 *PAVA1
         BCTR,EQ APAGN           ; nothing has arrived at all yet
APNZ:    LODA,R0 *PAVA0
         COMA,R0 *PPRA0
         BCFR,EQ APAGN
         LODA,R0 *PAVA1
         COMA,R0 *PPRA1
         BCTR,EQ APRD            ; two readings agree: it has landed
APAGN:   LODA,R0 *PAVA0
         STRA,R0 *PPRA0
         LODA,R0 *PAVA1
         STRA,R0 *PPRA1
         LODI,R3 3               ; ~3 frames between polls
APDLY:   BSTA,UN WAITVB
         BDRR,R3 APDLY
         LODA,R0 *PVTMP
         SUBI,R0 1
         STRA,R0 *PVTMP
         BCFA,EQ APSET
```

The wire format is the server's binary one (`?bin=1&v=2`), drawn straight out of the reply window: the 100-cell board is read in order, one byte a cell, and nothing is buffered.

V. PLACEMENT

The ships are seeded at random and then moved by hand, and each move is checked for overlap – with 84 bytes of RAM in the 26-row mode and a 100-byte board, there is nowhere to keep an occupancy map. So the screen is the map. `PLDRWO` repaints your board with every hull but the one being

moved, and `TESTSHP` reads the cells that ship would cover back through the same pointer the writer uses: legal if every one is open sea.

2650 ASSEMBLY (AS) · PLACE.INC

```
PLTRY:  BSTA,UN PLDRWO          ; the screen becomes the occupancy map
        BSTA,UN TESTSHP
        BCFR,EQ PLTBAD
        LODA,R0 V_CURS
        STRZ    R2
        LODA,R0 V_PLC
        STRA,R0 SHIPS,R2
        BSTA,UN PLCUR
        BCTA,UN SNDMOVE         ; tail call
PLTBAD: LODA,R0 V_CURS          ; put the settled one back on screen
        STRZ    R2
        LODA,R0 SHIPS,R2
        STRA,R0 V_PLC
        BSTA,UN PLCUR
        BCTA,UN SNDERR          ; tail call
```

VI. THE DEPTH

The placement redraw is the program's deepest chain. The client's README audits it by hand at six of the eight return addresses; `depthcheck.py` agrees, counting the main loop's `GLPLC` as the seventh routine:

```
GLPLC -> PLACSCR -> PLSHOW -> PLDRWO -> YBCLR -> DFILL -> DPUTC
```

The firmware's `checkdepth.py` reports less, for the reasons of Section 7.

VII. ON THE BENCH

`listings/games.sh` archives the client at its pin and builds it with `ENDPOINT=http://127.0.0.1:8792/`, the bench's own Battleship server; `emu/games.sh` walks it in real time: the name screen, the table list (the fifth table, AI-1, has one robot), READY, the five ships left where they were seeded, and a shot two squares right and two down. The robot answers; the walk passes when it is your turn again.

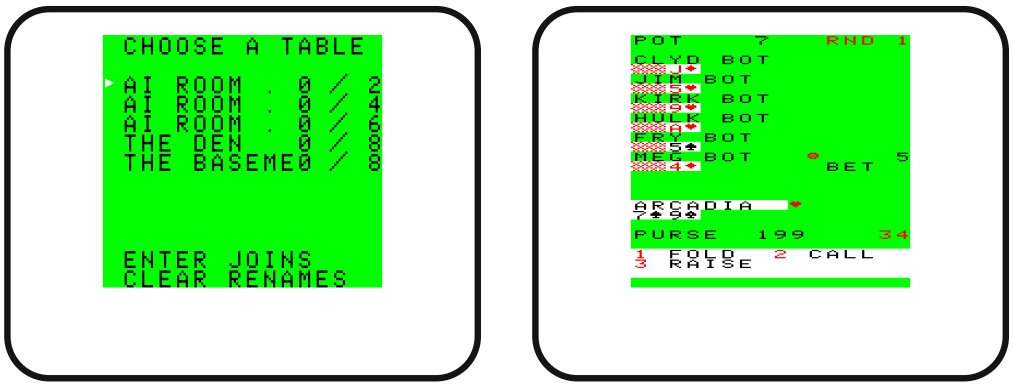
IMPORTANT: `/ready` toggles, and the server's countdown is wall-clock time. A script that presses the Action button again in the lobby un-readies the player and cancels the countdown: press once, and wait for the screen to change.

22 5 CARD STUD

HOST APPS.IRATA.ONLINE **FILE** ARCADIA/!FUJINET/5 Card Stud.bin

I. GENERAL

5 Card Stud is the FujiNet's poker game: up to eight players at a table on the server, robots among them if you like. The Arcadia client, `fujinet-5cardstud/arcadia/`, is 2650 assembly in an 8K image, built like Battleship and on the same mailbox library, with the Astrocade client's architecture: the server's binary wire format, everything drawn straight out of the reply window, and one opaque redraw per poll.



THE TABLES — A HAND AGAINST THE ROBOTS

FIG. 13
5 CARD STUD, AGAINST THE BENCH'S SERVER

II. THE SCREEN

Two modes, chosen by screen. The title, the name and the table list are 13 rows of 8 × 16 characters, and use `$1A00–$1ACF` as scratch while the lower screen is off. The game is the 26-row mode with 8 × 8 characters and the multicolour palette – black or red ink on green felt or white cards:

2650 ASSEMBLY (AS) · DISP.INC

```
DINI13: LODI,R0 $3B             ; ink A black(7), bg A green(3)
        STRA,R0 GFXMOD
        LODI,R0 $28             ; 8x16 doublescan: 13 big rows fill the frame
        BCTR,UN DINIC
DINI26: LODI,R0 $7B             ; + bit6: 26-line
        STRA,R0 GFXMOD
        LODI,R0 $A8             ; 8x8 cells
DINIC:  STRA,R0 BGCOL           ; (both: mux clear, ink B red, bg B white)
```

A card is two cells, the rank and the suit, both in the suit's colour on the white card; the ten has a character of its own so it stays one cell, and a hidden card is two cells of lattice at `$C0` – a red card back, and the one place a `$C0` cell is meant (its glyph is not 0, so it is not the block-graphics switch). The user-defined characters are reloaded by screen: the four suits on the title and the table, which are also the four sprites that bob under the title; an arrow, `/` and `-` in the lobby.

III. READING THE REPLY WHERE IT LIES

A table's state is 154 bytes and 33 more for each player, up to 418, more than one slice of the window. So the client never asks for "byte *n*": it keeps a cursor, a slice and an offset, and `RXNEXT` returns the byte under it and moves on, selecting the next slice when the offset wraps:

2650 ASSEMBLY (AS) · STATE.INC

```
RXNEXT: LODA,R0 *PRXSLC
        COMA,R0 *PCURSL
        BCTR,EQ RXN1
        STRA,R2 *PSAV2          ; shelter the caller's R2
        STRZ    R2              ; wanted slice
        STRA,R0 *PCURSL
        BSTA,UN MSLICE
        LODA,R2 *PSAV2
RXN1:   LODA,R1 *PRXOFF
        LODA,R0 FNRDATA,R1      ; the byte
        STRZ    R3
        ADDI,R1 1
        STRA,R1 *PRXOFF
        BCFR,EQ RXN2            ; wrapped to 0: next slice
        LODA,R0 *PRXSLC
        ADDI,R0 1
        STRA,R0 *PRXSLC
```

`MSLICE` is `FNSLICE` of Section 4 written for block 2, where the mailbox is in reach:

2650 ASSEMBLY (AS) · MAILBOX.INC

```
MSLICE: LODI,R1 FRRXSLC
        LODA,R0 FNRSEL,R1
        LODA,R0 FNRDAT,R2
        LODI,R3 64
MSLO:   LODI,R1 0
MSLI:   LODA,R0 FNSECHO
        COMZ    R2
        BCTR,EQ MSLOK
        BDRR,R1 MSLI
        BDRR,R3 MSLO
        LODI,R0 FETIMO
        RETC,UN
MSLOK:  EORZ    R0
        RETC,UN
```

The chain that reaches it is the program's deepest: drawing a seat prints a name from the reply, which crosses a slice. The README audits it at five return addresses; `depthcheck.py` agrees:

```
GLREQ -> RENDP -> SEATN2 -> RXPRNT -> RXNEXT -> MSLICE
```

IV. BELIEVING THE REPLY

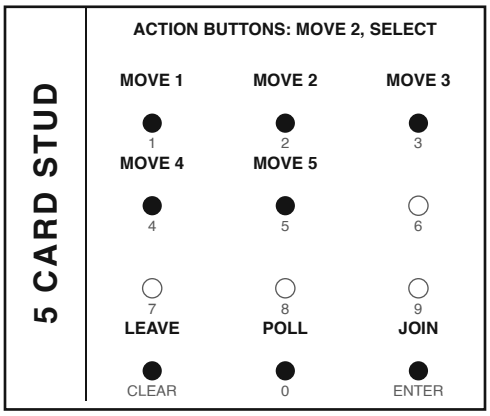
The reply is only believed when its length is exactly what the player count says it should be; a short one would leave the fixed offsets reading what the last reply left in the window:

2650 ASSEMBLY (AS) · NET.INC

```
VALGM:  LODA,R0 FNRDATA+GOPLCNT
        COMI,R0 9
        BCFR,LT VALBAD          ; more seats than the server has
        RRL,R0                  ; pc*2: table index
        STRZ    R1
        LODA,R0 EXPTB,R1
        COMA,R0 *PRXL1
        BCFR,EQ VALBAD
        LODA,R0 EXPTB+1,R1
        COMA,R0 *PRXL0
        BCFR,EQ VALBAD
```

The table of expected lengths is `DWBE` words, read big-endian, because 33 × the player count does not fit in eight bits.

V. THE CONTROLS



The keypad's 1 to 5 choose a move from the server's menu, checked against it before it is sent. The Action buttons are the 2's wire, so in a hand they are strictly "move 2"; on the other screens they select. The disc moves through lists and spins the name wheel, and 4 and 6 move the name cursor.

VI. ON THE BENCH

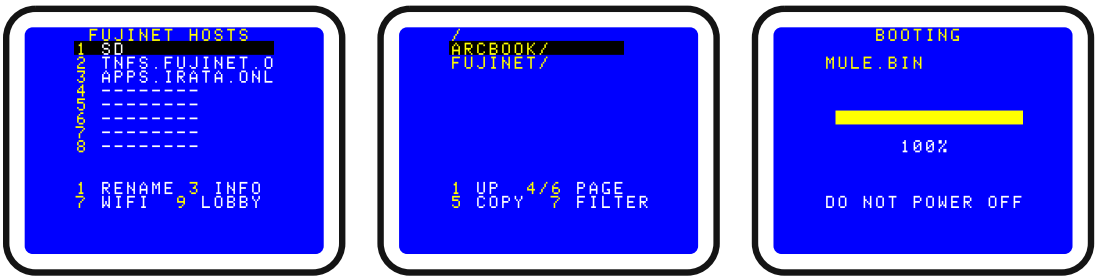
`listings/games.sh` builds the client with `ENDPOINT=http://127.0.0.1:8793/`, and `emu/games.sh` walks it in real time: the name, the first table (AI ROOM, two seats, a robot in the other), and the hand, until the move menu shows **1 FOLD**.

23 CONFIG

HOST THE CARTRIDGE **FILE firmware/baked/fujiconfigrom.h**

I. GENERAL

CONFIG is the program the cartridge serves when it is switched on: WiFi, the eight hosts and their names, the Game Lobby, a browser with paging, folders, a filter and a scrollbar, copying a file between hosts, the adapter's details, and the boot of what you choose, with a progress bar. The Arcadia's is `fujinet-config/arcadia/`, 2650 assembly, and the firmware bakes its 8,192 bytes into the game chip's flash (`pico/arcadia/firmware/baked/fujiconfigrom.h`). `listings/games.sh` builds it from its pin and checks it against the baked header byte for byte.



THE HOSTS THE MEMORY CARD BOOTING

FIG. 14
CONFIG, FROM FUJINET-CONFIG AT ITS PIN

II. THE BUDGET

A client has 6,912 bytes. The first complete CONFIG came to 7,115, and two features went (the long-name marquee and a byte counter during the boot); what is left is 3,955 bytes in block 1 and 2,546 in block 2, with 411 to spare. Three things made it fit, all of them Section 8's.

Zero-page vectors. The 30 routines CONFIG calls or jumps to most often, counted from the source, have pointer words at `$0004-$003F`, where every other program leaves the bytes between the header and the code empty. A `ZBSR *ZPRS` is a two-byte `BSTA,UN PRS` from either block:

2650 ASSEMBLY (AS) · CONFIG.ASM

```
; ---- zero-page vectors -----------------------------------------------------
; ZBSR / ZBRR reach $0000-$003F from ANY page in two bytes, and their
; indirect form jumps through a pointer word there -- so this table
; turns the 30 most-used unconditional calls and jumps (counted from the
; source) into 2-byte instructions: ZBSR *ZPRS is "BSTA,UN PRS".
ZPRS:    DWBE    PRS
ZHOSTPG: DWBE    HOSTPG
ZMPARB:  DWBE    MPARB
ZWAITVB: DWBE    WAITVB
ZMSTART: DWBE    MSTART
```

That saved about 180 bytes.

The split. Block 1 holds the screen, the RAM and the controller; block 2, in the mailbox's page, the transport, the Fuji command wrappers, the screens that mostly talk to the cartridge, and every table and string. Code in block 2 reads the hotspots directly – `LODA,R0 FNRSEL+FRDEVIC` – and so is smaller than the runtime's pointer-word version of the same routine:

2650 ASSEMBLY (AS) · MAILBOX.INC

```
MSTART: LODA,R0 FNRSEL+FRDATRS
        LODA,R0 FNRDAT
        LODA,R0 FNRSEL+FRDEVIC
        LODA,R0 FNRDAT+FNDFUJI
        LODA,R0 FNRSEL+FRCMD
        LODA,R0 FNRDAT,R3
        LODA,R0 FNRSEL+FRNPARM
        LODA,R0 FNRDAT,R2
        RETC,UN
```

Data as data. The boot stub, which must run from RAM, is kept in block 2 as raw `DB` bytes, because AS will not assemble its page-0 operand there.

III. THE RAM

CONFIG uses all 292 bytes, and its ledger at the head of `config.asm` names every one (Section 6). The lower screen holds, laid over one another, the path being browsed, the WiFi name and password and the host name being edited, the filter, the copy's source, a stack of parent folders and the editor's state.

IV. ABANDONING FRAMES

Every one of CONFIG's screens is entered with a branch, from wherever the program happens to be:

2650 ASSEMBLY (AS) · CONFIG.ASM

```
; Every state is entered with BCTA on whatever the return stack holds:
; the 2650's 8-entry stack wraps, so an abandoned frame costs nothing,
; and tools/checkdepth.py bounds every live call chain.
```

A failure deep in the boot jumps straight back to the browser's redraw, leaving the calls that led to it on the return stack. On the 2650 that is safe: the eight-entry stack is a ring, and the frames left behind are simply overwritten, as long as the screen they jump to never returns. The handbook's `depthcheck.py` sees those jumps, says so, and counts the deepest live chain at six routines – five return addresses, from the boot through re-reading the chosen entry to a commit.

V. THE BOOT

CONFIG boots as `fujiboot` does (Section 13), with one difference that matters on real hardware: it does not give up after a fixed time while the image is coming. `BCOMMIT` waits for the MOUNT_IMAGE reply and repaints the progress bar from BOOT_PCT while it waits, and only times out after a spell with no progress at all:

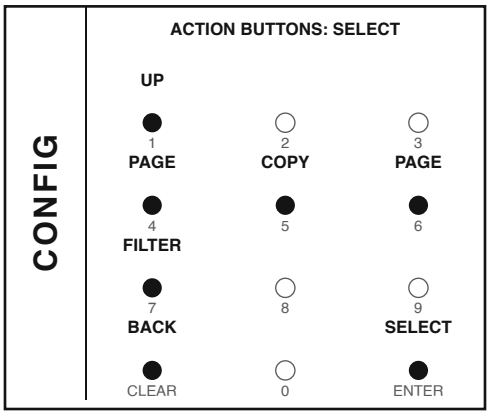
2650 ASSEMBLY (AS) · BOOT.INC

```
BCOMMIT: LODI,R0 $FF
        STRA,R0 *PVT2           ; the percent last drawn
        EORZ    R0
        STRA,R0 *PVT4           ; frames without progress, HI,LO
        STRA,R0 *PVT5
        BSTA,UN MSEQ
        STRA,R0 *PVT3           ; the ACKSEQ that means "done"
BCL:    LODA,R0 FNACKSQ
        COMA,R0 *PVT3
        BCTA,EQ BCACK
        LODA,R0 FNBPCT
```

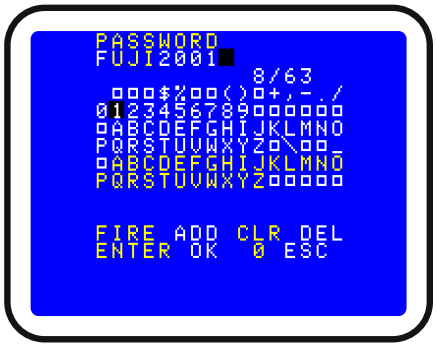
```
        COMA,R0 *PVT2
        BCTR,EQ BCL1
        STRA,R0 *PVT2           ; progress: repaint, restart the clock
        ZBSR    *ZBARPCT
        EORZ    R0
        STRA,R0 *PVT4
        STRA,R0 *PVT5
BCL1:   ZBSR    *ZWAITVB
        LODA,R0 *PVT5
        ADDI,R0 1
        STRA,R0 *PVT5
        BCFR,EQ BCL2
        LODA,R0 *PVT4
        ADDI,R0 1
        STRA,R0 *PVT4
BCL2:   LODA,R0 *PVT4
        COMI,R0 BTIMO>>8
        BCFR,EQ BCL
        LODA,R0 *PVT5
        COMI,R0 BTIMO&$FF
        BCFR,EQ BCL
        LODI,R0 FETIMO
        RETC,UN
```

The stub is the runtime's eleven bytes, and the verdict is read only after the reply is in, so a FAILED left over from an earlier mount is never mistaken for this one's. An image the cartridge will not take shows ROM TOO BIG.

VI. THE CONTROLS



The disc moves the bar and the Action buttons choose, everywhere; the keys on each screen are named on its bottom two rows. The overlay is the browser's: 1 up a folder, 4 and 6 a page, 5 copy, 7 the filter. On the host list 1 renames, 3 is INFO, 7 the WiFi and 9 the Lobby. The WiFi and the names are typed on a character grid, `edit.inc`: the disc moves over 96 characters, the Action buttons type, CLEAR deletes, ENTER accepts, 0 escapes.



THE CHARACTER GRID



INFO

FIG. 15
TWO MORE OF CONFIG'S SCREENS, FROM THE OWNER'S MANUAL

VII. BAKING IT IN

When CONFIG changes, the firmware takes the new image in two steps: fujinet-config's `arcadia/build.sh` writes `build/config.bin`; then, in the firmware, `pico/arcadia/build-cart.sh --bake` turns it into `firmware/baked/fujiconfigrom.h`, which the game chip's build compiles in. Until it is baked, a CONFIG under development runs like any other program: `emu/run.sh bench/run/src/config/arcadia/build/config.bin`.

24. THINGS THAT COST REAL TIME

Every one of these cost hours during the Arcadia's bring-up, its ports and this book. They are here so that they cost you less.

THE EMULATOR IS NOT THE CONSOLE

A transaction runs inside one read. In MAME, the read that writes SEQ does the whole transaction before the 2650's next instruction, so a program that waits too little works there and fails on a cartridge, where the network chip may take 90 seconds. Wait as the cartridge does (`FNGO`), or as CONFIG does, for as long as progress keeps coming.

The tick stands still. `$2C0E` is emulated time: it does not move across a transaction, and in an unthrottled run it races. Timing measured with it in MAME says nothing about a console.

MAME counts whole seconds. `machine.time.seconds` in MAME's Lua is an integer, so a test script written in fractions of a second presses a key and lets it go in the same frame, and the program never sees it. Count frames: `emu/walk.lua` does everything in them.

Unthrottled, it outruns the network. With `-nothrottle` a minute of console time passes in a fraction of a real second, so a program that waits in frames for a server gives up before the server has answered. Anything that talks to a live server – NetCat, the games – is tested with `THROTTLE=1`.

A key press is sixteen frames. `walk.lua` holds a key for eight frames and lets it go for eight. A program that auto-repeats a held key from its eighth frame sees one press as two; NetCat repeats from the sixteenth.

The bus has no timing. The game chip's two-sample rule for the address lines, and the console's 3.35 *µ*s cycle, are not in MAME's model. The cartridge's host tests cover them; a program cannot.

THE BENCH

One MAME to one fujinet-pc. Two consoles connected to one adapter's BoIP port interfere with each other. Run one at a time, and set `FUJINET_TCP` every time: unset, it means `127.0.0.1:9995`, which may be another rig's.

`pgrep -f` finds itself. A pattern such as `mame arcadia` is also in the command line of the shell that runs the `pgrep`. Kill by the pid the bench wrote down (`bench/down.sh` does), never by a pattern.

fujinet-pc is always on WiFi. It reports a network called `Dummy Cafe`, whatever the computer is connected to, so CONFIG's WiFi screens can only be tested with a build that pretends otherwise.

Servers keep their tables. A second test against the same game server can join the table the first left half played. `emu/games.sh` restarts the server before each walk. And Battleship's READY toggles: press it once.

THE ASSEMBLER

Parenthesise macro arguments inside the macro: `(adr)>>8`. AS substitutes the text.

Macros before their callers. AS resolves a label used before it is defined, but not a macro.

`&&` binds tighter than a comparison in AS's expressions: write `(A >= 1) && (B < 2)`.

`DW` is little-endian, and the 2650 reads a pointer high byte first: `DWBE` for every pointer word.

`p2bin -s` writes a checksum into the image. The build scripts never pass it.

THE 2650

`EORZ R1` does not clear R1. It sets R0 to R0 XOR R1.

Indexed loads and stores move R0. `LODA,R1 TAB,R2` does not exist.

A pointer word is page-local. The address it holds is 15 bits, but the word itself must be in the instruction's own page.

CC after a call is the callee's last. Follow a call whose result you test with `COMI,R0 0`, unless its comment promises CC.

`BCTA` against `BCFA`. "Branch on condition true" and "false": one letter, and NetGet threw away every good reading until it was the right one.

The return stack is eight deep and silent. `checkdepth.py`'s figure is a lower bound; `depthcheck.py` follows branches and fall-through, and agrees with the games' own hand audits.

Interrupts. The runtime starts with the upper status byte clear, which leaves interrupts enabled and trusts the `RETC` guard at `$0003`; CONFIG starts with `$20`, interrupts inhibited. Nothing on the ARCADIA-2001 interrupts the FujiNet programs, but `$20` costs nothing.

THE 2637

The pots are good only in the vertical blank. Outside it the left disc reads `$FF`: "down", for ever.

The multiplexer shares a register. Bit 6 of `$19F9` lives with the colours and the character height; rewrite the register from a saved base.

`$C0` is a switch. A blue space turns its row into block graphics.

`DINIT` draws 8 × 8 characters, so its 13 rows fill the top half of the picture. For the whole screen, clear bit 7 of `$19F9`.

A sprite is a character. Sprites 0–3 are drawn from user-defined characters `$38`–`$3B`, which `fujidisp.inc` uses for `/ - _ :`.

THE ROM

6,912 bytes. Block 1 fills first. Move strings, then tables, then whole mailbox-heavy modules to block 2; turn the most-used calls into zero-page vectors. After that there is protocol 2.

25. TROUBLE SHOOTING CHECK LIST

Symptom	Cause And Remedy
NO FUJINET CART, or FAILED FE	– The program is not on a FujiNet cartridge: an EPROM, or MAME without `-cartslot fujinet`. – An image without the claim was booted earlier this session and killed the mailbox. Press the cartridge's RESET.
FAILED FF on a console, never in MAME	– The program waited less than the cartridge allows (5, 60 or 90 s) and gave up: MAME answers inside one read. Wait with `FNGO`.
FAILED 15	– The FujiNet said NAK. Check the parameter count and sizes against the card in Sections 17–19: SET_SSID wants all 97 bytes, OPEN_DIRECTORY and SET_DEVICE_FULLPATH exactly 256, OPEN_APPKEY six, reserved byte and all. – The command is one this build refuses (the end of Sections 17 and 18): RANDOM_NUMBER, UNMOUNT_HOST, NET_GET_ERROR and others.
FAILED 01	– The USB link to the network chip is down: on a cartridge, its power or its firmware; in MAME, no fujinet-pc on `FUJINET_TCP`.
After the console's RESET, nothing answers	– The program keeps its own sequence counter and wrote a number the cartridge has already answered. Use ACKSEQ + 1 every time (`FNCOMMIT` does).
The text fills only the top half of the screen	– Bit 7 of `$19F9` is set: 8 × 8 characters. `DINIT` sets it; write `$07` after it (`BKINIT`) for 8 × 16.
The rest of a row turns into blocks	– A cell holds exactly `$C0`, a space with colour bits `11`. Paint spaces with other colour bits.
Wrong colours after reading the disc	– The pot multiplexer was written without the rest of `$19F9`. Keep the register's base value and OR bit 6 into it.
A menu scrolls down by itself	– The left disc is read outside the vertical blank, where it reads `$FF`. Wait with `TPSU $80` first.
A slash or a dash prints as a sprite's shape	– The program uses sprites, whose images are characters `$38`–`$3B`, and prints with `fujidisp.inc`'s `/ - _` : there.
The program returns somewhere it never called from	– More than eight calls deep. `depthcheck.py` counts what `checkdepth.py` misses: branches and fall-through into a routine's own labels.
AS: "target not on same page"	– A load, store or ALU instruction names an address in the other 8K page. Go through a pointer word (Section 8), or move the code.
A pointer goes to the wrong address	– It was written with `DW`, low byte first. Use `DWBE`.
`EORZ R1` did not clear R1	– It acts on R0. Use `LODI,R1 0`.
A folder shows without its `/`	– READ_DIR_ENTRY was asked for fewer than 36 characters.
A boot ends FAILED, BOOT_ERR 3	– The image is over 8K and not 8,192 + *k* × 2,048 bytes with the claim.

Symptom	Cause And Remedy
A protocol 2 program starts in the wrong page after RESET	– RESET keeps the page. Select page 0 in the start code.
A test script's keys are never seen	– It counts MAME's `machine.time.seconds`, which is whole seconds. Count frames.
A test against a server gives up early	– MAME is unthrottled and burns the program's waits in no real time. Run with `THROTTLE=1`.
Two test runs disturb each other	– Two MAMEs on one fujinet-pc. Run one at a time, and stop strays by pid.
A program cannot get back to CONFIG	– No ARCADIA-2001 program can. Press the cartridge's RESET, then the console's.

WHERE THE SOURCES DISAGREE

Where	What
`pico/arcadia/tools/checkdepth.py`	counts calls only, one node a label, so a fall-through (FNCOMMIT into FNCM1) or a branch (CHKACK to FAIL) runs at a depth it does not count. Its figure is a lower bound: `fujiboot` 3 where it is 4. Both game READMEs audit by hand for this reason.
`testrom/fujidisp.inc`, `DINIT`	writes `$87`, "8x8 chars": right, but the 13 rows then fill only the top half of the picture, unlike CONFIG's screens.
`testrom/fujidisp.inc`, its head	"never emit color bits 11": only `$C0` itself, colour bits `11` and glyph 0, switches the row; blue on any other glyph is safe, and 5 Card Stud uses it.
`testrom/*.asm`, `START`	clears the upper status byte, leaving interrupts enabled behind the `$0003` guard; CONFIG sets `$20`, inhibited.
`testrom/fujilib.inc`, `FNCOMMIT`	waits about 6 s, CONFIG's `MCOMMIT` 12, and the cartridge allows 60 or 90 for a mount or a network command.
the Arcadia Battleship and 5 Card Stud	do not read the shared player-name app key; they ask for a name each time.
`include/fujiCommandID.h`	names RANDOM_NUMBER (`$D3`), which the RS-232 build does not answer.

26. PARTS LIST

Every program, figure and fact in this book comes from a source at a fixed revision. `listings/VENDOR` names each file the book copies, and `tools/vendorcheck.py` compares each with its source.

THE CARTRIDGE AND THE ADAPTER

Board	FujiNet-Arcadia Rev0 (fujinet-hardware, `Emerson-Arcadia/FujiNet-Arcadia-Rev0`)
Game chip	RP2354A
Network chip	ESP32-S3
Cartridge firmware, runtime, tools, MAME device	fujinet-firmware `pico/arcadia`, `pico/common`, at `e3923e881` (pico/arcadia last changed `193d18f53`)
Adapter firmware, command dispatch	fujinet-firmware RS-232 build at `e3923e881`; the bench's fujinet-pc is `f0b62cb09`, whose dispatch is the same
The mailbox's definition	`pico/arcadia/firmware/include/fuji_mailbox.h`, `pico/common/include/fuji_mailbox_proto.h`

THE PROGRAMS

CONFIG	fujinet-config `arcadia/` at `6f0a60a`
Battleship	fujinet-battleship `arcadia/` at `3f2a548`
5 Card Stud	fujinet-5cardstud `arcadia/` at `d5db857`
Game servers	FujiNetWIFI/servers `fujinet-game-system` at `0945622`
This book's programs	`listings/asm/`: `pads`, `netget`, `netcat`, `fujidir`, `appkey`, `clock`, `sprites`; and the runtime's `hello`, `fujitest`, `fujiboot`, `fujibank`

THE TOOLS

Assembler	Macroassembler AS 1.42 Beta, build 306
Emulator	MAME `533800328a2` with the cartridge grafted in by `pico/arcadia/emu/apply.sh`
Typesetting	Typst 0.14.2
Fonts	Helvetica; Harry Obese; Nimbus Sans Narrow Bold; Source Code Pro

THE CHECKS

Check	Holds the book to
`snipcheck.py`	every 2650 example in the text assembles, inside a program, and builds an image
`cmdcheck.py`	every command the adapter dispatches has a card in Sections 17–19, or is in a NAK list
`cartcheck.py`	every `FN_R_`, `FN_H_`, `FN_REG_`, `FN_HOT_` and `FN_BOOT_` name in the mailbox headers is in Section 20
`defcheck.py`	`fujilib.inc` and `bookdefs.inc` agree with the firmware's headers
`vendorcheck.py`	every vendored file is what its source holds at its pin
`depthcheck.py`	no program goes more than eight calls deep, counting branches and fall-through
`widows.py`	no stranded head, widow or orphan
`fontcheck.py`	no font but the book's four

RELATED PUBLICATIONS

– *ARCADIA-2001 Owner's Manual* (Emerson, 1982), model ADV-1: the console.
– *FujiNet Owner's Manual for the EMERSON ARCADIA-2001*: the cartridge, CONFIG and every game, for the player.
– *The FujiNet Network Protocol Handbook*: everything a devicespec can say.
– The Signetics *2650 Microprocessor* manual: the instruction set.
– The Signetics *2637 Universal Video Interface* data sheet: the video chip.

A. MAILBOX QUICK REFERENCE

HOTSPOTS: A READ SENDS

Read	Sends
`$2D00`+*n*	REGSEL: arm register *n*
`$2E00`+*v*	REGDATA: the armed register = *v*
`$2F00`+*b*	TX: append *b*
`$2D80`+*n*	protocol 2: page *n* in the window
`$2DFE`	the swap, once armed

REGISTERS

No.	Name	Value
`$00`	DEVICE	`$70` Fuji, `$71`–`$78` N:, `$45` clock
`$01`	CMD	the command
`$02`	NPARAM	0–8
`$05`	DATA_RST	any
`$06`	RXSLICE	0–3
`$10`	SEQ	ACKSEQ + 1, never 0
`$11`	BOOTLOCK	`$B5`
`$12`/ `$13`	BOOTSEL	`$B5`, then `$4A`
`$14`–`$17`	RAMPTR	`0x000`–`0x300` + *v*
`$18`	RAMPOKE	then one REGDATA a byte

STATUS PAGE

At	Holds
`$2C00`	ACKSEQ, painted last
`$2C01`	bit 0: the link
`$2C02`	ERR: 0 ok
`$2C03`	REPLY: `$06` ACK, `$15` NAK
`$2C04`	RXLEN, low, high
`$2C06`	BOOT_STATE: 0 1 2 `$80`
`$2C07`	BOOT_PCT
`$2C08`	BOOT_ERR
`$2C09`	`'F'`, `'N'`
`$2C0B`	version: 2
`$2C0C`	SLICE_ECHO
`$2C0E`	tick, low, high
`$2B00`	the reply, 256 at a time
`$2CFC`	`FUJI`: the claim

THE RECIPE

1. `FNCHECK`: R0 = 0 on a FujiNet cartridge.
2. `FNBEGIN dev, cmd, nparam`.
3. Parameters: `FNPARB` (R2), `FNPARW` (R2, R3). Payload: `FNTXSTR`, `FNPATH` (256), `FNTXN` (R2 bytes).
4. `FNGO`: R0 = 0 on an ACK.
5. The reply at `$2B00`; `FNSLICE` for the rest.

THE RULES

– The next SEQ is ACKSEQ + 1, never a counter of your own.
– Wait as long as the cartridge may: 5 s, 60 for a mount or a copy, 90 for a network command. Never relaunch after a timeout of your own.
– Page-0 code reaches the mailbox only through `DWBE` pointer words; block-2 code reads it directly.
– Never store into the cartridge's space.
– An image without `FUJI` at `$2CFC` kills the mailbox when it boots.

B. ERROR AND STATUS CODES

ERR, $2C02

Value	Name	Means
0	FN_ERR_OK	the transaction went through; REPLY says what the FujiNet thought of it
1	FN_ERR_NOLINK	the USB link to the network chip is down
2	FN_ERR_TIMEOUT	the network chip did not answer in time (Section 20)
3	FN_ERR_BADFRAME	the parameters were malformed: a size other than 1, 2 or 4
4	FN_ERR_TOOBIG	the request or the reply did not fit

THE LIBRARIES' OWN

Value	Name	Means
$FF	FETIMO	FNCOMMIT or FNGO waited and saw no answer
$15	FCNAK	FNGO: the FujiNet said NAK
$FE	FENOCRT	FAIL's code: no FujiNet cartridge

fujiboot shows ERR *c dd*: *c* is the step (1 MOUNT_HOST, 2 SET_DEVICE_FULLPATH, 3 MOUNT_IMAGE, 4 staging) and *dd* the ERR, BOOT_ERR, or EE for a NAK.

REPLY, $2C03

$06 ACK, $15 NAK, 0 when there was no reply at all (ERR is not 0).

BOOT_STATE, $2C06, AND BOOT_ERR, $2C08

State		Error	
0	idle	1	too big: over 71,680 bytes
1	coming	2	cut short
2	ready	3	not a size the cartridge can map
$80	failed	4	the store is busy

A NETWORK CHANNEL'S STATUS

The fourth byte: 1 when all is well, 136 at the end of the file; anything else is the protocol's error, and an HTTP error page is still readable.

GET_WIFISTATUS

3 connected; anything else, not yet.

C. PROGRAM LISTINGS

The handbook's programs and libraries, then the runtime they are built on, exactly as `listings/` holds them.

BOOKDEFS.INC `asm/bookdefs.inc`

```
; bookdefs.inc -- the devices and commands the runtime's fujilib.inc does
; not name, for this handbook's programs. Every equate's comment carries
; the firmware's own name (include/fujiDeviceID.h, fujiCommandID.h, at the
; book's pin); tools/defcheck.py compares the values. Kept apart so the
; runtime stays verbatim. Include after fujilib.inc.

; ---- devices ------------------------------------------------------
DVDISK  EQU     $31             ; DISK (D1:; D2:-D8: are $32-$38)
DVPRN   EQU     $40             ; PRINTER
DVCLOCK EQU     $45             ; CLOCK
DVMODEM EQU     $50             ; SERIAL
DVNET   EQU     $71             ; NETWORK (N1:; N2:-N8: are $72-$78)

; ---- FUJI ($70) commands fujilib.inc does not name ----------------
FCRESET EQU     $FF             ; RESET
FCGSSID EQU     $FE             ; GET_SSID
FCSCAN  EQU     $FD             ; SCAN_NETWORKS
FCSCANR EQU     $FC             ; GET_SCAN_RESULT
FCSSSID EQU     $FB             ; SET_SSID
FCWIFIS EQU     $FA             ; GET_WIFISTATUS
FCWHSL  EQU     $F3             ; WRITE_HOST_SLOTS
FCRDSL  EQU     $F2             ; READ_DEVICE_SLOTS
FCWDSL  EQU     $F1             ; WRITE_DEVICE_SLOTS
FCWIFIE EQU     $EA             ; GET_WIFI_ENABLED
FCUIMG  EQU     $E9             ; UNMOUNT_IMAGE
FCACFG  EQU     $E8             ; GET_ADAPTERCONFIG
FCNEWD  EQU     $E7             ; NEW_DISK
FCUHOST EQU     $E6             ; UNMOUNT_HOST
FCGDPOS EQU     $E5             ; GET_DIRECTORY_POSITION
FCSHPFX EQU     $E1             ; SET_HOST_PREFIX
FCGHPFX EQU     $E0             ; GET_HOST_PREFIX
FCAKWR  EQU     $DE             ; WRITE_APPKEY
FCAKRD  EQU     $DD             ; READ_APPKEY
FCAKOP  EQU     $DC             ; OPEN_APPKEY
FCAKCL  EQU     $DB             ; CLOSE_APPKEY
FCGDFP  EQU     $DA             ; GET_DEVICE_FULLPATH
FCCBOOT EQU     $D9             ; CONFIG_BOOT
FCCOPY  EQU     $D8             ; COPY_FILE
FCMALL  EQU     $D7             ; MOUNT_ALL
FCSBOOT EQU     $D6             ; SET_BOOT_MODE
FCRAND  EQU     $D3             ; RANDOM_NUMBER
FCB64EI EQU     $D0             ; BASE64_ENCODE_INPUT
FCB64EC EQU     $CF             ; BASE64_ENCODE_COMPUTE
FCB64EL EQU     $CE             ; BASE64_ENCODE_LENGTH
FCB64EO EQU     $CD             ; BASE64_ENCODE_OUTPUT
FCB64DI EQU     $CC             ; BASE64_DECODE_INPUT
FCB64DC EQU     $CB             ; BASE64_DECODE_COMPUTE
FCB64DL EQU     $CA             ; BASE64_DECODE_LENGTH
FCB64DO EQU     $C9             ; BASE64_DECODE_OUTPUT
FCHSHI  EQU     $C8             ; HASH_INPUT
FCHSHC  EQU     $C7             ; HASH_COMPUTE
FCHSHL  EQU     $C6             ; HASH_LENGTH
FCHSHO  EQU     $C5             ; HASH_OUTPUT
FCHSHN  EQU     $C3             ; HASH_COMPUTE_NO_CLEAR
FCHSHX  EQU     $C2             ; HASH_CLEAR
FCQRO   EQU     $BF             ; QRCODE_OUTPUT
FCQRL   EQU     $BE             ; QRCODE_LENGTH
FCQRE   EQU     $BD             ; QRCODE_ENCODE
FCQRI   EQU     $BC             ; QRCODE_INPUT
FCGUID  EQU     $BB             ; GENERATE_GUID
FCSTAT  EQU     $53             ; STATUS
FCREADY EQU     $00             ; DEVICE_READY

; ---- app keys -----------------------------------------------------
; OPEN_APPKEY's payload is one six-byte struct: the creator (2 bytes,
; low first), the app, the key, the mode, and a reserved 0 that is not
; optional.
AKMREAD EQU     0               ; mode: read
AKMWRIT EQU     1               ; mode: write
AKCREAT EQU     $0001           ; the shared user-name key: creator 1,
AKAPP   EQU     $01             ; app 1, key 0, which every FujiNet game
AKKEY   EQU     $00             ; reads the player's name from

; ---- the clock ($45) ----------------------------------------------
CKGTIME EQU     $93             ; GETTIME
CKISOL  EQU     $49             ; GET_ISO_LOCAL
CKISOU  EQU     $5A             ; GET_ISO_UTC
CKTZLEN EQU     $4C             ; GETTZ_LEN
CKSETTZ EQU     $99             ; SETTZ
CKSETTA EQU     $74             ; SETTZ_ALT
CKTZTIM EQU     $9A             ; GETTZTIME

; ---- hash algorithms (HASH_COMPUTE's parameter) --------------------
HAMD5   EQU     0
```

```
HASHA1  EQU     1
HASH256 EQU     3
HASH512 EQU     4

; ---- the N: device ($71-$78) ---------------------------------------------
NCOPEN  EQU     $4F             ; OPEN
NCCLOSE EQU     $43             ; CLOSE
NCREAD  EQU     $52             ; READ
NCWRITE EQU     $57             ; WRITE
NCSTAT  EQU     $53             ; STATUS
NCPARSE EQU     $50             ; PARSE
NCQUERY EQU     $51             ; QUERY
NCPARSR EQU     $FC             ; SET_PARSER
NCSETP  EQU     $FB             ; SET_PARAMETER
NCTRANS EQU     $54             ; TRANSLATION
NCEOL   EQU     $4C             ; SET_EOL
NCRATE  EQU     $5A             ; SET_INT_RATE
NCSEEK  EQU     $25             ; SEEK
NCTELL  EQU     $26             ; TELL
NCRENAM EQU     $20             ; RENAME
NCDEL   EQU     $21             ; DELETE
NCLOCK  EQU     $23             ; LOCK
NCUNLCK EQU     $24             ; UNLOCK
NCMKDIR EQU     $2A             ; MKDIR
NCRMDIR EQU     $2B             ; RMDIR
NCCHDIR EQU     $2C             ; CHDIR
NCGETCW EQU     $30             ; GETCWD
NCACCPT EQU     $41             ; ACCEPT
NCCCLI  EQU     $63             ; CLOSE_CLIENT
NCRMOTE EQU     $72             ; GET_REMOTE
NCDEST  EQU     $44             ; SET_DESTINATION
NCHTTPM EQU     $4D             ; SET_HTTP_MODE
NCGERR  EQU     $45             ; GET_ERROR
NCUSER  EQU     $FD             ; USERNAME
NCPASS  EQU     $FE             ; PASSWORD
NPNONE  EQU     0               ; parsers: none, JSON, HTML, XML
NPJSON  EQU     1
NPHTML  EQU     2
NPXML   EQU     3
NMREAD  EQU     4               ; OPEN modes: read (HTTP GET)
NMWRITE EQU     8               ;   write (HTTP PUT)
NMRDWR  EQU     12              ;   read and write (GET with headers)
NMPOST  EQU     13              ;   HTTP POST
NTRNONE EQU     0               ; translation: none, CR, LF, CR LF
NTRCR   EQU     1
NTRLF   EQU     2
NTRCRLF EQU     3
NSAVLO  EQU     0               ; STATUS reply: bytes waiting, low
NSAVHI  EQU     1               ;   bytes waiting, high
NSCONN  EQU     2               ;   connected (1) or not
NSDEVST EQU     3               ;   device status: 1 is fine

```

BOOKLIB.INC asm/booklib.inc

```
; booklib.inc -- what this handbook's programs share: beginning a
; transaction on any device, a two-byte parameter, a transaction that
; waits as long as the cartridge may and checks its reply, printing,
; waiting frames, and the failure screen. Include after fujidisp.inc and
; fujilib.inc (macros before their callers).
;
; RAM: zone A ($18D0-$18EF) belongs to the runtime (FSTRP, DSTRP, the
; boot stub). The book's programs keep their own bytes in zone B, the
; lower screen at $1A00-$1AFF, which nothing shows in 13-line mode.

BKTRY   EQU     $1A00           ; FNGO: sweeps left
BKERR   EQU     $1A01           ; FAIL: the code
DRPOS   EQU     $1A06           ; DREPLY: the first reply byte
DRCNT   EQU     $1A07           ; DREPLY: how many
BKVAR   EQU     $1A08           ; the first byte a program may take

; FNBEGIN dev, cmd, np -- rewind the TX stream and set the device, the
; command and the number of parameters. Clobbers R0,R1,R2.
FNBEGIN MACRO   dev, cmd, np
        LODI,R1 FRDATRS
        LODI,R2 0
        BSTA,UN FNREGWR
        LODI,R1 FRDEVIC
        LODI,R2 (dev)
        BSTA,UN FNREGWR
        LODI,R1 FRCMD
        LODI,R2 (cmd)
        BSTA,UN FNREGWR
        LODI,R1 FRNPARM
        LODI,R2 (np)
        BSTA,UN FNREGWR
        ENDM

; PUTS cell, str -- print a string from a screen cell (row * 16 + column).
; Clobbers R0-R3.
PUTS    MACRO   cell, str
        SETSTR  str
        LODI,R2 ((cell)-1)&$FF
        BSTA,UN DPRINT
        ENDM
```

```
41
42 ; BKINIT -- the display as CONFIG has it: DINIT's 13-line text mode, but
43 ; with 8x16 characters (BGCOL bit 7 clear), so the 13 rows fill the
44 ; picture instead of its top half; then a clear screen. Calls two down
45 ; (DINIT -> DUDCLD). Clobbers R0,R1,R3.
46 BKBG    EQU     $07             ; BGCOL: 8x16, character colour 0, screen 7
47 BKINIT: BSTA,UN DINIT
48         LODI,R0 BKBG
49         STRA,R0 BGCOL
50         BSTA,UN DCLS
51         RETC,UN
52
53 ; FNPARW -- append a 2-byte parameter: R2 the low byte, R3 the high.
54 ; Leaf. Clobbers R0,R1.
55 FNPARW: LODI,R1 2
56         LODA,R0 *FTXP,R1        ; size byte
57         LODA,R0 *FTXP,R2        ; low
58         LODA,R0 *FTXP,R3        ; high
59         RETC,UN
60
61 ; FNTXN -- append the R2 bytes at (FSTRP) to the TX stream, NULs and all
62 ; (R2 = 0: 256). Leaf. Clobbers R0,R1,R2,R3.
63 FNTXN:  LODI,R3 $FF
64 FNTXN1: LODA,R0 *FSTRP,R3,+     ; buf[++R3]
65         STRZ    R1
66         LODA,R0 *FTXP,R1        ; append it
67         BDRR,R2 FNTXN1
68         RETC,UN
69
70 ; FNGO -- commit, wait, and check the reply. FNCOMMIT gives up after about
71 ; 6 s, but the cartridge allows a mount 60 s and a network OPEN, STATUS,
72 ; READ or CLOSE 90 s, so on its timeout keep polling the SAME sequence
73 ; number (never write a new one: that would start a second transaction)
74 ; for 15 more sweeps, about 90 s in all. R0 = 0 on an ACK, the cart's
75 ; FNERR, FETIMO, or FCNAK on a NAK; CC is EQ exactly when R0 is 0.
76 ; Calls two down (FNCOMMIT -> FNREGWR). Clobbers R1,R2,R3.
77 FNGO:   BSTA,UN FNCOMMIT        ; R2 = the SEQ it wrote
78         COMI,R0 FETIMO
79         BCFR,EQ FNGO4           ; answered, or failed, in time
80         LODI,R0 15
81         STRA,R0 BKTRY
82 FNGO1:  LODI,R3 0               ; one sweep: 64K polls of ~90 us
83 FNGO2:  LODI,R1 0
84 FNGO3:  NOP
85         NOP
86         NOP
87         NOP
88         NOP
89         NOP
90         LODA,R0 *FACKP
91         COMZ    R2
92         BCTR,EQ FNGO5           ; the reply is in
93         BDRR,R1 FNGO3
94         BDRR,R3 FNGO2
95         LODA,R0 BKTRY
96         SUBI,R0 1
97         STRA,R0 BKTRY
98         BCFR,EQ FNGO1
99         LODI,R0 FETIMO
100        RETC,UN
101 FNGO5:  LODA,R0 *FERRP
102 FNGO4:  COMI,R0 0
103        BCFR,EQ FNGO7           ; the cart's error: return it
104        LODA,R0 *FREPP
105        COMI,R0 FCACK
106        BCTR,EQ FNGO6
107        LODI,R0 FCNAK
108        RETC,UN
109 FNGO6:  EORZ    R0
110 FNGO7:  RETC,UN
111
112 ; WAITF -- wait R3 frames (R3 = 0: 256). Leaf. Clobbers R3.
113 WAITF:  TPSU    $80             ; until the vertical blank starts
114        BCFR,EQ WAITF
115 WAITF2: TPSU    $80             ; and until it ends
116        BCTR,EQ WAITF2
117        BDRR,R3 WAITF
118        RETC,UN
119
120 ; DREPLY -- print DRCNT bytes of the reply window, from index DRPOS, at
121 ; screen cell R2 + 1 (none when DRCNT is 0). Calls one down (DXLAT).
122 ; Clobbers R0-R3.
123 DREPLY: LODA,R0 DRCNT
124        BCTR,EQ DRP2
125        LODA,R3 DRPOS
126 DRP1:   LODA,R0 *FRDP,R3        ; reply[R3]
127        BSTA,UN DXLAT
128        STRA,R0 SCREEN,R2,+     ; screen[++R2]
129        ADDI,R3 1
130        LODA,R0 DRCNT
131        SUBI,R0 1
132        STRA,R0 DRCNT
133        BCFR,EQ DRP1
134 DRP2:   RETC,UN
135
136 ; FAIL -- branch here (BCTA) with R0 = the code: FAILED and the code on
137 ; the last row, and stop. $FE means no FujiNet cartridge.
```

```
FAIL:   STRA,R0 BKERR
        PUTS    $C0, FLSTR
        LODA,R0 BKERR
        BSTA,UN DHEX            ; after the text: DPRINT left R2 there
FAILH:  BCTR,UN FAILH
FLSTR:  DB      "FAILED ",0
FENOCRT EQU     $FE             ; FAIL's code: no FujiNet cartridge

```

NETLIB.INC asm/netlib.inc

```
; netlib.inc -- the N: device (N1:, $71), for netget and netcat.
;
; The round trip is OPEN, then STATUS until bytes are waiting, then READ
; them, and CLOSE at the end. Every routine returns R0 = 0 on success
; (CC EQ) and the failure otherwise, as FNGO does. Include after
; booklib.inc and bookdefs.inc.

NAVL    EQU     $1A02           ; STATUS: bytes waiting, low
NAVH    EQU     $1A03           ;   and high
NCON    EQU     $1A04           ;   connected
NDST    EQU     $1A05           ;   device status

; NOPEN -- OPEN the devicespec at (FSTRP), mode R2 (NMREAD for an HTTP
; GET, NMRDWR for TCP), no translation. Calls three down (FNGO; a
; BSTA and RETC, not a tail BCTA, so tools/checkdepth.py counts it).
; Clobbers R0-R3.
NOPEN:  STRA,R2 NDST            ; the mode, until the parameters go
        FNBEGIN DVNET, NCOPEN, 2
        LODA,R2 NDST
        BSTA,UN FNPARB          ; mode
        LODI,R2 NTRNONE
        BSTA,UN FNPARB          ; translation
        BSTA,UN FNTXSTR         ; the devicespec: the payload
        BSTA,UN FNGO
        RETC,UN

; NSTAT -- one STATUS: the four reply bytes into NAVL..NDST.
; Calls three down (FNGO). Clobbers R0-R3.
NSTAT:  FNBEGIN DVNET, NCSTAT, 2
        LODI,R2 0
        BSTA,UN FNPARB
        LODI,R2 0
        BSTA,UN FNPARB
        BSTA,UN FNGO
        BCFR,EQ NSTRET
        LODI,R3 3               ; copy the reply: R0 is the only data
NSTCP:  LODA,R0 *FRDP,R3        ; register an indexed load may use
        STRA,R0 NAVL,R3
        BDRR,R3 NSTCP
        LODA,R0 *FRDP           ; [0]: BDRR stops before it
        STRA,R0 NAVL
        EORZ    R0
NSTRET: RETC,UN

; NREAD -- READ R2 (low) R3 (high) bytes into the reply window; at most
; 1024 fit. Calls three down (FNGO). Clobbers R0-R3.
NREAD:  STRA,R2 NDST            ; the count, until the parameter goes
        STRA,R3 NCON
        FNBEGIN DVNET, NCREAD, 1
        LODA,R2 NDST
        LODA,R3 NCON
        BSTA,UN FNPARW
        BSTA,UN FNGO
        RETC,UN

; NWRITE -- WRITE the R2 bytes at (FSTRP): one 2-byte parameter, the
; count, then the bytes as the payload. Calls three down (FNGO).
; Clobbers R0-R3.
NWRITE: STRA,R2 NDST
        FNBEGIN DVNET, NCWRITE, 1
        LODA,R2 NDST
        LODI,R3 0
        BSTA,UN FNPARW
        LODA,R2 NDST            ; the bytes: the payload
        BSTA,UN FNTXN
        BSTA,UN FNGO
        RETC,UN

; NCLOSE -- CLOSE. Calls three down (FNGO). Clobbers R0-R3.
NCLOSE: FNBEGIN DVNET, NCCLOSE, 0
        BSTA,UN FNGO
        RETC,UN

```

PADS.ASM asm/pads.asm

```
; pads.asm -- both hand controllers, the console's buttons and the tone.
;
; Every frame, in the vertical blank: both keypads drawn live (a pressed
; key shows as a solid block), both discs read through the pot
; multiplexer -- vertical on one frame, horizontal on the next, each kept
; -- and the console's START, OPTION and SELECT. Each key plays its own
```

```
; tone while it is held. The last key, direction and button stay on the
; bottom rows. No FujiNet: this is the console alone.
;
; Build: ./build.sh pads

        CPU     2650

FRAME   EQU     BKVAR           ; counts frames: bit 0 picks the pot axis
DV1     EQU     BKVAR+1         ; the pots: left disc vertical, horizontal
DH1     EQU     BKVAR+2
DV2     EQU     BKVAR+3         ; right disc
DH2     EQU     BKVAR+4
LASTK   EQU     BKVAR+5         ; the last key's glyph (0: none yet)
ANYK    EQU     BKVAR+6         ; a key is down this frame
LDIR    EQU     BKVAR+8         ; the left disc's direction this frame
BGBASE  EQU     BKBG            ; BGCOL as BKINIT sets it

        ORG     $0000
        BCTA    UN,START        ; cartridge header: jump past the
        DB      $17             ; RETC,UN interrupt guard at $0003

        ORG     $0020
        INCLUDE "fujidisp.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "bookdefs.inc"
        INCLUDE "booklib.inc"

START:  EORZ    R0
        LPSU
        LODI,R0 $02             ; COM=1: logical compares
        LPSL
        BSTA,UN BKINIT
        PUTS    $00, TITLE
        PUTS    $20, THEAD
        PUTS    $80, TDISC
        PUTS    $90, TTONE
        PUTS    $A0, TKEY
        PUTS    $B0, TLDISC
        PUTS    $C0, TLAST
        EORZ    R0
        STRA,R0 FRAME
        STRA,R0 LASTK

; ---- once a frame, in the vertical blank -----------------------------------
MAIN:   TPSU    $80             ; wait for the vertical blank
        BCFR,EQ MAIN
        LODA,R0 FRAME           ; the pots: valid only now
        ANDI,R0 1
        BCFR,EQ MHORZ
        LODA,R0 $19FF           ; mux clear: the vertical axes
        STRA,R0 DV1
        LODA,R0 $19FE
        STRA,R0 DV2
        LODI,R0 BGBASE|$40      ; the next frame reads horizontal
        STRA,R0 BGCOL
        BCTR,UN MKEYS
MHORZ:  LODA,R0 $19FF           ; mux set: the horizontal axes
        STRA,R0 DH1
        LODA,R0 $19FE
        STRA,R0 DH2
        LODI,R0 BGBASE          ; the next frame reads vertical
        STRA,R0 BGCOL
MKEYS:  LODA,R0 FRAME
        ADDI,R0 1
        STRA,R0 FRAME
        BSTA,UN KEYS
        BSTA,UN TONE
        BSTA,UN DISCS
        BSTA,UN PANEL
MWAIT:  TPSU    $80             ; and for it to end
        BCTR,EQ MWAIT
        BCTA,UN MAIN

; ---- the keypads: 24 keys from a table -------------------------------------
; KTAB, 4 bytes a key: the column's offset from $1900, the key's bit, its
; screen cell, its label's glyph.
KEYS:   EORZ    R0
        STRA,R0 ANYK
        LODI,R3 0
KEY1:   LODA,R0 KTAB,R3         ; the column
        STRZ    R1
        LODA,R0 $1900,R1
        ANDA,R0 KTAB+1,R3       ; this key's bit
        STRZ    R2              ; R2: down or not
        LODA,R0 KTAB+2,R3       ; its cell
        STRZ    R1
        LODA,R0 KTAB+3,R3       ; its label
        COMI,R2 0
        BCTR,EQ KEY2            ; up: the label
        STRA,R0 LASTK           ; down: remember it, show a block
        STRA,R0 ANYK
        LODI,R0 GBLOCK
KEY2:   STRA,R0 SCREEN,R1
        ADDI,R3 4
        COMI,R3 24*4
        BCFR,EQ KEY1
        LODA,R0 LASTK           ; the last key, on row 10
```

```
104         STRA,R0 SCREEN+$AB
105         RETC,UN
106
107 ; ---- the tone: on while a key is down -------------------------------------
108 ; 1/f = 2 (pitch + 1) line periods: pitch 15 is about 490 Hz.
109 TONE:   LODA,R0 ANYK
110         BCTR,EQ TONE1
111         ANDI,R0 $1F             ; 0-9, C, E: higher keys, higher notes
112         RRL,R0
113         STRZ    R1
114         LODI,R0 $40
115         SUBZ    R1
116         STRA,R0 PITCH
117         LODI,R0 $0F             ; on, loudest
118         BCTR,UN TONE2
119 TONE1:  EORZ    R0              ; off
120 TONE2:  STRA,R0 VOLUME
121         LODA,R0 PITCH
122         LODI,R2 $95-1           ; row 9: the pitch
123         BSTA,UN DHEX
124         RETC,UN
125
126 ; ---- the discs: both, live; the left one's last direction ------------------
127 DISCS:  LODA,R1 DV1
128         LODA,R2 DH1
129         BSTA,UN DIROF
130         STRA,R3 LDIR            ; hold it
131         LODI,R2 $85-1           ; row 8, column 5
132         BSTA,UN PWORD
133         LODA,R1 DV2
134         LODA,R2 DH2
135         BSTA,UN DIROF
136         LODI,R2 $8B-1           ; row 8, column 11
137         BSTA,UN PWORD
138         LODA,R3 LDIR
139         COMI,R3 24              ; centred: keep the last one
140         BCTR,EQ DISCS1
141         LODI,R2 $BB-1           ; row 11, column 11
142         BSTA,UN PWORD
143 DISCS1: RETC,UN
144
145 ; DIROF -- R1 = a vertical pot, R2 = a horizontal one: R3 = the word.
146 ; Leaf.
147 DIROF:  LODI,R3 0               ; up: under $40
148         COMI,R1 $40
149         BCTR,LT DIROF9
150         LODI,R3 6               ; down: over $C0
151         COMI,R1 $C0
152         BCTR,GT DIROF9
153         LODI,R3 12              ; left
154         COMI,R2 $40
155         BCTR,LT DIROF9
156         LODI,R3 18              ; right
157         COMI,R2 $C0
158         BCTR,GT DIROF9
159         LODI,R3 24              ; centred
160 DIROF9: RETC,UN
161
162 ; ---- the console's buttons: the last one, on row 12 --------------------------
163 PANEL:  LODA,R0 $1908
164         ANDI,R0 $07
165         BCTR,EQ PANEL9
166         LODI,R3 30              ; bit 0: START
167         TMI,R0  $01
168         BCTR,EQ PANEL1
169         LODI,R3 37              ; bit 1: OPTION
170         TMI,R0  $02
171         BCTR,EQ PANEL1
172         LODI,R3 44              ; bit 2: SELECT
173 PANEL1: LODI,R2 $C5-1           ; row 12, column 5
174         BSTA,UN PWORD
175 PANEL9: RETC,UN
176
177 ; PWORD -- the word at WORDS + R3, up to its NUL, from cell R2 + 1.
178 ; Calls one down (DXLAT). Clobbers R0-R3.
179 PWORD:  LODA,R0 WORDS,R3        ; a load sets CC
180         BCTR,EQ PWORD9
181         BSTA,UN DXLAT
182         STRA,R0 SCREEN,R2,+
183         ADDI,R3 1
184         BCTR,UN PWORD
185 PWORD9: RETC,UN
186
187 ; five characters a direction (6 bytes with the NUL), six a button
188 WORDS:  DB      "UP   ",0,"DOWN ",0,"LEFT ",0,"RIGHT",0,"     ",0
189         DB      "START ",0,"OPTION",0,"SELECT",0
190
191 ; column, bit, cell, label: left controller at column 1, right at 9
192 KTAB:   DB      0,8,$31,$11,  1,8,$33,$12,  2,8,$35,$13   ; 1 2 3
193         DB      0,4,$41,$14,  1,4,$43,$15,  2,4,$45,$16   ; 4 5 6
194         DB      0,2,$51,$17,  1,2,$53,$18,  2,2,$55,$19   ; 7 8 9
195         DB      0,1,$61,$1C,  1,1,$63,$10,  2,1,$65,$1E   ; C 0 E
196         DB      4,8,$39,$11,  5,8,$3B,$12,  6,8,$3D,$13
197         DB      4,4,$49,$14,  5,4,$4B,$15,  6,4,$4D,$16
198         DB      4,2,$59,$17,  5,2,$5B,$18,  6,2,$5D,$19
199         DB      4,1,$69,$1C,  5,1,$6B,$10,  6,1,$6D,$1E
200
```

```
TITLE:  DB      "PADS",0
THEAD:  DB      " LEFT    RIGHT",0
TDISC:  DB      "DISC",0
TTONE:  DB      "TONE",0
TKEY:   DB      "LAST KEY",0
TLDISC: DB      "LAST DISC",0
TLAST:  DB      "LAST",0

```

NETGET.ASM asm/netget.asm

```
; netget.asm -- fetch a file over HTTP through the N: device, and show it.
;
; OPEN the URL as an HTTP GET; STATUS until the count of bytes waiting is
; not 0 and two readings agree; READ them (at most 192, a screen's worth)
; into the reply window; show them a line at a time (LF ends a row, CR is
; skipped); CLOSE. The URL is the bench's, from build.sh.
;
; Build: ./build.sh netget

        CPU     2650

ROW0    EQU     $20             ; the first row of the file: row 2
NMAX    EQU     192             ; the most we read
NTRIES  EQU     30              ; STATUS readings before giving up
PREVL   EQU     BKVAR           ; the reading before: low, high
PREVH   EQU     BKVAR+1
TRIES   EQU     BKVAR+2
CELL    EQU     BKVAR+3         ; where the next character goes, minus 1
ROWAT   EQU     BKVAR+4         ; this row's first cell

        ORG     $0000
        BCTA    UN,START        ; cartridge header: jump past the
        DB      $17             ; RETC,UN interrupt guard at $0003

        ORG     $0020
        INCLUDE "fujidisp.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "bookdefs.inc"
        INCLUDE "booklib.inc"
        INCLUDE "netlib.inc"
        INCLUDE "bookcfg.inc"

START:  EORZ    R0
        LPSU
        LODI,R0 $02             ; COM=1: logical compares
        LPSL
        BSTA,UN BKINIT
        PUTS    $00, TITLE
        BSTA,UN FNCHECK
        BCTR,EQ OPEN
        LODI,R0 FENOCRT
        BCTA,UN FAIL

; ---- OPEN: the URL, as an HTTP GET ----------------------------------------
OPEN:   SETFSTR URL
        LODI,R2 NMREAD
        BSTA,UN NOPEN
        BCFA,EQ FAIL

; ---- STATUS until two readings agree, and are not 0 -----------------------
        LODI,R0 $FF
        STRA,R0 PREVL
        STRA,R0 PREVH
        EORZ    R0
        STRA,R0 TRIES
STAT1:  BSTA,UN NSTAT
        BCFA,EQ FAIL
        LODA,R0 NAVL            ; the same as the reading before?
        COMA,R0 PREVL
        BCFR,EQ AGAIN
        LODA,R0 NAVH
        COMA,R0 PREVH
        BCFR,EQ AGAIN
        IORA,R0 NAVL            ; and not 0?
        BCTA,EQ AGAIN
        BCTA,UN READ
AGAIN:  LODA,R0 NAVL            ; this reading is the one before
        STRA,R0 PREVL
        LODA,R0 NAVH
        STRA,R0 PREVH
        LODA,R0 TRIES
        ADDI,R0 1
        STRA,R0 TRIES
        COMI,R0 NTRIES
        BCFR,EQ WAIT
        LODI,R0 FETMOUT         ; it never settled
        BCTA,UN FAIL
WAIT:   LODI,R3 3               ; three frames
        BSTA,UN WAITF
        BCTA,UN STAT1

; ---- READ what is waiting, at most NMAX -----------------------------------
READ:   LODA,R2 NAVL
        LODA,R0 NAVH
```

```
         BCFR,EQ RDMAX           ; 256 or more
         LODZ    R2
         COMI,R0 NMAX
         BCTR,LT RDGO            ; under NMAX
RDMAX:   LODI,R2 NMAX
RDGO:    LODI,R3 0
         BSTA,UN NREAD
         BCFA,EQ FAIL

; ---- show it: R3 walks the reply, LF ends a row, CR is skipped -----------
         LODI,R0 ROW0
         STRA,R0 ROWAT
         SUBI,R0 1
         STRA,R0 CELL
         LODI,R3 $FF             ; the reply's index, minus 1
SHOW1:   ADDI,R3 1
         LODZ    R3
         COMA,R0 *FRXLP          ; the reply's length (under 256 here)
         BCTA,EQ CLOSE
         LODA,R0 *FRDP,R3        ; reply[R3]
         COMI,R0 $0D
         BCTR,EQ SHOW1           ; CR: skip it
         COMI,R0 $0A
         BCTR,EQ SHOW2           ; LF: the next row
         BSTA,UN DXLAT           ; ASCII to a glyph (clobbers R1)
         LODA,R2 CELL
         STRA,R0 SCREEN,R2,+     ; screen[++R2]
         STRA,R2 CELL
         BCTR,UN SHOW1
SHOW2:   LODA,R0 ROWAT           ; the next row's first cell
         ADDI,R0 $10
         STRA,R0 ROWAT
         SUBI,R0 1
         STRA,R0 CELL
         BCTR,UN SHOW1

; ---- CLOSE, and say so ---------------------------------------------------
CLOSE:   BSTA,UN NCLOSE
         BCFA,EQ FAIL
         PUTS    $C0, DONE
HANG:    BCTR,UN HANG

URL:     HTTPURL
TITLE:   DB      "NETGET",0
DONE:    DB      "CLOSED",0

```

NETCAT.ASM asm/netcat.asm

```
; netcat.asm -- a TCP line terminal for the Arcadia 2001.
;
; OPEN the bench's line server as N:TCP://, then, once a frame: read the
; left controller, and every sixth frame ask STATUS; whatever is waiting
; is READ and printed in the terminal, rows 0-9, which scrolls. The line
; being typed is row 10; the legend rows 11-12.
;
; The keyboard is a wheel, as an arcade machine takes a high-score name:
; the disc turns the character under the cursor (held, it repeats), the
; Action button types it, CLEAR takes the last one back, ENTER sends the
; line with CR LF, and 0 CLOSEs the connection and stops. The Action
; button is the keypad's 2 (one wire), so the 2 types, too.
;
; Build: ./build.sh netcat

         CPU     2650

TROWS    EQU     10              ; the terminal: rows 0-9
LROW     EQU     $A0             ; row 10: the line being typed
LMAX     EQU     15              ; characters a line
NPOLL    EQU     6               ; frames between STATUS readings
TROW     EQU     BKVAR           ; the terminal's row, column
TCOL     EQU     BKVAR+1
TCH      EQU     BKVAR+2         ; TPUTC: the glyph
WIDX     EQU     BKVAR+3         ; the wheel: which character
LLEN     EQU     BKVAR+4         ; the line's length
HELD     EQU     BKVAR+5         ; frames the disc has been held
PREVK    EQU     BKVAR+6         ; the keys last frame
POLL     EQU     BKVAR+7         ; frames to the next STATUS
RIDX     EQU     BKVAR+8         ; RECV: the reply byte being printed
LBUF     EQU     BKVAR+16        ; the line, then CR LF: 17 bytes
WLEN     EQU     WHEND-WHEEL     ; 45 characters on the wheel

         ORG     $0000
         BCTA    UN,START        ; cartridge header: jump past the
         DB      $17             ; RETC,UN interrupt guard at $0003

         ORG     $0020
         INCLUDE "fujidisp.inc"
         INCLUDE "fujilib.inc"
         INCLUDE "bookdefs.inc"
         INCLUDE "booklib.inc"
         INCLUDE "netlib.inc"
         INCLUDE "bookcfg.inc"

START:   EORZ    R0
```

```
        LPSU
        LODI,R0 $02             ; COM=1: logical compares
        LPSL
        BSTA,UN BKINIT
        EORZ    R0
        STRA,R0 TROW
        STRA,R0 TCOL
        STRA,R0 WIDX
        STRA,R0 LLEN
        STRA,R0 HELD
        STRA,R0 PREVK
        STRA,R0 POLL
        PUTS    $B0, LEG1
        PUTS    $C0, LEG2
        BSTA,UN FNCHECK
        BCTR,EQ OPEN
        LODI,R0 FENOCRT
        BCTA,UN FAIL

OPEN:   SETFSTR URL             ; read and write: a TCP connection
        LODI,R2 NMRDWR
        BSTA,UN NOPEN
        BCFA,EQ FAIL
        BSTA,UN LSHOW

; ---- once a frame ------------------------------------------------------
MAIN:   TPSU    $80             ; the vertical blank: the disc is valid
        BCFR,EQ MAIN
        BSTA,UN INPUT
        LODA,R0 POLL
        SUBI,R0 1
        STRA,R0 POLL
        BCFR,GT MAIN1           ; not yet
        BSTA,UN RECV
        LODI,R0 NPOLL
        STRA,R0 POLL
MAIN1:  TPSU    $80
        BCTR,EQ MAIN1
        BCTA,UN MAIN

; ---- the controller: the wheel, and the keys on their edges ------------
INPUT:  LODA,R0 $19FF           ; the disc, vertical (the mux is clear)
        COMI,R0 $40
        BCTR,LT INUP
        COMI,R0 $C0
        BCTR,GT INDN
        EORZ    R0              ; centred: the next turn is at once
        STRA,R0 HELD
        BCTA,UN INKEYS
INUP:   LODI,R1 WLEN-1          ; up: the character before
        BCTR,UN INTURN
INDN:   LODI,R1 1               ; down: the one after
INTURN: LODA,R0 HELD            ; at once; held, from the 16th frame
        ADDI,R0 1               ; on, every 8th
        STRA,R0 HELD
        COMI,R0 1
        BCTR,EQ INT1
        COMI,R0 16
        BCTA,LT INKEYS
        ANDI,R0 7
        BCFA,EQ INKEYS
INT1:   LODA,R0 WIDX            ; WIDX = (WIDX + R1) mod WLEN
        ADDZ    R1
        COMI,R0 WLEN
        BCTR,LT INT2
        SUBI,R0 WLEN
INT2:   STRA,R0 WIDX
        BSTA,UN LSHOW
INKEYS: LODA,R0 $1901           ; b3: Action (and 2), b0: 0
        ANDI,R0 $09
        STRZ    R1
        LODA,R0 $1900           ; b0: CLEAR
        ANDI,R0 $01
        RRL,R0                  ; to b1
        IORZ    R1
        STRZ    R1
        LODA,R0 $1902           ; b0: ENTER
        ANDI,R0 $01
        RRL,R0
        RRL,R0                  ; to b2
        IORZ    R1
        STRZ    R1              ; R1: the keys now
        LODA,R0 PREVK
        EORI,R0 $FF
        ANDZ    R1              ; R0: the keys just pressed
        STRA,R1 PREVK
        TMI,R0  $08
        BCTA,EQ TYPE
        TMI,R0  $02
        BCTA,EQ DELETE
        TMI,R0  $04
        BCTA,EQ SEND
        TMI,R0  $01
        BCTA,EQ QUIT
        RETC,UN

TYPE:   LODA,R1 LLEN            ; the wheel's character onto the line
```

```
144         COMI,R1 LMAX
145         BCFR,LT TYPE9           ; full
146         LODA,R3 WIDX
147         LODA,R0 WHEEL,R3
148         STRA,R0 LBUF,R1
149         ADDI,R1 1
150         STRA,R1 LLEN
151         BSTA,UN LSHOW
152 TYPE9:  RETC,UN
153
154 DELETE: LODA,R0 LLEN
155         BCTR,EQ DEL9
156         SUBI,R0 1
157         STRA,R0 LLEN
158         BSTA,UN LSHOW
159 DEL9:   RETC,UN
160
161 SEND:   LODA,R1 LLEN            ; the line, CR, LF
162         LODI,R0 $0D
163         STRA,R0 LBUF,R1
164         ADDI,R1 1
165         LODI,R0 $0A
166         STRA,R0 LBUF,R1
167         ADDI,R1 1
168         LODZ    R1              ; R2: the count
169         STRZ    R2
170         SETFSTR LBUF
171         BSTA,UN NWRITE
172         BCFA,EQ FAIL
173         EORZ    R0
174         STRA,R0 LLEN
175         BSTA,UN LSHOW
176         RETC,UN
177
178 QUIT:   BSTA,UN NCLOSE
179         PUTS    $B0, TCLOSED
180         PUTS    $C0, TBLANK
181 QUIT9:  BCTR,UN QUIT9
182
183 ; LSHOW -- row 10: the line, then the wheel's character in magenta, then
184 ; spaces. Calls one down (DXLAT). Clobbers R0-R3.
185 LSHOW:  LODI,R3 0
186 LSH1:   COMA,R3 LLEN
187         BCTR,EQ LSH2
188         LODA,R0 LBUF,R3
189         BSTA,UN DXLAT
190         STRA,R0 SCREEN+LROW,R3
191         ADDI,R3 1
192         BCTR,UN LSH1
193 LSH2:   LODA,R1 WIDX            ; the candidate
194         LODA,R0 WHEEL,R1
195         BSTA,UN DXLAT
196         IORI,R0 $80             ; colour bits 10: magenta
197         STRA,R0 SCREEN+LROW,R3
198 LSH3:   ADDI,R3 1               ; the rest of the row
199         COMI,R3 16
200         BCTR,EQ LSH9
201         EORZ    R0
202         STRA,R0 SCREEN+LROW,R3
203         BCTR,UN LSH3
204 LSH9:   RETC,UN
205
206 ; ---- receiving: STATUS, then READ what is waiting, then print it -----------
207 RECV:   BSTA,UN NSTAT
208         BCFA,EQ FAIL
209         LODA,R0 NAVH
210         BCFR,EQ RECV1           ; 256 or more: take 255
211         LODA,R2 NAVL
212         BCTR,EQ RECV9           ; nothing
213         BCTR,UN RECV2
214 RECV1:  LODI,R2 255
215 RECV2:  LODI,R3 0
216         BSTA,UN NREAD
217         BCFA,EQ FAIL
218         EORZ    R0
219         STRA,R0 RIDX
220 RECV3:  LODA,R0 RIDX
221         COMA,R0 *FRXLP          ; the reply's length (under 256)
222         BCTR,EQ RECV9
223         STRZ    R3
224         LODA,R0 *FRDP,R3        ; reply[RIDX]
225         BSTA,UN TPUTC
226         LODA,R0 RIDX
227         ADDI,R0 1
228         STRA,R0 RIDX
229         BCTR,UN RECV3
230 RECV9:  RETC,UN
231
232 ; TPUTC -- the ASCII character in R0 onto the terminal: CR ignored, LF a
233 ; new line, the rest at the cursor, wrapping at 16. Calls two down
234 ; (TNL -> TSCRL). Clobbers R0-R3.
235 TPUTC:  COMI,R0 $0D
236         BCTR,EQ TPC9
237         COMI,R0 $0A
238         BCTR,EQ TNL
239         BSTA,UN DXLAT           ; to a glyph (clobbers R1)
240         STRA,R0 TCH
```

```
241         LODA,R0 TROW            ; the cell: row * 16 + column
242         RRL,R0
243         RRL,R0
244         RRL,R0
245         RRL,R0
246         ADDA,R0 TCOL
247         STRZ    R1
248         LODA,R0 TCH
249         STRA,R0 SCREEN,R1
250         LODA,R0 TCOL
251         ADDI,R0 1
252         STRA,R0 TCOL
253         COMI,R0 16
254         BCTR,EQ TNL             ; wrap
255 TPC9:   RETC,UN
256 TNL:    EORZ    R0              ; a new line: column 0, the next row
257         STRA,R0 TCOL
258         LODA,R0 TROW
259         ADDI,R0 1
260         COMI,R0 TROWS
261         BCTR,LT TNL1
262         BSTA,UN TSCRL           ; off the bottom: scroll, stay on row 9
263         LODI,R0 TROWS-1
264 TNL1:   STRA,R0 TROW
265         RETC,UN
266
267 ; TSCRL -- rows 1-9 up one, row 9 cleared. Leaf. Clobbers R0,R3.
268 TSCRL:  LODI,R3 0
269 TSC1:   LODA,R0 SCREEN+16,R3
270         STRA,R0 SCREEN,R3
271         ADDI,R3 1
272         COMI,R3 16*(TROWS-1)
273         BCFR,EQ TSC1
274 TSC2:   EORZ    R0
275         STRA,R0 SCREEN,R3
276         ADDI,R3 1
277         COMI,R3 16*TROWS
278         BCFR,EQ TSC2
279         RETC,UN
280
281 URL:    NCURL
282 WHEEL:  DB      "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 .,+$/-_:"
283 WHEND:
284 LEG1:   DB      "DISC PICK 2 TYPE",0
285 LEG2:   DB      "CLR DEL ENT SEND",0
286 TCLOSED: DB     "CLOSED",0
287 TBLANK: DB      "                ",0
288
```

FUJIDIR.ASM asm/fujidir.asm

```
1  ; fujidir.asm -- list a directory on a FujiNet host.
2  ;
3  ; MOUNT_HOST, then OPEN_DIRECTORY with the path (NUL-padded to 256 bytes),
4  ; then READ_DIR_ENTRY until the $7F,$7F end marker or a full screen, each
5  ; name printed straight from the reply window (16 characters at most: the
6  ; row), then CLOSE_DIRECTORY.
7  ;
8  ; Build: ./build.sh fujidir
9
10         CPU     2650
11
12 HOSTSL  EQU     0               ; host slot 0: the SD card
13 MAXLEN  EQU     36              ; never below 36: a folder's '/' needs it
14 ROW0    EQU     $20             ; the first name: row 2
15 NROWS   EQU     10              ; rows 2-11
16 ROWAT   EQU     BKVAR           ; this name's first cell
17 LEFT    EQU     BKVAR+1         ; rows left
18
19         ORG     $0000
20         BCTA    UN,START        ; cartridge header: jump past the
21         DB      $17             ; RETC,UN interrupt guard at $0003
22
23         ORG     $0020
24         INCLUDE "fujidisp.inc"
25         INCLUDE "fujilib.inc"
26         INCLUDE "bookdefs.inc"
27         INCLUDE "booklib.inc"
28
29 START:  EORZ    R0
30         LPSU
31         LODI,R0 $02             ; COM=1: logical compares
32         LPSL
33         BSTA,UN BKINIT
34         PUTS    $00, TITLE
35         PUTS    $10, PATH
36         BSTA,UN FNCHECK
37         BCTR,EQ MOUNT
38         LODI,R0 FENOCRT
39         BCTA,UN FAIL
40
41 ; ---- mount the host, open the directory ----------------------------------
42 MOUNT:  FNBEGIN FNDFUJI, FCMHOST, 1
43         LODI,R2 HOSTSL
44         BSTA,UN FNPARB
```

```
        BSTA,UN FNGO
        BCFA,EQ FAIL
        FNBEGIN FNDFUJI, FCODIR, 1
        LODI,R2 HOSTSL
        BSTA,UN FNPARB
        SETF$TR PATH
        BSTA,UN FNPATH          ; the path, NUL-padded to 256
        BSTA,UN FNGO
        BCFA,EQ FAIL

; ---- READ_DIR_ENTRY until $7F,$7F or the screen is full --------------------
        LODI,R0 ROW0
        STRA,R0 ROWAT
        LODI,R0 NROWS
        STRA,R0 LEFT
NEXT:   FNBEGIN FNDFUJI, FCRDIR, 2
        LODI,R2 MAXLEN
        BSTA,UN FNPARB          ; the longest name we take
        LODI,R2 0
        BSTA,UN FNPARB          ; no extra fields (no icons)
        BSTA,UN FNGO
        BCFA,EQ FAIL
        LODA,R0 *FRDP           ; $7F,$7F: the end
        COMI,R0 $7F
        BCFR,EQ SHOW
        LODI,R1 1
        LODA,R0 *FRDP,R1
        COMI,R0 $7F
        BCTA,EQ DONE
SHOW:   LODA,R2 ROWAT           ; the name, 16 characters at most
        SUBI,R2 1
        LODI,R3 $FF
SHOW1:  LODA,R0 *FRDP,R3,+      ; name[++R3]; a load sets CC
        BCTR,EQ SHOW2           ; its NUL
        BSTA,UN DXLAT
        STRA,R0 SCREEN,R2,+
        COMI,R3 15
        BCFR,EQ SHOW1
SHOW2:  LODA,R0 ROWAT
        ADDI,R0 $10
        STRA,R0 ROWAT
        LODA,R0 LEFT
        SUBI,R0 1
        STRA,R0 LEFT
        BCFA,EQ NEXT

; ---- close the directory ---------------------------------------------------
DONE:   FNBEGIN FNDFUJI, FCCDIR, 0
        BSTA,UN FNGO
        BCFA,EQ FAIL
        PUTS    $C0, CLOSED
HANG:   BCTR,UN HANG

TITLE:  DB      "FUJIDIR",0
PATH:   DB      "/arcbook/",0
CLOSED: DB      "CLOSED",0

```

APPKEY.ASM asm/appkey.asm

```
; appkey.asm -- read the shared FujiNet player name, then write a key of
; our own and read it back.
;
; An app key is a small file on the adapter's memory card, named by a
; creator, an app and a key number. Creator 1, app 1, key 0 holds the
; player's name, which every FujiNet game reads. Ours is creator $2650,
; app $FF (this handbook's scratch key), key 0.
;
; OPEN_APPKEY's payload is one six-byte struct: the creator (low first),
; the app, the key, the mode and a reserved 0. READ_APPKEY's reply is a
; two-byte length, then the value. WRITE_APPKEY takes one parameter, the
; length, then the bytes.
;
; Build: ./build.sh appkey

        CPU     2650

OURCRE  EQU     $2650
OURAPP  EQU     $FF
OURKEY  EQU     0

        ORG     $0000
        BCTA    UN,START        ; cartridge header: jump past the
        DB      $17             ; RETC,UN interrupt guard at $0003

        ORG     $0020
        INCLUDE "fujidisp.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "bookdefs.inc"
        INCLUDE "booklib.inc"

START:  EORZ    R0
        LPSU
        LODI,R0 $02             ; COM=1: logical compares
        LPSL
```

```
         BSTA,UN BKINIT
         PUTS    $00, TITLE
         BSTA,UN FNCHECK
         BCTR,EQ GO
         LODI,R0 FENOCRT
         BCTA,UN FAIL

GO:      PUTS    $20, TNAME      ; the player's name
         SETFSTR KNAME
         BSTA,UN AKOPEN
         BSTA,UN AKREAD
         LODI,R2 $30-1
         BSTA,UN AKSHOW
         PUTS    $60, TOURS      ; ours: write it...
         SETFSTR KOURW
         BSTA,UN AKOPEN
         BSTA,UN AKWRIT
         SETFSTR KOURR           ; ...and read it back
         BSTA,UN AKOPEN
         BSTA,UN AKREAD
         LODI,R2 $70-1
         BSTA,UN AKSHOW
HANG:    BCTR,UN HANG

; AKOPEN -- OPEN_APPKEY: no parameters, the six-byte struct at (FSTRP)
; as the payload. Calls three down (FNGO).
AKOPEN: FNBEGIN FNDFUJI, FCAKOP, 0
         LODI,R2 6
         BSTA,UN FNTXN
         BSTA,UN FNGO
         BCFA,EQ FAIL
         RETC,UN

; AKREAD -- READ_APPKEY: the value is left in the reply window after its
; two-byte length. Calls three down (FNGO).
AKREAD: FNBEGIN FNDFUJI, FCAKRD, 0
         BSTA,UN FNGO
         BCFA,EQ FAIL
         RETC,UN

; AKWRIT -- WRITE_APPKEY: the length as the parameter, then the value.
; Calls three down (FNGO).
AKWRIT: FNBEGIN FNDFUJI, FCAKWR, 1
         LODI,R2 VALLEN
         BSTA,UN FNPARB
         SETFSTR VALUE
         LODI,R2 VALLEN
         BSTA,UN FNTXN
         BSTA,UN FNGO
         BCFA,EQ FAIL
         RETC,UN

; AKSHOW -- the value just read, at cell R2 + 1; EMPTY when there is none.
; Calls two down (DREPLY -> DXLAT).
AKSHOW: LODA,R0 *FRDP           ; the length, under 16 here
         STRA,R0 DRCNT
         BCTR,EQ AKSHW1
         LODI,R0 2              ; after the length
         STRA,R0 DRPOS
         BSTA,UN DREPLY
         RETC,UN
AKSHW1: SETSTR  TEMPTY
         BSTA,UN DPRINT
         RETC,UN

KNAME:  DB      AKCREAT&$FF, AKCREAT>>8, AKAPP, AKKEY, AKMREAD, 0
KOURW:  DB      OURCRE&$FF, OURCRE>>8, OURAPP, OURKEY, AKMWRIT, 0
KOURR:  DB      OURCRE&$FF, OURCRE>>8, OURAPP, OURKEY, AKMREAD, 0
VALUE:  DB      "HELLO ARCADIA"
VALLEN  EQU     $-VALUE
TITLE:  DB      "APP KEYS",0
TNAME:  DB      "PLAYER NAME",0
TOURS:  DB      "OURS, READ BACK",0
TEMPTY: DB      "EMPTY",0

```

CLOCK.ASM asm/clock.asm

```
; clock.asm -- the time of day, from the adapter's clock.
;
; The clock is a device of its own ($45). GET_ISO_LOCAL answers with the
; local time as text -- "2026-11-03T10:45:31" and more -- in the adapter's
; time zone, GET_ISO_UTC the same in UTC. The date and the time go on the
; screen straight out of the reply window. First the adapter's own zone
; is set with SETTZ_ALT ($74; SETTZ, $99, sets the alternate zone).
;
; Build: ./build.sh clock

         CPU     2650

         ORG     $0000
         BCTA    UN,START        ; cartridge header: jump past the
         DB      $17             ; RETC,UN interrupt guard at $0003

         ORG     $0020
```

```
        INCLUDE "fujidisp.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "bookdefs.inc"
        INCLUDE "booklib.inc"

START:  EORZ    R0
        LPSU
        LODI,R0 $02             ; COM=1: logical compares
        LPSL
        BSTA,UN BKINIT
        PUTS    $00, TITLE
        BSTA,UN FNCHECK
        BCTR,EQ GO
        LODI,R0 FENOCRT
        BCTA,UN FAIL

GO:     FNBEGIN DVCLOCK, CKSETTA, 0
        SETFSTR TZONE           ; the zone: the payload
        BSTA,UN FNTXSTR
        BSTA,UN FNGO
        BCFA,EQ FAIL
        PUTS    $20, TLOCAL
        LODI,R1 CKISOL
        LODI,R2 $30-1
        BSTA,UN ASK
        PUTS    $70, TUTC
        LODI,R1 CKISOU
        LODI,R2 $80-1
        BSTA,UN ASK
HANG:   BCTR,UN HANG

; ASK -- clock command R1, no parameters; the date at cell R2 + 1 and the
; time on the row under it. Calls three down (FNGO).
ASK:    STRA,R1 BKVAR           ; hold them across the transaction
        STRA,R2 BKVAR+1
        LODI,R1 FRDATRS
        LODI,R2 0
        BSTA,UN FNREGWR
        LODI,R1 FRDEVIC
        LODI,R2 DVCLOCK
        BSTA,UN FNREGWR
        LODI,R1 FRCMD
        LODA,R2 BKVAR
        BSTA,UN FNREGWR
        LODI,R1 FRNPARM
        LODI,R2 0
        BSTA,UN FNREGWR
        BSTA,UN FNGO
        BCFA,EQ FAIL
        EORZ    R0              ; YYYY-MM-DD: reply[0..9]
        STRA,R0 DRPOS
        LODI,R0 10
        STRA,R0 DRCNT
        LODA,R2 BKVAR+1
        BSTA,UN DREPLY
        LODI,R0 11              ; HH:MM:SS: reply[11..18]
        STRA,R0 DRPOS
        LODI,R0 8
        STRA,R0 DRCNT
        LODA,R2 BKVAR+1
        ADDI,R2 $10
        BSTA,UN DREPLY
        RETC,UN

TZONE:  DB      "EST5EDT",0
TITLE:  DB      "CLOCK",0
TLOCAL: DB      "LOCAL",0
TUTC:   DB      "UTC",0

```

SPRITES.ASM asm/sprites.asm

```
; sprites.asm -- the 2637 up close: character colours, a block-graphics
; row, the built-in box glyphs, and two sprites with both collision
; registers. No FujiNet.
;
; Sprite 0 runs to and fro along row 7, through a box drawn with glyphs
; $04-$0B, and into sprite 1, parked at column 13. Each frame, in the
; vertical blank, the program reads $19FD (sprite against sprite) and
; $19FC (sprite against character); a bit reads 0 where a collision
; happened in the frame just drawn. The sprites' images are the
; user-defined characters $38 and $39, so this program cannot print
; fujidisp.inc's / and - : those are its slash and dash.
;
; Build: ./build.sh sprites

        CPU     2650

S0Y     EQU     $18F0           ; sprite 0: vertical, horizontal
S0X     EQU     $18F1
S1Y     EQU     $18F2           ; sprite 1
S1X     EQU     $18F3
SPCOL   EQU     $19FB           ; sprites 0 and 1: size, colour
SPHIT   EQU     $19FC           ; sprite against character, low = hit
SSHIT   EQU     $19FD           ; sprite against sprite, low = hit
```

```
XLEFT   EQU     43+8*1          ; column 1
XRIGHT  EQU     43+8*14         ; column 14
YROW7   EQU     256-16*7        ; row 7: 16 lines a row
XPOS    EQU     BKVAR           ; sprite 0's horizontal register
STEP    EQU     BKVAR+1         ; +1 or -1

        ORG     $0000
        BCTA    UN,START        ; cartridge header: jump past the
        DB      $17             ; RETC,UN interrupt guard at $0003

        ORG     $0020
        INCLUDE "fujidisp.inc"
        INCLUDE "fujilib.inc"
        INCLUDE "bookdefs.inc"
        INCLUDE "booklib.inc"

START:  EORZ    R0
        LPSU
        LODI,R0 $02             ; COM=1: logical compares
        LPSL
        BSTA,UN BKINIT
        PUTS    $00, TITLE
        LODI,R3 15              ; rows 2 and 4: colours, blocks
ROWS1:  LODA,R0 COLROW,R3
        STRA,R0 SCREEN+$20,R3
        LODA,R0 BLKROW,R3
        STRA,R0 SCREEN+$40,R3
        BDRR,R3 ROWS1
        LODA,R0 COLROW
        STRA,R0 SCREEN+$20
        LODA,R0 BLKROW
        STRA,R0 SCREEN+$40
        LODI,R3 4               ; rows 6-8: the box, columns 0-4
BOX1:   LODA,R0 BOXTOP,R3
        STRA,R0 SCREEN+$60,R3
        LODA,R0 BOXMID,R3
        STRA,R0 SCREEN+$70,R3
        LODA,R0 BOXBOT,R3
        STRA,R0 SCREEN+$80,R3
        BDRR,R3 BOX1
        LODA,R0 BOXTOP
        STRA,R0 SCREEN+$60
        LODA,R0 BOXMID
        STRA,R0 SCREEN+$70
        LODA,R0 BOXBOT
        STRA,R0 SCREEN+$80
        LODI,R3 15              ; the two images: UDCs $38 and $39
IMG1:   LODA,R0 SPIMG,R3
        STRA,R0 UDC,R3
        BDRR,R3 IMG1
        LODA,R0 SPIMG
        STRA,R0 UDC
        LODI,R0 $0D             ; 8x16 both; sprite 0 yellow, 1 red
        STRA,R0 SPCOL
        LODI,R0 YROW7
        STRA,R0 S0Y
        STRA,R0 S1Y
        LODI,R0 43+8*13         ; sprite 1: column 13
        STRA,R0 S1X
        LODI,R0 XLEFT
        STRA,R0 XPOS
        LODI,R0 1
        STRA,R0 STEP
        PUTS    $A0, TX

; ---- once a frame: move, then read what the frame just drawn hit ----------
MAIN:   TPSU    $80             ; the vertical blank
        BCFR,EQ MAIN
        LODA,R0 XPOS
        STRA,R0 S0X
        LODI,R2 $A2-1           ; row 10: the horizontal register
        BSTA,UN DHEX
        LODA,R0 SSHIT           ; bit 0 low: sprites 0 and 1 met
        ANDI,R0 $01
        BCFR,EQ MAIN1
        PUTS    $B0, TSPR
        BCTR,UN MAIN2
MAIN1:  PUTS    $B0, TBLANK
MAIN2:  LODA,R0 SPHIT           ; bit 0 low: sprite 0 met a character
        ANDI,R0 $01
        BCFR,EQ MAIN3
        PUTS    $C0, TCHR
        BCTR,UN MAIN4
MAIN3:  PUTS    $C0, TBLANK
MAIN4:  LODA,R0 XPOS            ; the next position, turning at the ends
        ADDA,R0 STEP
        STRA,R0 XPOS
        COMI,R0 XRIGHT
        BCTR,EQ MAIN5
        COMI,R0 XLEFT
        BCFR,EQ MAIN6
MAIN5:  LODA,R0 STEP            ; turn: 1 <-> $FF
        EORI,R0 $FE
        STRA,R0 STEP
MAIN6:  TPSU    $80             ; to the end of the blank
        BCTR,EQ MAIN6
        BCTA,UN MAIN
```

```
121
122 ; row 2: the colour bits (6-7) of the same digit, four ways
123 COLROW: DB      $1C,$28,$25,$28,$2E,$2B,$2C,$00 ; COLOURS
124         DB      $11,$51,$91,$D1                 ; 1 in 00 01 10 11
125         DB      $00,$00,$00,$00
126 ; row 4: block graphics -- $C0 first turns the row over; each cell's low
127 ; 6 bits are 3 x 2 blocks, top row in bits 0-2
128 BLKROW: DB      $C0,$3F,$07,$38,$15,$2A,$01,$02
129         DB      $04,$08,$10,$20,$3F,$00,$3F,$00
130 BOXTOP: DB      $09,$04,$04,$04,$08     ; the built-in corners and edges
131 BOXMID: DB      $07,$00,$00,$00,$05
132 BOXBOT: DB      $0A,$06,$06,$06,$0B
133 ; sprite 0: a ship; sprite 1: a block with a hole
134 SPIMG:  DB      %00011000,%00111100,%01111110,%11011011
135         DB      %11111111,%00100100,%01011010,%10100101
136         DB      %11111111,%10000001,%10111101,%10100101
137         DB      %10100101,%10111101,%10000001,%11111111
138
139 TITLE:  DB      "SPRITES",0
140 TX:     DB      "X",0
141 TSPR:   DB      "SPRITE HIT",0
142 TCHR:   DB      "BOX HIT",0
143 TBLANK: DB      "          ",0
144
```

FUJILIB.INC runtime/fujilib.inc

```
  1 ; fujilib.inc -- 2650 client half of the FujiNet cartridge mailbox.
  2 ;
  3 ; Hand-mirrors firmware/include/fuji_mailbox.h; keep them in step. Equates are console
  4 ; addresses (image offset + 1000H from image 1000H up). A register write is a read of
  5 ; FNRSEL+reg then FNRDAT+value, back to back: the cart ignores an unpaired REGDATA.
  6 ;
  7 ; 2650 rules:
  8 ;   - Non-branch memory instructions reach only their own 8K page, so mailbox access
  9 ;     goes through the 15-bit indirect pointer words at the end of this file.
 10 ;   - Indexed addressing always moves data through R0. R0 data, R1/R2 arguments,
 11 ;     R3 counters; register bank 1 is never selected.
 12 ;   - The return stack is the on-chip 8-entry RAS; tools/checkdepth.py enforces it.
 13 ;   - Clients run with PSL = 02H (COM=1): compares are logical. A load sets CC.
 14 ;   - DW emits little-endian but indirect pointers are fetched high byte first: use DWBE.
 15
 16 DWBE    MACRO   val             ; big-endian word: what indirection reads
 17         DB      ((val)>>8)&$FF, (val)&$FF
 18         ENDM
 19
 20 SETFSTR MACRO   adr             ; point FNTXSTR at a string (clobbers R0)
 21         LODI,R0 (adr)>>8
 22         STRA,R0 FSTRP
 23         LODI,R0 (adr)&$FF
 24         STRA,R0 FSTRP+1
 25         ENDM
 26
 27 ; ---- reply bytes (repainted ROM), console addresses --------------------
 28 FNRDATA EQU     $2B00           ; 256-byte reply slice
 29 FNACKSQ EQU     $2C00           ; echoes SEQ when the reply is ready
 30 FNSTATS EQU     $2C01           ; bit0 link up
 31 FNERR   EQU     $2C02           ; fb_status_t of the last transaction
 32 FNREPLY EQU     $2C03           ; 06H ACK / 15H NAK
 33 FNRXLL  EQU     $2C04           ; reply length, low
 34 FNRXLH  EQU     $2C05           ; reply length, high
 35 FNBSTAT EQU     $2C06           ; boot state
 36 FNBPCT  EQU     $2C07           ; boot progress 0-100
 37 FNBERR  EQU     $2C08           ; boot error
 38 FNMAGF  EQU     $2C09           ; 'F'
 39 FNMAGN  EQU     $2C0A           ; 'N'
 40 FNPVER  EQU     $2C0B           ; protocol version (2 since APPBANK)
 41 FNSECHO EQU     $2C0C           ; echoes RXSLICE after each repaint
 42 FNTIKL  EQU     $2C0E           ; 60 Hz tick, low (read HI, LO, HI)
 43 FNTIKH  EQU     $2C0F           ; 60 Hz tick, high
 44
 45 ; ---- hotspot pages -----------------------------------------------------
 46 FNRSEL  EQU     $2D00           ; read +reg: arm register
 47 FNRDAT  EQU     $2E00           ; read +val: armed register = val
 48 FNDATA  EQU     $2F00           ; read +val: append val to TX stream
 49 FNSWAP  EQU     $2DFE           ; armed-only: serve the staged image
 50 FNBANK  EQU     $2D80           ; read +n: APPBANK window shows page n
 51
 52 ; ---- register numbers --------------------------------------------------
 53 FRDEVIC EQU     $00
 54 FRCMD   EQU     $01
 55 FRNPARM EQU     $02
 56 FRDATRS EQU     $05
 57 FRRXSLC EQU     $06
 58 FRSEQ   EQU     $10
 59 FRBOOTL EQU     $11             ; BOOTLOCK; magic 0B5H arms the swap
 60 FRBSEL1 EQU     $12             ; BOOTSEL pair: 0B5H then 4AH on FRBSEL2
 61 FRBSEL2 EQU     $13
 62 FRRAMP0 EQU     $14             ; cart RAM pointer = 000H + value
 63 FRRAMP3 EQU     $17             ; cart RAM pointer = 300H + value
 64 FRRAMPK EQU     $18             ; each REGDATA stores and advances
 65
 66 ; ---- values -----------------------------------------------------------
 67 FNDFUJI EQU     $70             ; FujiBus device id of the FujiNet itself
 68 FCACFGX EQU     $C4             ; GET_ADAPTERCONFIG_EXTENDED
```

```
FCMHOST EQU     $F9             ; MOUNT_HOST
FCMIMG  EQU     $F8             ; MOUNT_IMAGE
FCSDFP  EQU     $E2             ; SET_DEVICE_FULLPATH
FCRHSL  EQU     $F4             ; READ_HOST_SLOTS
FCODIR  EQU     $F7             ; OPEN_DIRECTORY
FCRDIR  EQU     $F6             ; READ_DIR_ENTRY
FCCDIR  EQU     $F5             ; CLOSE_DIRECTORY
FCSDPS  EQU     $E4             ; SET_DIRECTORY_POSITION
FCACK   EQU     $06             ; reply command: ACK
FCNAK   EQU     $15             ; reply command: NAK
FMREAD  EQU     1               ; disk access mode: read
FEOK    EQU     0               ; FNERR values
FENOLNK EQU     1
FETMOUT EQU     2
FETIMO  EQU     $FF             ; helper-local: FNCOMMIT/FNSLICE gave up
FBIDLE  EQU     0               ; FNBSTAT values
FBXFER  EQU     1
FBREADY EQU     2
FBFAIL  EQU     $80
FNBLMAG EQU     $B5             ; BOOTLOCK magic

; ---- RAM the helpers own (zone A scratch; see the client RAM maps) ----
FSTRP   EQU     $18EB           ; FNTXSTR string pointer, big-endian word

; ---- routines ---------------------------------------------------------

; Write mailbox register: R1 = register number, R2 = value.
; Leaf. Clobbers R0.
FNREGWR:
        LODA,R0 *FSELP,R1       ; arm the register
        LODA,R0 *FDATP,R2       ; deliver the value
        RETC,UN

; Rewind the TX pointer and set the device id. Calls one down. Clobbers R0,R1,R2.
FNSTART:
        LODI,R1 FRDATRS
        LODI,R2 0
        BSTA,UN FNREGWR
        LODI,R1 FRDEVIC
        LODI,R2 FNDFUJI
        BCTA,UN FNREGWR         ; tail call: FNREGWR returns to our caller

; Append a 1-byte parameter with value R2. Leaf. Clobbers R0,R1.
FNPARB:
        LODI,R1 1
        LODA,R0 *FTXP,R1        ; size byte
        LODA,R0 *FTXP,R2        ; value
        RETC,UN

; Append the 0-terminated string at (FSTRP), which may be in the reply window. Caps at
; 255. Leaf. Clobbers R0,R1,R3.
FNTXSTR:
        LODI,R3 $FF
FNTXS1: LODA,R0 *FSTRP,R3,+    ; str[++R3]; a load sets CC
        BCTR,EQ FNTXS2          ; 0 terminator
        STRZ    R1
        LODA,R0 *FTXP,R1        ; append
        BCTR,UN FNTXS1
FNTXS2: RETC,UN

; Stream the string at (FSTRP), NUL-padded to the 256 bytes SET_DEVICE_FULLPATH and
; OPEN_DIRECTORY require. Caps at 255. Leaf. Clobbers R0,R1,R3.
FNPATH: LODI,R3 0               ; bytes sent so far (mod 256)
FNPP1:  LODA,R0 *FSTRP,R3       ; path[R3]; a load sets CC
        BCTR,EQ FNPP2           ; terminator: do not send it
        STRZ    R1
        LODA,R0 *FTXP,R1        ; append the path byte
        ADDI,R3 1
        BCTR,UN FNPP1
FNPP2:  LODI,R1 0               ; pad byte 0 (EORZ acts on R0 only, not R1)
FNPP3:  LODA,R0 *FTXP,R1        ; append 0
        ADDI,R3 1               ; until R3 wraps to 0 (256 total)
        BCFR,EQ FNPP3
        RETC,UN

; Commit with SEQ = the cart's ACKSEQ + 1 (RESET does not reset the cart; 255 wraps to 1)
; and wait. Returns R0 = FNERR or FETIMO after ~6 s (64K polls of ~90 us, past the cart's
; 5 s transact budget). Calls one down. Clobbers R1,R2,R3.
FNCOMMIT:
        LODA,R0 *FACKP
        ADDI,R0 1
        BCFR,EQ FNCM1
        LODI,R0 1               ; 255 wraps past the reserved 0
FNCM1:  STRZ    R2              ; R2 = SEQ to write = ACKSEQ to expect
        LODI,R1 FRSEQ
        BSTA,UN FNREGWR
        LODI,R3 0               ; outer x256
FNCMO:  LODI,R1 0               ; inner x256
FNCMI:  NOP                     ; pad the poll to ~90 us so the full sweep
        NOP                     ; outlasts the cart's transact timeout
        NOP
        NOP
        NOP
        NOP
        LODA,R0 *FACKP
        COMZ    R2
        BCTR,EQ FNCMOK
```

```
166         BDRR,R1 FNCMI
167         BDRR,R3 FNCMO
168         LODI,R0 FETIMO
169         RETC,UN
170 FNCMOK: LODA,R0 *FERRP
171         RETC,UN
172
173 ; Select reply slice R2 (0-3) and wait for FNSECHO, painted last, to echo it.
174 ; Returns R0 = 0 ok, FETIMO on timeout (~1.5 s). Preserves R2.
175 ; Calls one down. Clobbers R0,R1,R3.
176 FNSLICE:
177         LODI,R1 FRRXSLC
178         BSTA,UN FNREGWR
179         LODI,R3 64              ; outer x64
180 FNSLO:  LODI,R1 0               ; inner x256
181 FNSLI:  LODA,R0 *FSECP
182         COMZ    R2
183         BCTR,EQ FNSLOK
184         BDRR,R1 FNSLI
185         BDRR,R3 FNSLO
186         LODI,R0 FETIMO
187         RETC,UN
188 FNSLOK: EORZ    R0
189         RETC,UN
190
191 ; R0 = 0 when the 'F','N' magic is present, else nonzero. Leaf.
192 FNCHECK:
193         LODA,R0 *FMAGFP
194         COMI,R0 'F'
195         BCFR,EQ FNCKNO
196         LODA,R0 *FMAGNP
197         COMI,R0 'N'
198         BCFR,EQ FNCKNO
199         EORZ    R0
200         RETC,UN
201 FNCKNO: LODI,R0 $FF
202         RETC,UN
203
204 ; ---- pointer words (ROM, page 0): what the indirection dereferences ----
205 FSELP:  DWBE    FNRSEL
206 FDATP:  DWBE    FNRDAT
207 FTXP:   DWBE    FNDATA
208 FRDP:   DWBE    FNRDATA
209 FACKP:  DWBE    FNACKSQ
210 FERRP:  DWBE    FNERR
211 FREPP:  DWBE    FNREPLY
212 FRXLP:  DWBE    FNRXLL
213 FBSTP:  DWBE    FNBSTAT
214 FBPCP:  DWBE    FNBPCT
215 FBERP:  DWBE    FNBERR
216 FMAGFP: DWBE    FNMAGF
217 FMAGNP: DWBE    FNMAGN
218 FSECP:  DWBE    FNSECHO
219 FSWAPP: DWBE    FNSWAP
220
```

FUJIDISP.INC `runtime/fujidisp.inc`

```
 1 ; fujidisp.inc -- 2637 UVI display helpers for the Arcadia FujiNet clients.
 2 ;
 3 ; The UVI's 64 glyphs lack lowercase and / - _ :, so those live in the 8 user-defined
 4 ; characters (codes $38-$3F; no client uses sprites). A screen byte is 2 color bits + a
 5 ; 6-bit glyph; never emit color bits 11, since a $C0 byte flips the row to block graphics.
 6
 7 SCREEN  EQU     $1800           ; upper screen, 16 cols x 13 rows
 8 UDC     EQU     $1980           ; 8 user-defined characters, 8 bytes each
 9 GFXMOD  EQU     $19F8           ; bit7 gfx mode, bit6 26-line, 0-5 alt cols
10 BGCOL   EQU     $19F9           ; bit7 char size, 6 pot mux, 3-5 char col,
11                                 ; 0-2 screen col (colors inverted: 0=white,
12                                 ; 7=black)
13 PITCH   EQU     $18FD           ; bit7 multicolor, 0-6 sound frequency
14 VOLUME  EQU     $18FE           ; 0-2 loudness, 3 snd en, 5-7 horiz shift
15
16 DSTRP   EQU     $18ED           ; DPRINT string pointer, big-endian word
17
18 SETSTR  MACRO   adr             ; point DPRINT at a string (clobbers R0)
19         LODI,R0 (adr)>>8
20         STRA,R0 DSTRP
21         LODI,R0 (adr)&$FF
22         STRA,R0 DSTRP+1
23         ENDM
24
25 ; UDC glyph codes (assignments shared by every client)
26 GSLASH  EQU     $38
27 GDASH   EQU     $39
28 GUSCOR  EQU     $3A
29 GCOLON  EQU     $3B
30 GCHEVR  EQU     $3C             ; cursor chevron
31 GBULLT  EQU     $3D             ; password mask
32 GMORE   EQU     $3E             ; more-pages hint
33 GBLOCK  EQU     $03             ; built-in solid block (progress bar fill)
34 GBLINE  EQU     $06             ; built-in baseline (progress bar empty)
35
36 ; 13-line text mode, white on black, sound off, UDCs loaded.
37 ; Calls one down (DUDCLD). Clobbers R0,R1,R3.
```

```
DINIT:  EORZ    R0
        STRA,R0 GFXMOD
        STRA,R0 PITCH
        STRA,R0 VOLUME
        LODI,R0 $87             ; 8x8 chars, white text, black screen
        STRA,R0 BGCOL
        LODI,R3 63              ; load the 8 UDCs (64 bytes)
DUDCLD: LODA,R0 UDCTAB,R3
        STRA,R0 UDC,R3
        BDRR,R3 DUDCLD
        LODA,R0 UDCTAB
        STRA,R0 UDC
        RETC,UN

; Clear the upper screen (208 cells to spaces). Leaf. Clobbers R0,R3.
DCLS:   LODI,R3 207
DCLS1:  EORZ    R0
        STRA,R0 SCREEN,R3
        BDRR,R3 DCLS1
        EORZ    R0
        STRA,R0 SCREEN
        RETC,UN

; ASCII in R0 -> glyph code in R0 ($00-$3F only). Lowercase folds to
; uppercase, anything unprintable becomes '.'. Leaf. Clobbers R0,R1.
DXLAT:  ANDI,R0 $7F
        SUBI,R0 $20
        BCTR,LT DXLBAD          ; control character
        STRZ    R1
        LODA,R0 XLAT,R1
        RETC,UN
DXLBAD: LODI,R0 $34             ; '.'
        RETC,UN

; Print the string at (DSTRP) from screen offset R2 + 1 (R2 = target cell minus one for
; auto-increment; returns the last cell written). No line wrap. Calls one down (DXLAT).
; Clobbers R0,R1,R2,R3.
DPRINT: LODI,R3 $FF
DPRT1:  LODA,R0 *DSTRP,R3,+     ; str[++R3]; a load sets CC
        BCTR,EQ DPRT2
        BSTR,UN DXLAT
        STRA,R0 SCREEN,R2,+     ; screen[++R2]
        BCTR,UN DPRT1
DPRT2:  RETC,UN

; Print R0 as two hex digits at screen offset R2+1 ($10+nibble is 0-9, A-F). Leaf.
; Clobbers R0,R1,R2.
DHEX:   STRZ    R1
        RRR,R0
        RRR,R0
        RRR,R0
        RRR,R0
        ANDI,R0 $0F
        IORI,R0 $10
        STRA,R0 SCREEN,R2,+
        LODZ    R1
        ANDI,R0 $0F
        IORI,R0 $10
        STRA,R0 SCREEN,R2,+
        RETC,UN

; ---- input: joystick disc + keypad ---------------------------------
;   - FIRE shares a line with keypad '2' ($1901 b3), so that bit is SELECT and up/down
;     live on the disc.
;   - The disc is a pot the 2637 digitizes, valid only in vblank: $19FF with the pot mux
;     ($19F9 bit 6) clear reads up 0x00, down 0xFF, center ~0x6F. Outside vblank it reads
;     0xFF, which looks like down.
; Keypad, active high, masked $0F, one column per address:
;   $1900 b3=1 b2=4 b1=7 b0=Clear     $1901 b3=2/FIRE b2=5 b1=8 b0=0
;   $1902 b3=3 b2=6 b1=9 b0=Enter
; UI: disc up/down move; FIRE or Enter select; Clear back; 4/6 page; 8 down.
KUP     EQU     1
KDOWN   EQU     2
KPGUP   EQU     3
KPGDN   EQU     4
KSEL    EQU     5
KBACK   EQU     6

JOYP1   EQU     $19FF           ; P1 vertical pot (mux clear): up 00, down FF
JOYUP   EQU     $40             ; < this = up
JOYDN   EQU     $C0             ; > this = down

; SCAN: R0 = code of the first control found (0 if none). Leaf. Clobbers R0,R1.
SCAN:   TPSU    $80             ; wait for vertical blank (pot A/D valid there)
        BCFR,EQ SCAN
        LODA,R0 JOYP1           ; disc up/down first
        STRZ    R1
        COMI,R0 JOYUP
        BCTA,LT SCU             ; < 0x40 -> up
        LODZ    R1
        COMI,R0 JOYDN
        BCTA,GT SCD             ; > 0xC0 -> down
        LODA,R0 $1901           ; '8' down (b1)
        ANDI,R0 $02
        BCFA,EQ SCD
        LODA,R0 $1901           ; '2'/FIRE select (b3)
        ANDI,R0 $08
```

```
        BCFA,EQ SCS
        LODA,R0 $1902           ; Enter select (b0)
        ANDI,R0 $01
        BCFA,EQ SCS
        LODA,R0 $1900           ; Clear back (b0)
        ANDI,R0 $01
        BCFA,EQ SCB
        LODA,R0 $1900           ; '4' page up (b2)
        ANDI,R0 $04
        BCFA,EQ SCPU
        LODA,R0 $1902           ; '6' page down (b2)
        ANDI,R0 $04
        BCFA,EQ SCPD
        EORZ    R0
        RETC,UN
SCU:    LODI,R0 KUP
        RETC,UN
SCD:    LODI,R0 KDOWN
        RETC,UN
SCS:    LODI,R0 KSEL
        RETC,UN
SCB:    LODI,R0 KBACK
        RETC,UN
SCPU:   LODI,R0 KPGUP
        RETC,UN
SCPD:   LODI,R0 KPGDN
        RETC,UN

; GETKEY: R0 = code of a fresh press; a held control steps once. Calls one down (SCAN).
GETKEY: BSTA,UN SCAN            ; wait for release
        COMI,R0 0
        BCFA,EQ GETKEY
GKW:    BSTA,UN SCAN            ; wait for a press
        COMI,R0 0
        BCTA,EQ GKW
        RETC,UN

; ---- ASCII $20-$7F -> glyph, 96 entries --------------------------------
XLAT:   DB      $00,$34,$34,$34,$37,$34,$34,$34 ; spc ! " # $ % & '
        DB      $34,$34,$34,$36,$35,GDASH,$34,GSLASH ; ( ) * + , - . /
        DB      $10,$11,$12,$13,$14,$15,$16,$17 ; 0-7
        DB      $18,$19,GCOLON,$34,$34,$34,$34,$34 ; 8 9 : ; < = > ?
        DB      $34,$1A,$1B,$1C,$1D,$1E,$1F,$20 ; @ A-G
        DB      $21,$22,$23,$24,$25,$26,$27,$28 ; H-O
        DB      $29,$2A,$2B,$2C,$2D,$2E,$2F,$30 ; P-W
        DB      $31,$32,$33,$34,$34,$34,$34,GUSCOR ; X Y Z [ \ ] ^ _
        DB      $34,$1A,$1B,$1C,$1D,$1E,$1F,$20 ; ` a-g
        DB      $21,$22,$23,$24,$25,$26,$27,$28 ; h-o
        DB      $29,$2A,$2B,$2C,$2D,$2E,$2F,$30 ; p-w
        DB      $31,$32,$33,$34,$34,$34,$34,$34 ; x y z { | } ~ del

; ---- the 8 UDCs, 8 bytes each ($38-$3F) --------------------------------
UDCTAB: DB      %00000010       ; $38 /
        DB      %00000100
        DB      %00001000
        DB      %00010000
        DB      %00100000
        DB      %01000000
        DB      %00000000
        DB      %00000000
        DB      %00000000       ; $39 -
        DB      %00000000
        DB      %00000000
        DB      %01111100
        DB      %00000000
        DB      %00000000
        DB      %00000000
        DB      %00000000
        DB      %00000000       ; $3A _
        DB      %00000000
        DB      %00000000
        DB      %00000000
        DB      %00000000
        DB      %00000000
        DB      %01111110
        DB      %00000000
        DB      %00000000       ; $3B :
        DB      %00011000
        DB      %00011000
        DB      %00000000
        DB      %00011000
        DB      %00011000
        DB      %00000000
        DB      %00000000
        DB      %01000000       ; $3C chevron cursor
        DB      %01100000
        DB      %01110000
        DB      %01111000
        DB      %01110000
        DB      %01100000
        DB      %01000000
        DB      %00000000
        DB      %00000000       ; $3D bullet (password mask)
        DB      %00111000
        DB      %01111100
        DB      %01111100
        DB      %00111000
```

```
232         DB      %00000000
233         DB      %00000000
234         DB      %00000000
235         DB      %00010000       ; $3E more-pages hint (up+down)
236         DB      %00111000
237         DB      %00000000
238         DB      %00000000
239         DB      %00000000
240         DB      %00111000
241         DB      %00010000
242         DB      %00000000
243         DB      %00000000       ; $3F spare
244         DB      %00000000
245         DB      %00000000
246         DB      %00000000
247         DB      %00000000
248         DB      %00000000
249         DB      %00000000
250         DB      %00000000
251
```

HELLO.ASM runtime/hello.asm

```
  1 ; hello.asm -- Arcadia 2001 toolchain + display smoke test. No FujiNet.
  2 ;
  3 ; Paints the 64-glyph UVI charset, a user-defined character and a graphics-mode probe,
  4 ; and echoes the keypad columns as hex ($10+nibble is a hex digit: the charset runs 0-9, A-Z).
  5 ;
  6 ; Screen: row 0 title, rows 2-5 the charset, row 7 keypad echo, row 9 the
  7 ; UDC, row 11 the $C0/$40 graphics-mode probe.
  8
  9         CPU     2650
 10
 11 SCREEN  EQU     $1800           ; upper screen, 16x13
 12 UDC     EQU     $1980           ; user-defined characters, 8x8 bytes
 13 GFXMOD  EQU     $19F8           ; bit7 gfx, bit6 26-line, 0-5 alt colors
 14 BGCOL   EQU     $19F9           ; bit7 char size, 3-5 char col, 0-2 screen col
 15 PITCH   EQU     $18FD           ; bit7 multicolor, 0-6 sound freq
 16 VOLUME  EQU     $18FE           ; 0-2 loudness, 3 snd en, 5-7 horiz shift
 17
 18         ORG     $0000
 19         BCTA    UN,START        ; cartridge header: jump past the
 20         DB      $17             ; RETC,UN interrupt guard at $0003
 21
 22         ORG     $0020
 23 START:  EORZ    R0
 24         LPSU                    ; power-on-like PSW
 25         LPSL
 26
 27 ; --- UVI setup: 13-line text mode, white on black, sound off -------------
 28         EORZ    R0
 29         STRA,R0 GFXMOD          ; no gfx mode, 13 lines
 30         STRA,R0 PITCH
 31         STRA,R0 VOLUME
 32         LODI,R0 $87             ; 8x8 chars, char color 0 (white),
 33         STRA,R0 BGCOL           ; screen color 7 (black)
 34
 35 ; --- clear the upper screen ($1800-$18CF, 208 cells) ----------------------
 36         LODI,R3 207
 37 CLRLP:  EORZ    R0
 38         STRA,R0 SCREEN,R3
 39         BDRR,R3 CLRLP
 40         EORZ    R0
 41         STRA,R0 SCREEN
 42
 43 ; --- title row ----------------------------------------------------------
 44         LODI,R3 12
 45 TTLLP:  LODA,R0 MSG,R3
 46         STRA,R0 SCREEN,R3
 47         BDRR,R3 TTLLP
 48         LODA,R0 MSG
 49         STRA,R0 SCREEN
 50
 51 ; --- rows 2-5: the whole character set, codes $00-$3F ---------------------
 52         LODI,R3 63
 53 CHRLP:  LODZ    R3
 54         STRA,R0 SCREEN+32,R3
 55         BDRR,R3 CHRLP
 56         EORZ    R0
 57         STRA,R0 SCREEN+32
 58
 59 ; --- a UDC in slot $38 (asymmetric, to expose the byte layout) ------------
 60         LODI,R3 7
 61 UDCLP:  LODA,R0 UDCDAT,R3
 62         STRA,R0 UDC,R3
 63         BDRR,R3 UDCLP
 64         LODA,R0 UDCDAT
 65         STRA,R0 UDC
 66         LODI,R0 $38             ; show it at row 9
 67         STRA,R0 SCREEN+144
 68
 69 ; --- row 11: graphics-mode probe ($C0 flips the row, $40 flips it back) ---
 70         LODI,R3 9
 71 GFXLP:  LODA,R0 GFXPRB,R3
 72         STRA,R0 SCREEN+176,R3
 73         BDRR,R3 GFXLP
```

```
        LODA,R0 GFXPRB
        STRA,R0 SCREEN+176

; --- main loop: echo keypad columns + panel as hex at row 7 ---------------
MAIN:   TPSU    $80             ; wait for vertical retrace start
        BCFR,EQ MAIN

        LODA,R0 $1900           ; P1 column 1 (1/4/7/Clear)
        ANDI,R0 $0F
        IORI,R0 $10
        STRA,R0 SCREEN+112
        LODA,R0 $1901           ; P1 column 2 (2/5/8/0 + fire)
        ANDI,R0 $0F
        IORI,R0 $10
        STRA,R0 SCREEN+114
        LODA,R0 $1902           ; P1 column 3 (3/6/9/Enter)
        ANDI,R0 $0F
        IORI,R0 $10
        STRA,R0 SCREEN+116
        LODA,R0 $1908           ; panel (Start/Option/Select)
        ANDI,R0 $0F
        IORI,R0 $10
        STRA,R0 SCREEN+118

MAINW:  TPSU    $80             ; wait for retrace end
        BCTR,EQ MAINW
        BCTA    UN,MAIN

; --- data -----------------------------------------------------------------
MSG:    DB      $21,$1E,$25,$25,$28,$00         ; HELLO
        DB      $1F,$2E,$23,$22,$27,$1E,$2D     ; FUJINET

UDCDAT: DB      %10000000       ; asymmetric F-like shape: if rendered
        DB      %11111110       ; mirrored or byte-swapped it is obvious
        DB      %10000000
        DB      %11111100
        DB      %10000000
        DB      %10000000
        DB      %10000000
        DB      %00000000

GFXPRB: DB      $C0,$01,$02,$03,$3F,$40,$21,$1E,$25,$28

```

FUJITEST.ASM `runtime/fujitest.asm`

```
; fujitest.asm -- FujiNet cartridge link probe for the Emerson Arcadia 2001.
;
; On a plain 8K EPROM it shows NO FUJINET CART and the two bytes read in place of the
; magic. On the FujiNet cart it runs GET_ADAPTERCONFIG_EXTENDED and prints the SSID,
; firmware version and IP address.
;
; Screen: row 0 title, rows 4-5 SSID, row 7 version, row 9 IP.
; Build: ../build.sh fujitest

        CPU     2650

        ORG     $0000
        BCTA    UN,START        ; cartridge header: jump past the
        DB      $17             ; RETC,UN interrupt guard at $0003

        ORG     $0020
        INCLUDE "fujidisp.inc"  ; macros must precede their callers (AS
        INCLUDE "fujilib.inc"   ; resolves forward labels, not macros)

START:  EORZ    R0
        LPSU
        LODI,R0 $02             ; COM=1: logical compares (fujilib relies
        LPSL                    ; on it)
        BSTA,UN DINIT
        BSTA,UN DCLS

        SETSTR  TSTR            ; title
        LODI,R2 $FF             ; row 0 col 0, minus-one convention
        BSTA,UN DPRINT

        BSTA,UN FNCHECK
        BCTR,EQ FOUND           ; R0=0: magic present

; ---- no cartridge magic: the EPROM path -------------------------------
        SETSTR  NCSTR
        LODI,R2 $20-1           ; row 2
        BSTA,UN DPRINT
        LODA,R0 *FMAGFP         ; the two bytes living where the magic
        LODI,R2 $30-1           ; should be, row 3
        BSTA,UN DHEX
        LODA,R0 *FMAGNP
        BSTA,UN DHEX
HALT1:  BCTA,UN HALT1

; ---- cartridge present: ask the adapter about itself ------------------
FOUND:  BSTA,UN FNSTART
        LODI,R1 FRCMD
        LODI,R2 FCACFGX
        BSTA,UN FNREGWR
        LODI,R1 FRNPARM
```

```
        LODI,R2 0
        BSTA,UN FNREGWR
        BSTA,UN FNCOMMIT
        COMI,R0 0
        BCTR,EQ GOTCFG

        STRZ    R3              ; hold the error code
        SETSTR  ERSTR
        LODI,R2 $20-1           ; row 2: ERR nn
        BSTA,UN DPRINT
        LODZ    R3
        BSTA,UN DHEX
HALT2:  BCTA,UN HALT2

; AdapterConfigExtended reply: ssid at +0, fn_version at +125, sLocalIP at +140.
GOTCFG: LODI,R2 0
        BSTA,UN FNSLICE

        SETSTR  FNRDATA         ; SSID: flows from row 4 into row 5
        LODI,R2 $40-1
        BSTA,UN DPRINT

        SETSTR  FNRDATA+125     ; row 7: firmware version
        LODI,R2 $70-1
        BSTA,UN DPRINT

        SETSTR  FNRDATA+140     ; row 9: IP (sLocalIP is dotted-decimal
        LODI,R2 $90-1           ; text)
        BSTA,UN DPRINT
IPDONE: BCTA,UN IPDONE

TSTR:   DB      "FUJITEST",0
NCSTR:  DB      "NO FUJINET CART",0
ERSTR:  DB      "ERR ",0

```

FUJIBOOT.ASM runtime/fujiboot.asm

```
; fujiboot.asm -- mount a cartridge image over the network and boot it.
;
; MOUNT_HOST -> SET_DEVICE_FULLPATH -> MOUNT_IMAGE. Before MOUNT_IMAGE replies, the ESP32
; streams the image to the DBC device; the cart stages it and reports BOOT_STATE. Then arm
; the swap and run the stub at $18E0 that reads FN_HOT_SWAP and jumps to $0000.
;
; The host slot and path are baked in by build.sh:
;   BOOT_HOST=0 BOOT_PATH=/game.bin ./build.sh fujiboot

        CPU     2650

DEVSLOT EQU     0
STUB    EQU     $18E0           ; the swap stub, in free RAM
STBPTA  EQU     $18E9           ; where the FNSWAP pointer lands at runtime

        ORG     $0000
        BCTA    UN,START
        DB      $17

        ORG     $0020
        INCLUDE "fujidisp.inc"
        INCLUDE "fujilib.inc"

START:  EORZ    R0
        LPSU
        LODI,R0 $02             ; COM=1: logical compares
        LPSL
        BSTA,UN DINIT
        BSTA,UN DCLS

        SETSTR  TSTR
        LODI,R2 $FF
        BSTA,UN DPRINT

        BSTA,UN FNCHECK
        BCTR,EQ HAVE
        SETSTR  ENOC
        LODI,R2 $20-1
        BSTA,UN DPRINT
HLTNC:  BCTA,UN HLTNC

HAVE:   SETSTR  BOOTPTH         ; show the target path on row 1
        LODI,R2 $10-1
        BSTA,UN DPRINT

; ---- MOUNT_HOST(host) ------------------------------------------------
        BSTA,UN FNSTART
        LODI,R1 FRCMD
        LODI,R2 FCMHOST
        BSTA,UN FNREGWR
        LODI,R1 FRNPARM
        LODI,R2 1
        BSTA,UN FNREGWR
        LODI,R2 BOOTHST
        BSTA,UN FNPARB
        BSTA,UN FNCOMMIT
        LODI,R1 1               ; err class 1 = host
        BSTA,UN CHKACK
```

```
59
60 ; ---- SET_DEVICE_FULLPATH(dev, host, mode, path) ------------------------
61         BSTA,UN FNSTART
62         LODI,R1 FRCMD
63         LODI,R2 FCSDFP
64         BSTA,UN FNREGWR
65         LODI,R1 FRNPARM
66         LODI,R2 3
67         BSTA,UN FNREGWR
68         LODI,R2 DEVSLOT
69         BSTA,UN FNPARB
70         LODI,R2 BOOTHST
71         BSTA,UN FNPARB
72         LODI,R2 FMREAD
73         BSTA,UN FNPARB
74         SETFSTR BOOTPTH
75         BSTA,UN FNPATH          ; path, NUL-padded to 256
76         BSTA,UN FNCOMMIT
77         LODI,R1 2               ; err class 2 = path
78         BSTA,UN CHKACK
79
80 ; ---- MOUNT_IMAGE(dev, mode) ---------------------------------------------
81         SETSTR  MNTG
82         LODI,R2 $20-1
83         BSTA,UN DPRINT
84         BSTA,UN FNSTART
85         LODI,R1 FRCMD
86         LODI,R2 FCMIMG
87         BSTA,UN FNREGWR
88         LODI,R1 FRNPARM
89         LODI,R2 2
90         BSTA,UN FNREGWR
91         LODI,R2 DEVSLOT
92         BSTA,UN FNPARB
93         LODI,R2 FMREAD
94         BSTA,UN FNPARB
95         BSTA,UN FNCOMMIT
96         LODI,R1 3               ; err class 3 = mount
97         BSTA,UN CHKACK
98
99 ; ---- wait for the staged image (progress bar on row 3) -----------------
100 WAITRD: LODA,R0 *FBSTP
101         COMI,R0 FBREADY
102         BCTR,EQ READY
103         LODA,R0 *FBSTP
104         COMI,R0 FBFAIL
105         BCFR,LT WSHOW           ; < FBFAIL: still working
106         LODI,R1 4               ; stage failure
107         LODA,R0 *FBERP
108         STRZ    R2
109         BCTA,UN FAIL
110 WSHOW:  LODA,R0 *FBPCP          ; 0-100 -> a 10-cell bar on row 3
111         BSTA,UN BAR
112         BCTR,UN WAITRD
113
114 ; ---- ready: arm and swap ------------------------------------------------
115 READY:  SETSTR  BOOTG
116         LODI,R2 $30-1
117         BSTA,UN DPRINT
118         LODI,R1 FRBOOTL         ; arm the swap
119         LODI,R2 FNBLMAG
120         BSTA,UN FNREGWR
121         LODI,R3 STUBLEN-1       ; copy the stub into RAM
122 STCPY:  LODA,R0 STUBSRC,R3
123         STRA,R0 STUB,R3
124         BDRR,R3 STCPY
125         LODA,R0 STUBSRC
126         STRA,R0 STUB
127         BCTA,UN STUB
128
129 ; Run from $18E0, so the FNSWAP pointer must land at STBPTA ($18E9): 9 bytes precede it.
130 STUBSRC:
131         EORZ    R0              ; +0
132         LPSU                    ; +1  PSU = 0
133         LPSL                    ; +2  PSL = 0 (power-on-like for the image)
134         LODA,R0 *STBPTA         ; +3  read $2DFE indirect -> swap
135         BCTA,UN $0000           ; +6  the new image's header
136         DWBE    FNSWAP          ; +9  = $18E9 at runtime
137 STUBLEN EQU     $-STUBSRC
138
139 ; ---- helpers -----------------------------------------------------------
140
141 ; After FNCOMMIT: R0 = FNERR (or FETIMO), R1 = error class. On error or NAK jump to FAIL
142 ; with R2 = detail. Clobbers R0,R2.
143 CHKACK: COMI,R0 0
144         BCFR,EQ CKBAD           ; nonzero FNERR (incl FETIMO)
145         LODA,R0 *FREPP
146         COMI,R0 FCACK
147         BCFR,EQ CKNAK
148         RETC,UN
149 CKBAD:  STRZ    R2              ; detail = FNERR value
150         BCTA,UN FAIL
151 CKNAK:  LODI,R2 $EE             ; detail = NAK marker
152         BCTA,UN FAIL
153
154 ; Progress bar on row 3 for percentage R0. Leaf. Clobbers R0-R3.
155 BAR:    LODI,R1 0               ; filled cells = R0 / 10
```

```
BARDIV: COMI,R0 10
        BCTR,LT BARDRW
        SUBI,R0 10
        ADDI,R1 1
        BCTR,UN BARDIV
BARDRW: LODI,R3 0               ; cell index
BARC:   LODZ    R3              ; fill while index R3 < filled R1
        COMZ    R1
        BCTR,LT BARFIL
        LODI,R0 GBLINE
        BCTR,UN BARPUT
BARFIL: LODI,R0 GBLOCK
BARPUT: STRA,R0 SCREEN+48,R3    ; row 3
        ADDI,R3 1
        COMI,R3 10
        BCFR,EQ BARC
        RETC,UN

; R1 = error class, R2 = detail byte. Show "ERR c dd" on row 5 and halt.
FAIL:   LODZ    R1              ; stash across the clobbering DPRINT
        STRA,R0 FTMPA
        LODZ    R2
        STRA,R0 FTMPB
        SETSTR  ESTR
        LODI,R2 $50-1
        BSTA,UN DPRINT          ; "ERR "
        LODA,R0 FTMPA
        ADDI,R0 $10             ; class as a single digit
        STRA,R0 SCREEN+$54
        LODI,R2 $56-1
        LODA,R0 FTMPB
        BSTA,UN DHEX
HLTF:   BCTA,UN HLTF

FTMPA   EQU     $18DE           ; zone A scratch (V_TMP2)
FTMPB   EQU     $18DF

; ---- data ------------------------------------------------------------
TSTR:   DB      "FUJINET BOOT",0
MNTG:   DB      "MOUNTING",0
BOOTG:  DB      "BOOTING",0
ENOC:   DB      "NO FUJINET CART",0
ESTR:   DB      "ERR ",0

        INCLUDE "../build/bootcfg.inc"
```

FUJIBANK.ASM `runtime/fujibank.asm`

```
; fujibank.asm -- protocol 2 probe for the Emerson Arcadia 2001 FujiNet cart.
;
; An APPBANK image (8K + 3 pages = 14336 bytes) that checks, on screen and in
; RAM for emu/bankprobe.lua:
;   RES+0..4   the id byte at $0FFF after selecting pages 0..4: 00 01 02 03 FF
;              (page 4 does not exist, so the window reads $FF)
;   RES+5      what a call to page 2's entry at $0800 returned: C2
;   RES+6      01 when a poke stream into RAM-L ($0600) reads back
;   RES+7      01 when a poke stream into RAM-H ($2AFC) reads back
;   RES+8      01 when stray REGDATAs after a closed stream stored nothing
;   RES+9      the protocol version: 02
;   RES+10     00 when 'F','N' are present
;   RES+11..14 the tick (lo, hi) before and after 60 vblanks
;   RES+15     5A: done
; then shows the live tick on row 9.
;
; Build: ../build.sh fujibank

        CPU     2650

RES     EQU     $18D0           ; results, 16 bytes (zone A)
RAMH    EQU     $2AFC           ; where the RAM-H stream lands
NFRAME  EQU     $18E0           ; vblank counter (the boot stub's area: free here)

        ORG     $0000
        BCTA    UN,START        ; cartridge header: jump past the
        DB      $17             ; RETC,UN interrupt guard at $0003

        ORG     $0020
        INCLUDE "fujidisp.inc"
        INCLUDE "fujilib.inc"

START:  EORZ    R0
        LPSU
        LODI,R0 $02             ; COM=1: logical compares
        LPSL
        LODI,R1 0               ; RESET leaves the page alone: take page 0
        LODA,R0 *FBNKP,R1
        BSTA,UN DINIT
        BSTA,UN DCLS
        SETSTR  TSTR
        LODI,R2 $FF
        BSTA,UN DPRINT

        LODI,R3 15              ; clear the results
        EORZ    R0
CLR1:   STRA,R0 RES,R3
```

```
        BDRR,R3 CLR1
        STRA,R0 RES

; ---- 1. page ids, page 4 missing -------------------------------------
        LODI,R3 0
IDS1:   LODZ    R3
        STRZ    R1
        LODA,R0 *FBNKP,R1       ; select page R1: the next fetch sees it
        LODA,R0 $0FFF
        STRA,R0 RES,R3
        ADDI,R3 1
        COMI,R3 5
        BCFR,EQ IDS1

; ---- 2. a call into page 2's entry table -----------------------------
        LODI,R1 2
        LODA,R0 *FBNKP,R1
        BSTA,UN $0800
        STRA,R0 RES+5
        LODI,R1 0
        LODA,R0 *FBNKP,R1

; ---- 3. RAM-L and RAM-H ----------------------------------------------------
        LODI,R1 FRRAMP0         ; pointer = 0x000 ($0600)
        LODI,R2 0
        BSTA,UN FNREGWR
        LODI,R1 FRRAMPK         ; stream A1 A2 A3 A4
        LODA,R0 *FSELP,R1
        LODI,R1 $A1
RAML1:  LODA,R0 *FDATP,R1
        ADDI,R1 1
        COMI,R1 $A5
        BCFR,EQ RAML1
        LODI,R1 FRRAMP3         ; pointer = 0x3FC ($2AFC)
        LODI,R2 $FC
        BSTA,UN FNREGWR
        LODI,R1 FRRAMPK
        LODA,R0 *FSELP,R1
        LODI,R1 $B1
        LODA,R0 *FDATP,R1
        LODI,R1 $B2
        LODA,R0 *FDATP,R1
        LODI,R1 FRRAMP0         ; close the burst
        LODA,R0 *FSELP,R1

        LODI,R3 0               ; RAM-L reads back directly (page 0)
        LODI,R2 $A1
RAML2:  LODA,R0 $0600,R3
        COMZ    R2
        BCFR,EQ RAML3
        ADDI,R2 1
        ADDI,R3 1
        COMI,R3 4
        BCFR,EQ RAML2
        LODI,R0 1
        STRA,R0 RES+6
RAML3:  LODA,R0 *FRAMHP         ; RAM-H through a pointer (page 1)
        COMI,R0 $B1
        BCFR,EQ RAMH1
        LODI,R1 1
        LODA,R0 *FRAMHP,R1
        COMI,R0 $B2
        BCFR,EQ RAMH1
        LODI,R0 1
        STRA,R0 RES+7

; ---- 4. strays after the close store nothing -----------------------------
RAMH1:  LODI,R1 $55             ; moves the pointer only
        LODA,R0 *FDATP,R1
        LODI,R1 $66             ; goes on to fujimail: nothing is armed there
        LODA,R0 *FDATP,R1
        LODA,R0 $0655
        BCFR,EQ STR1
        LODA,R0 $0666
        BCFR,EQ STR1
        LODA,R0 $0604           ; just past the stream
        BCFR,EQ STR1
        LODI,R0 1
        STRA,R0 RES+8

; ---- 5. version and magic ------------------------------------------------
STR1:   LODA,R0 *FPVERP
        STRA,R0 RES+9
        BSTA,UN FNCHECK
        STRA,R0 RES+10

; ---- 6. the tick over 60 vblanks -----------------------------------------
        BSTA,UN TICKRD
        STRA,R1 RES+11
        STRA,R2 RES+12
        LODI,R0 60
        STRA,R0 NFRAME
TK1:    TPSU    $80             ; wait out this vblank...
        BCTR,EQ TK1
TK2:    TPSU    $80             ; ...and for the next one
        BCFR,EQ TK2
        LODA,R0 NFRAME
```

```
145         SUBI,R0 1
146         STRA,R0 NFRAME
147         BCFR,EQ TK1
148         BSTA,UN TICKRD
149         STRA,R1 RES+13
150         STRA,R2 RES+14
151         LODI,R0 $5A
152         STRA,R0 RES+15
153
154 ; ---- show the results, then the live tick --------------------------------
155         LODI,R3 0
156 SHOW1:  LODA,R0 RES,R3
157         STRZ    R1
158         LODZ    R3              ; cell = 32 + 2 * index - 1
159         ADDZ    R3
160         ADDI,R0 $20-1
161         STRZ    R2
162         LODZ    R1
163         BSTA,UN DHEX
164         ADDI,R3 1
165         COMI,R3 16
166         BCFR,EQ SHOW1
167 LIVE:   BSTA,UN TICKRD
168         LODZ    R1
169         STRZ    R3              ; DHEX keeps R3: the low byte waits there
170         LODZ    R2
171         LODI,R2 $90-1
172         BSTA,UN DHEX
173         LODZ    R3
174         BSTA,UN DHEX
175         BCTA,UN LIVE
176
177 ; Read the tick tear-free: R1 = lo, R2 = hi. Leaf. Clobbers R0.
178 TICKRD: LODA,R0 *FTKHP
179         STRZ    R2
180         LODA,R0 *FTKLP
181         STRZ    R1
182         LODA,R0 *FTKHP
183         COMZ    R2
184         BCFR,EQ TICKRD
185         RETC,UN
186
187 TSTR:   DB      "FUJIBANK",0
188
189 FBNKP:  DWBE    FNBANK
190 FPVERP: DWBE    FNPVER
191 FTKLP:  DWBE    FNTIKL
192 FTKHP:  DWBE    FNTIKH
193 FRAMHP: DWBE    RAMH
194
195 ; ---- page 0, in the base image -------------------------------------------
196         ORG     $0800
197         LODI,R0 $C0
198         RETC,UN
199         ORG     $0FFF
200         DB      0
201
202 ; ---- pages 1-3: the flat's $3000+, folded on by mkimage.py --pages ---------
203 BANKPG  MACRO   n
204         ORG     $3000+((n)-1)*$800
205         PHASE   $0800
206         LODI,R0 $C0+(n)
207         RETC,UN
208         DS      $0FFF-$
209         DB      n
210         DEPHASE
211         ENDM
212
213         BANKPG  1
214         BANKPG  2
215         BANKPG  3
216
```

)

D. GAME AND CONFIG LISTINGS

The files Sections 21 to 23 quote, at the revisions in the Parts List.

BATTLESHIP/INPUT.INC `battleship/input.inc`

```
; input.inc -- keypad + two-axis disc, folded into one code space.
;
; Three hardware facts shape this:
;   - FIRE is wired to the SAME line as keypad '2' ($1901 b3): they are
;     electrically indistinguishable. Outside the game screen that wire
;     is SELECT; inside it fires at the cursor.
;   - The P1 disc is an analog pot valid ONLY during vertical blank; read
;     elsewhere it looks like "down". Every scan syncs to a fresh vblank
;     EDGE, which also gives the game loop its frame clock for free: one
;     KEYSCN = one frame.
;   - BOTH disc axes arrive on $19FF; $19F9 bit 6 (the pot mux) selects
;     which -- CLEAR = vertical, SET = horizontal (MAME arcadia_v.cpp
;     case 0x1ff). Battleship needs both, for a cursor on a 10x10 grid.
;
; Reading two axes off one register without breaking edge detection is
; the whole design here. The mux is flipped ONCE PER FRAME and only the
; selected axis is sampled, but BOTH raw values are CACHED (AXV/AXH) and
; the direction is decoded from the cache. Sampling one axis per frame
; and reporting from both means a held direction reads the same every
; frame -- which is what edge detection needs. Setting the mux and then
; waiting a whole frame before the read also gives real 2637 silicon
; time to digitise with the new mux, which a flip-and-read-immediately
; scheme would not.
;
; Codes: 0-9 = the digits, KUP/KDOWN/KLEFT/KRIGHT = disc, KENT = Enter,
; KCLR = Clear, KNONE = nothing. '2' doubles as FIRE. Column bits
; priority-encode through 16-byte tables so a mashed pair cannot alias
; to a third key.

KNONE   EQU     $FF
; The four disc directions are CONTIGUOUS ($80-$83) so the auto-repeat
; gate below is two compares; Enter/Clear sit above them and never repeat.
KUP     EQU     $80
KDOWN   EQU     $81
KLEFT   EQU     $82
KRIGHT  EQU     $83
KENT    EQU     $84
KCLR    EQU     $85

JOYP1   EQU     $19FF           ; P1 pot; the axis depends on BGCOL b6
JOYLO   EQU     $40             ; < this = up / left
JOYHI   EQU     $C0             ; > this = down / right
JOYCTR  EQU     $6F             ; the resting value

RPTDLY  EQU     18              ; frames before a held disc repeats
RPTINT  EQU     4               ; frames between repeats

; WAITVB: wait for the next vertical-blank EDGE (SENSE low then high).
; Leaf. Preserves everything but the PSW condition code.
WAITVB: TPSU    $80
        BCTR,EQ WAITVB          ; still in the previous vblank
WVB1:   TPSU    $80
        BCFR,EQ WVB1            ; wait for the next one to start
        RETC,UN

; KEYRAW: R0 = code of the first control found this frame (KNONE if
; none). Costs exactly one frame. Calls one down (WAITVB).
; Clobbers R0,R1. Updates AXV/AXH, AXSEL.
KEYRAW: LODA,R0 AXSEL           ; arm the mux for the axis we will sample
        BCTR,EQ KRVERT
        LODA,R0 BGCBAS
        IORI,R0 $40             ; mux SET: horizontal
        BCTR,UN KRARM
KRVERT: LODA,R0 BGCBAS          ; mux CLEAR: vertical
KRARM:  STRA,R0 BGCOL
        BSTR,UN WAITVB          ; a whole frame with the mux settled
        LODA,R0 JOYP1
        STRZ    R1
        LODA,R0 AXSEL
        BCTR,EQ KRSTV
        LODZ    R1
        STRA,R0 AXH
        LODI,R0 0               ; alternate for next frame
        BCTR,UN KRSTD
KRSTV:  LODZ    R1
        STRA,R0 AXV
        LODI,R0 1
KRSTD:  STRA,R0 AXSEL
; Decode from the CACHE, not from this frame's sample: vertical wins
; over horizontal so a diagonal push resolves to one axis.
        LODA,R0 AXV
        COMI,R0 JOYLO
        BCTR,LT KRUP
        COMI,R0 JOYHI
        BCTR,GT KRDN
        LODA,R0 AXH
```

```
        COMI,R0 JOYLO
        BCTR,LT KRLF
        COMI,R0 JOYHI
        BCTR,GT KRRT
        LODA,R0 $1900           ; b3='1' b2='4' b1='7' b0=Clear
        ANDI,R0 $0F
        BCFR,EQ KRC0
        LODA,R0 $1901           ; b3='2'/FIRE b2='5' b1='8' b0='0'
        ANDI,R0 $0F
        BCFR,EQ KRC1
        LODA,R0 $1902           ; b3='3' b2='6' b1='9' b0=Enter
        ANDI,R0 $0F
        BCFR,EQ KRC2
        LODI,R0 KNONE
        RETC,UN
KRUP:   LODI,R0 KUP
        RETC,UN
KRDN:   LODI,R0 KDOWN
        RETC,UN
KRLF:   LODI,R0 KLEFT
        RETC,UN
KRRT:   LODI,R0 KRIGHT
        RETC,UN
KRC0:   STRZ    R1
        LODA,R0 *PKT0,R1
        RETC,UN
KRC1:   STRZ    R1
        LODA,R0 *PKT1,R1
        RETC,UN
KRC2:   STRZ    R1
        LODA,R0 *PKT2,R1
        RETC,UN


PKT0:   DWBE    KT0             ; the tables live in block 2 (ROM budget)
PKT1:   DWBE    KT1
PKT2:   DWBE    KT2

; KEYSCN: edge-detected KEYRAW -- a control reports once per press, and
; a rocked disc or second key reports without a full release. A HELD
; DISC DIRECTION auto-repeats (RPTDLY then every RPTINT frames): nine
; presses to cross the board would be miserable, and the repeat lives
; here rather than in the targeting loop so every screen gets it.
; Calls two down. Clobbers R0,R1.
KEYSCN: BSTA,UN KEYRAW
        COMA,R0 V_KEY
        BCTR,EQ KSHELD          ; unchanged: held or still idle
        STRA,R0 V_KEY           ; a fresh control: latch and arm the repeat
        STRZ    R1
        LODI,R0 RPTDLY
        STRA,R0 V_RPT
        LODZ    R1
        COMI,R0 KNONE
        BCTR,EQ KSNONE          ; the release edge reports nothing
        RETC,UN
KSHELD: COMI,R0 KUP             ; digits are < $80: never repeat
        BCTR,LT KSNONE
        COMI,R0 KRIGHT          ; KENT/KCLR/KNONE are > $83: never repeat
        BCTR,GT KSNONE
        LODA,R0 V_RPT
        SUBI,R0 1
        STRA,R0 V_RPT
        BCFR,EQ KSNONE
        LODI,R0 RPTINT
        STRA,R0 V_RPT
        LODA,R0 V_KEY           ; the held direction fires again
        RETC,UN
KSNONE: LODI,R0 KNONE
        RETC,UN

; GETKEY: block for a fresh control; R0 = code. Calls three down.
GETKEY: BSTR,UN KEYSCN
        COMI,R0 KNONE
        BCTR,EQ GETKEY
        RETC,UN

```

BATTLESHIP/NET.INC battleship/net.inc

```
; net.inc -- the N: device round trip.
;
; A 2650 transcription of astrocade/net.inc's proven sequence (which is
; o2/net.inc's), over mailbox.inc. The rules it carries forward, learned
; the hard way on the Odyssey 2 and Intellivision:
;
;  * The CLOSE belongs at the START of the next request, never after the
;    READ: every transaction repaints the whole reply window, a CLOSE
;    reply is empty, and every screen renders straight out of the window
;    between polls.
;  * FNRXLL/FNRXLH belong to the most recent transaction, so the reply
;    length is captured immediately after the READ.
;  * The ESP32 reports STATUS as soon as SOME of the response has
;    arrived; poll until two consecutive readings agree (bounded) rather
;    than guessing a delay.
;  * The 4th STATUS reply byte is nDevStatus_t and checking it is not
```

```
;    optional: the HTTP GET is deferred until the first STATUS, and an
;    HTTP error page still has a readable body.
;  * Validate the reply length against what playerCount implies before
;    believing a byte of it.
;
; All routines return R0 = 0 on success, nonzero on failure; the load
; sets CC, so callers branch on the return directly. MCOMMIT clobbers
; every register, so loop counters live in page-0 RAM cells reached
; through the pointer words.

; NCLOSE: close the previous request's connection. A first call has
; nothing to close and earns a NAK; both results are ignored.
; Calls one down. Clobbers R0-R3.
NCLOSE: BSTA,UN MSTART
        LODI,R1 FRCMD
        LODI,R2 NCCLOSE
        BSTA,UN MREGWR
        LODI,R1 FRNPARM
        LODI,R2 0
        BSTA,UN MREGWR
        BSTA,UN MCOMMIT
        EORZ    R0
        RETC,UN

; NOPEN: OPEN the devicespec as HTTP GET, no translation.
; Calls two down (BLDURL -> FNTXSTR). Clobbers R0-R3, FSTRP.
NOPEN:  BSTA,UN MSTART
        LODI,R1 FRCMD
        LODI,R2 NCOPEN
        BSTA,UN MREGWR
        LODI,R1 FRNPARM
        LODI,R2 2
        BSTA,UN MREGWR
        LODI,R2 NMHTTPG
        BSTA,UN MPARB
        LODI,R2 NTRNONE
        BSTA,UN MPARB
        BSTA,UN BLDURL
        BSTA,UN MCOMMIT
        BCFR,EQ NOBAD
        LODA,R0 FNREPLY
        COMI,R0 FCACK
        BCFR,EQ NOBAD
        EORZ    R0
        RETC,UN
NOBAD:  LODI,R0 $FF
        RETC,UN

; NSTATUS: one STATUS poll. On success AVAIL = bytes waiting.
; Calls one down. Clobbers R0-R3.
NSTATUS: BSTA,UN MSTART
        LODI,R1 FRCMD
        LODI,R2 NCSTAT
        BSTA,UN MREGWR
        LODI,R1 FRNPARM
        LODI,R2 2
        BSTA,UN MREGWR
        LODI,R2 0
        BSTA,UN MPARB
        LODI,R2 0
        BSTA,UN MPARB
        BSTA,UN MCOMMIT
        BCFR,EQ NSBAD
        LODA,R0 FNREPLY
        COMI,R0 FCACK
        BCFR,EQ NSBAD
        LODA,R0 FNRDATA+NSDEVST
        COMI,R0 1               ; SUCCESS -- anything else is an HTTP error
        BCFR,EQ NSBAD
        LODA,R0 FNRDATA+NSAVLO
        STRA,R0 *PAVA0
        LODA,R0 FNRDATA+NSAVHI
        STRA,R0 *PAVA1
        EORZ    R0
        RETC,UN
NSBAD:  LODI,R0 $FF
        RETC,UN

; NREAD: READ min(AVAIL, GMAXLEN) bytes into the reply window, then
; capture the reply length and re-home the slice cache.
; Calls one down. Clobbers R0-R3.
NREAD:  BSTA,UN MSTART
        LODI,R1 FRCMD
        LODI,R2 NCREAD
        BSTA,UN MREGWR
        LODI,R1 FRNPARM
        LODI,R2 1
        BSTA,UN MREGWR
        LODI,R1 2               ; one 2-byte parameter: the length, LE
        LODA,R0 FNDATA,R1
        LODA,R0 *PAVA1
        COMI,R0 GMAXLEN>>8
        BCTR,GT NRCLMP
        BCFR,EQ NRTAKE          ; hi < clamp hi: take avail
        LODA,R0 *PAVA0
        COMI,R0 GMAXLEN&$FF
        BCTR,GT NRCLMP
```

```
NRTAKE: LODA,R1 *PAVA0
        LODA,R0 FNDATA,R1
        LODA,R1 *PAVA1
        LODA,R0 FNDATA,R1
        BCTR,UN NRGO
NRCLMP: LODI,R1 GMAXLEN&$FF
        LODA,R0 FNDATA,R1
        LODI,R1 GMAXLEN>>8
        LODA,R0 FNDATA,R1
NRGO:   BSTA,UN MCOMMIT
        BCFR,EQ NRBAD
        LODA,R0 FNREPLY
        COMI,R0 FCACK
        BCFR,EQ NRBAD
        LODA,R0 FNRXLL          ; capture NOW: the next transaction's
        STRA,R0 *PRXL0          ; reply length would replace it
        LODA,R0 FNRXLH
        STRA,R0 *PRXL1
        EORZ    R0
        STRA,R0 *PCURSL         ; the cart republished slice 0
        RETC,UN
NRBAD:  LODI,R0 $FF
        RETC,UN

; ------------------------------------------------------------------------
; APICALL: R0 = request (UTABLES/USTATE/UREADY/UPLACE/UATTACK/ULEAVE). One full
; round trip; on success (R0 = 0) the reply is in the window with the
; slice cache homed, and stays valid until the next APICALL.
; Calls three down (NOPEN -> BLDURL -> FNTXSTR). Clobbers R0-R3 and the
; net RAM cells.
APICALL: STRA,R0 *PVURL
        BSTA,UN NCLOSE          ; the PREVIOUS request's connection
        BSTA,UN NOPEN
        BCFA,EQ APRET           ; open failed: report it (already closed)
        EORZ    R0
        STRA,R0 *PPRA0
        STRA,R0 *PPRA1
        LODI,R0 20              ; settle-loop bound
        STRA,R0 *PVTMP
APSET:  BSTA,UN NSTATUS
        BCFA,EQ APFAIL
        LODA,R0 *PAVA0
        BCFR,EQ APNZ
        LODA,R0 *PAVA1
        BCTR,EQ APAGN           ; nothing has arrived at all yet
APNZ:   LODA,R0 *PAVA0
        COMA,R0 *PPRA0
        BCFR,EQ APAGN
        LODA,R0 *PAVA1
        COMA,R0 *PPRA1
        BCTR,EQ APRD            ; two readings agree: it has landed
APAGN:  LODA,R0 *PAVA0
        STRA,R0 *PPRA0
        LODA,R0 *PAVA1
        STRA,R0 *PPRA1
        LODI,R3 3               ; ~3 frames between polls
APDLY:  BSTA,UN WAITVB
        BDRR,R3 APDLY
        LODA,R0 *PVTMP
        SUBI,R0 1
        STRA,R0 *PVTMP
        BCFA,EQ APSET
APFAIL: BSTA,UN NCLOSE          ; failed before the READ: nothing to keep
        LODI,R0 $FF
APRET:  RETC,UN
APRD:   BSTA,UN NREAD
        BCFA,EQ APFAIL
; Validation failure after the READ returns nonzero WITHOUT a close --
; the next APICALL's leading NCLOSE retires the connection, and closing
; here would wipe the window a caller may still be rendering the last
; good reply from. Falls through into VALID8.

; VALID8: sanity-check RXLEN for the request in V_URL, transcribed from
; the astrocade port's (itself the Intellivision's validate_state).
; Tables: nonempty -- the count byte bounds the walk. Otherwise:
; playerCount <= 4, a status byte on the enum, and AT LEAST the length
; that status and playerCount imply:
;     lobby  59 + 10*pc     place  49 (records are name+status only)
;     game   49 + 115*pc
; Minimum, not exact: the serializer's per-player append makes the place
; phase variable and a longer reply is harmless; a SHORTER one means the
; fixed offsets would read stale bytes out of a window that is never
; cleared. Leaf. Clobbers R0-R3.
VALID8: LODA,R0 *PVURL
        BCFR,EQ VALGM
        LODA,R0 *PRXL0          ; tables: any nonzero length
        BCFA,EQ VALOK
        LODA,R0 *PRXL1
        BCFA,EQ VALOK
        BCTA,UN VALBAD
VALGM:  LODA,R0 FNRDATA+GOPLCNT
        COMI,R0 5
        BCFA,LT VALBAD          ; more seats than the game has
        STRZ    R3              ; R3 = playerCount
        LODA,R0 FNRDATA+GOSTAT
        LODI,R1 6               ; status must be on the enum: everything
VALS1:  COMA,R0 STATTB,R1       ;   downstream dispatches on it
```

```
211         BCTR,EQ VALS2
212         BDRR,R1 VALS1
213         COMA,R0 STATTB          ; BDRR never runs the body for index 0
214         BCFA,EQ VALBAD
215 VALS2:  COMI,R0 STLOBBY
216         BCFR,EQ VALS3
217         LODZ    R3              ; lobby: 59 + 10*pc
218         RRL,R0
219         STRZ    R1
220         LODA,R0 EXPLOB,R1
221         STRZ    R2
222         LODA,R0 EXPLOB+1,R1
223         BCTR,UN VALCMP
224 VALS3:  COMI,R0 STPLACE
225         BCFR,EQ VALS4
226         LODI,R2 0               ; place: header + lastAttack + myShips
227         LODI,R0 GIPLYRS
228         BCTR,UN VALCMP
229 VALS4:  LODZ    R3              ; game/over: 49 + 115*pc
230         RRL,R0
231         STRZ    R1
232         LODA,R0 EXPGAM,R1
233         STRZ    R2
234         LODA,R0 EXPGAM+1,R1
235 ; RXLEN >= (R2:R0)? Compare high, then low only when the highs agree.
236 VALCMP: STRZ    R1              ; R1 = expected low
237         LODA,R0 *PRXL1
238         COMZ    R2
239         BCTR,LT VALBAD          ; RXLEN high < expected high: short
240         BCFR,EQ VALOK           ; RXLEN high > expected high: long enough
241         LODA,R0 *PRXL0
242         COMZ    R1
243         BCTR,LT VALBAD
244 VALOK:  EORZ    R0
245         RETC,UN
246 VALBAD: LODI,R0 $FF
247         RETC,UN
248
249 STATTB: DB      STLOBBY,STPLACE,STSTART,STMISS,STHIT,STSUNK,STOVER
250
251 EXPLOB: DWBE    GLPLYRS+0*LBSTRID
252         DWBE    GLPLYRS+1*LBSTRID
253         DWBE    GLPLYRS+2*LBSTRID
254         DWBE    GLPLYRS+3*LBSTRID
255         DWBE    GLPLYRS+4*LBSTRID
256
257 EXPGAM: DWBE    GIPLYRS+0*PLSTRID
258         DWBE    GIPLYRS+1*PLSTRID
259         DWBE    GIPLYRS+2*PLSTRID
260         DWBE    GIPLYRS+3*PLSTRID
261         DWBE    GIPLYRS+4*PLSTRID
262
```

BATTLESHIP/PLACE.INC battleship/place.inc

```
  1 ; place.inc -- laying out your five ships.
  2 ;
  3 ; A random legal layout is seeded first and then adjusted, which is the
  4 ; astrocade port's model: the player is never made to build a board from
  5 ; nothing, and the seed is guaranteed legal.
  6 ;
  7 ; OVERLAP TESTING USES THE SCREEN AS THE OCCUPANCY MAP. A 10x10 byte
  8 ; array is 100 bytes and this machine has 84 in total, so instead the
  9 ; board is repainted carrying every ship EXCEPT the one being moved, and
 10 ; a candidate is legal when every cell it would cover reads as open sea.
 11 ; Storing NOPLC into the moving ship's slot for the duration of the
 12 ; repaint is what excludes it -- YSHIPR already skips NOPLC -- so the
 13 ; whole trick costs nothing but the two bytes that shelter the slot.
 14 ;
 15 ; V_CURS, not V_SHP, holds the ship being adjusted: V_SHP is YSHIPR's
 16 ; loop variable and does not survive a board repaint.
 17
 18 PLACSCR: BSTA,UN SEEDRD         ; reseed -- by now ACKSEQ has moved, so
 19         BSTA,UN PLSEED          ;   the layout differs run to run
 20         LODI,R0 0
 21         STRA,R0 V_CURS
 22         BSTA,UN PLSHOW
 23 PLKEY:  BSTA,UN GETKEY
 24         STRZ    R1
 25         COMI,R0 KUP
 26         BCTR,EQ PLMV
 27         COMI,R0 KDOWN
 28         BCTR,EQ PLMV
 29         COMI,R0 KLEFT
 30         BCTR,EQ PLMV
 31         COMI,R0 KRIGHT
 32         BCTR,EQ PLMV
 33         COMI,R0 5               ; keypad 5 rotates
 34         BCTA,EQ PLROT
 35         COMI,R0 KENT            ; Enter or FIRE commits this ship
 36         BCTR,EQ PLCOMM
 37         COMI,R0 2
 38         BCTR,EQ PLCOMM
 39         COMI,R0 KCLR            ; Clear leaves the table
 40         BCTA,EQ PLQUIT
```

```
         BCTA,UN PLKEY
PLMV:    BSTA,UN PLMOVE
         BCTA,UN PLKEY
PLCOMM:  BSTA,UN SNDPLC
         LODA,R0 V_CURS
         ADDI,R0 1
         STRA,R0 V_CURS
         COMI,R0 NSHIPS
         BCTR,EQ PLDONE
         BSTA,UN PLSHOW
         BCTA,UN PLKEY
PLDONE:  BSTA,UN PLFINL          ; every hull settled, none highlighted
         LODI,R0 RQPLACE         ; the layout rides the next poll
         STRA,R0 PENDACT
         RETC,UN
PLQUIT:  LODI,R0 RQLEAVE
         STRA,R0 PENDACT
         RETC,UN

; PLSHOW: repaint the board with the settled hulls, then the ship under
; the cursor in red on top, and the prompt.
; Calls four down. Clobbers R0-R3.
PLSHOW:  BSTA,UN PLDRWO
         BSTA,UN PLCUR
         BCTA,UN PLHINT          ; tail call

; PLFINL: the finished layout, nothing highlighted.
; Calls four down. Clobbers R0-R3.
PLFINL:  BSTA,UN YBCLR
         BCTA,UN YSHIPR          ; tail call

; PLDRWO: repaint your board carrying every hull EXCEPT the one being
; adjusted, which is what turns the screen into the occupancy map.
; Calls four down (YSHIPR -> SHIPDR -> PLCDEC/DPOS). Clobbers R0-R3.
PLDRWO:  LODA,R0 V_CURS
         STRZ    R2
         LODA,R0 SHIPS,R2
         STRA,R0 V_SAVP          ; shelter it across the repaint
         LODI,R0 NOPLC
         STRA,R0 SHIPS,R2
         BSTA,UN YBCLR
         BSTA,UN YSHIPR
         LODA,R0 V_CURS
         STRZ    R2
         LODA,R0 V_SAVP
         STRA,R0 SHIPS,R2
         RETC,UN

; PLCUR: draw the ship under the cursor, red, over the repainted board.
; Calls three down. Clobbers R0-R3.
PLCUR:   LODA,R0 V_CURS
         STRA,R0 V_SHP
         STRZ    R2
         LODA,R0 SHIPS,R2
         COMI,R0 NOPLC
         BCTR,EQ PLC9
         STRA,R0 V_PLC
         LODI,R0 AHIT
         STRA,R0 DCOLOR
         BCTA,UN SHIPDR          ; tail call
PLC9:    RETC,UN

; PLMOVE: shift the ship under the cursor one cell in direction R1.
; Works in (x, y, dir) rather than on the packed position so a sideways
; step can never wrap across a row. Illegal moves are refused, not
; clamped. Calls four down. Clobbers R0-R3.
PLMOVE:  STRA,R1 V_BTMP
         LODA,R0 V_CURS
         STRA,R0 V_SHP
         STRZ    R2
         LODA,R0 SHIPS,R2
         STRA,R0 V_PLC
         BSTA,UN PLCDEC          ; -> V_SX, V_SY, V_DIR, V_LEN
         LODA,R0 V_BTMP
         COMI,R0 KUP
         BCFR,EQ PLMD
         LODA,R0 V_SY
         BCTA,EQ PLMBAD
         SUBI,R0 1
         STRA,R0 V_SY
         BCTR,UN PLMTRY
PLMD:    COMI,R0 KDOWN
         BCFR,EQ PLML
         LODA,R0 V_SY
         ADDI,R0 1
         COMI,R0 BRDDIM
         BCTA,EQ PLMBAD
         STRA,R0 V_SY
         BCTR,UN PLMTRY
PLML:    COMI,R0 KLEFT
         BCFR,EQ PLMR
         LODA,R0 V_SX
         BCTA,EQ PLMBAD
         SUBI,R0 1
         STRA,R0 V_SX
         BCTR,UN PLMTRY
PLMR:    LODA,R0 V_SX
```

```
138         ADDI,R0 1
139         COMI,R0 BRDDIM
140         BCTA,EQ PLMBAD
141         STRA,R0 V_SX
142 PLMTRY: BSTA,UN PLENC
143         BCTA,UN PLTRY           ; tail call
144 PLMBAD: BCTA,UN SNDERR          ; tail call
145
146 ; PLROT: flip the ship under the cursor between horizontal and vertical.
147 ; Calls four down. Clobbers R0-R3.
148 PLROT:  LODA,R0 V_CURS
149         STRA,R0 V_SHP
150         STRZ    R2
151         LODA,R0 SHIPS,R2
152         STRA,R0 V_PLC
153         BSTA,UN PLCDEC
154         LODA,R0 V_DIR
155         EORI,R0 1
156         STRA,R0 V_DIR
157         BSTA,UN PLENC
158         BCTA,UN PLTRY           ; tail call
159
160 ; PLENC: V_PLC = V_SY*10 + V_SX + 100*V_DIR.
161 ; Calls one down (MUL10). Clobbers R0,R1,R3.
162 PLENC:  LODA,R0 V_SY
163         BSTA,UN MUL10
164         ADDA,R0 V_SX
165         STRZ    R3
166         LODA,R0 V_DIR
167         BCTR,EQ PLE1
168         LODZ    R3
169         ADDI,R0 100
170         STRZ    R3
171 PLE1:   LODZ    R3
172         STRA,R0 V_PLC
173         RETC,UN
174
175 ; PLTRY: commit V_PLC for the ship under the cursor if it is legal,
176 ; otherwise leave the old placement alone and complain.
177 ; Calls four down. Clobbers R0-R3.
178 PLTRY:  BSTA,UN PLDRWO          ; the screen becomes the occupancy map
179         BSTA,UN TESTSHP
180         BCFR,EQ PLTBAD
181         LODA,R0 V_CURS
182         STRZ    R2
183         LODA,R0 V_PLC
184         STRA,R0 SHIPS,R2
185         BSTA,UN PLCUR
186         BCTA,UN SNDMOVE         ; tail call
187 PLTBAD: LODA,R0 V_CURS          ; put the settled one back on screen
188         STRZ    R2
189         LODA,R0 SHIPS,R2
190         STRA,R0 V_PLC
191         BSTA,UN PLCUR
192         BCTA,UN SNDERR          ; tail call
193
194 ; TESTSHP: is V_PLC legal for the ship under the cursor? R0 = 0 if so.
195 ; Assumes the board currently carries every OTHER hull (PLDRWO). Reads
196 ; the screen back through the same auto-increment cursor the writer uses.
197 ; Calls one down (PLCDEC, DPOS). Clobbers R0-R3.
198 TESTSHP: LODA,R0 V_CURS
199         STRA,R0 V_SHP
200         BSTA,UN PLCDEC
201         LODA,R0 V_DIR           ; does it run off the edge?
202         BCTR,EQ TSH
203         LODA,R0 V_SY
204         BCTR,UN TSB
205 TSH:    LODA,R0 V_SX
206 TSB:    ADDA,R0 V_LEN
207         COMI,R0 BRDDIM
208         BCTA,GT TSBAD
209         LODI,R0 0
210         STRA,R0 V_SEG
211 TS1:    LODA,R0 V_SY
212         ADDI,R0 YBROW
213         STRZ    R1
214         LODA,R0 V_SX
215         ADDI,R0 BCOL
216         STRZ    R2
217         BSTA,UN DPOS
218         LODA,R0 *SCRP,R2,+
219         ANDI,R0 $3F
220         BCFR,EQ TSBAD           ; not open sea: another hull is there
221         LODA,R0 V_DIR
222         BCTR,EQ TSSX
223         LODA,R0 V_SY
224         ADDI,R0 1
225         STRA,R0 V_SY
226         BCTR,UN TSNX
227 TSSX:   LODA,R0 V_SX
228         ADDI,R0 1
229         STRA,R0 V_SX
230 TSNX:   LODA,R0 V_SEG
231         ADDI,R0 1
232         STRA,R0 V_SEG
233         COMA,R0 V_LEN
234         BCFA,EQ TS1
```

```
235         EORZ    R0
236         RETC,UN
237 TSBAD:  LODI,R0 $FF
238         RETC,UN
239
240 ; PLSEED: a random legal layout. Each ship is tried against a board
241 ; carrying only the ships already seeded -- the later ones are still
242 ; NOPLC, so YSHIPR skips them and one repaint serves every attempt.
243 ; Calls five down. Clobbers R0-R3.
244 PLSEED: LODI,R3 NSHIPS-1        ; clear the slate first
245         LODI,R0 NOPLC
246 PLS1:   STRA,R0 SHIPS,R3
247         BDRR,R3 PLS1
248         LODI,R0 NOPLC
249         STRA,R0 SHIPS
250         LODI,R0 0
251         STRA,R0 V_CURS
252 PLS2:   BSTA,UN YBCLR           ; the ships seeded so far
253         BSTA,UN YSHIPR
254         LODI,R0 200             ; a bounded hunt; the board is roomy
255         STRA,R0 V_COL
256 PLS3:   LODI,R1 2
257         BSTA,UN RNDMOD
258         COMI,R0 0               ; RNDMOD exits on COMZ R1, so the CC
259         BCTR,EQ PLS4            ;   reflects the COMPARE, not the value --
260                                 ;   branching on it directly seeded every
261                                 ;   ship vertical
262         LODI,R0 100             ; vertical
263         BCTR,UN PLS5
264 PLS4:   LODI,R0 0
265 PLS5:   STRA,R0 V_BTMP
266         LODI,R1 BRDCELL
267         BSTA,UN RNDMOD
268         ADDA,R0 V_BTMP
269         STRA,R0 V_PLC
270         BSTA,UN TESTSHP
271         BCTR,EQ PLS6
272         LODA,R0 V_COL           ; rejected: try again
273         SUBI,R0 1
274         STRA,R0 V_COL
275         BCFA,EQ PLS3
276         LODI,R0 0               ; gave up: park it at the origin so the
277         STRA,R0 V_PLC           ;   slot is never left NOPLC
278 PLS6:   LODA,R0 V_CURS
279         STRZ    R2
280         LODA,R0 V_PLC
281         STRA,R0 SHIPS,R2
282         LODA,R0 V_CURS
283         ADDI,R0 1
284         STRA,R0 V_CURS
285         COMI,R0 NSHIPS
286         BCFA,EQ PLS2
287         RETC,UN
288
289 ; PLHINT: the placement prompt.
290 ; Calls three down. Clobbers R0-R3.
291 PLHINT: DAT     STATROW,0
292         DCOL    ATEXT
293         LODI,R1 16
294         SETSTR  TPLACE
295         BSTA,UN DPRTN
296         DAT     FOOTROW,0
297         LODI,R1 16
298         SETSTR  TPLKEY
299         BCTA,UN DPRTN           ; tail call
300
301
```

5CARDSTUD/STATE.INC 5cardstud/state.inc

```
1 ; state.inc -- reading the reply where it lies.
2 ;
3 ; The reply stays painted in the cart's 256-byte slice window until the
4 ; next transaction, so nothing is buffered: page-0 code sets the flat
5 ; cursor (RXPTR = slice, RXPTR+1 = offset -- both page-0 RAM it can
6 ; store directly) and then calls RXNEXT per byte. RXNEXT switches slices
7 ; only when the cursor's slice differs from the cached CURSLC; every
8 ; transaction ends with the cart republishing slice 0 and net.inc
9 ; re-homing the cache.
10
11 ; RXNEXT: R0 = the reply byte at the cursor; the cursor advances.
12 ; PRESERVES R2 (render loops keep their screen cell there). Calls one
13 ; down (MSLICE, on a slice cross only). Clobbers R0,R1,R3.
14 RXNEXT: LODA,R0 *PRXSLC
15         COMA,R0 *PCURSL
16         BCTR,EQ RXN1
17         STRA,R2 *PSAV2          ; shelter the caller's R2
18         STRZ    R2              ; wanted slice
19         STRA,R0 *PCURSL
20         BSTA,UN MSLICE
21         LODA,R2 *PSAV2
22 RXN1:   LODA,R1 *PRXOFF
23         LODA,R0 FNRDATA,R1      ; the byte
24         STRZ    R3
25         ADDI,R1 1
```

```
        STRA,R1 *PRXOFF
        BCFR,EQ RXN2            ; wrapped to 0: next slice
        LODA,R0 *PRXSLC
        ADDI,R0 1
        STRA,R0 *PRXSLC
RXN2:   LODZ    R3
        RETC,UN

```

5CARDSTUD/NET.INC 5cardstud/net.inc

```
; net.inc -- the N: device round trip.
;
; A 2650 transcription of astrocade/net.inc's proven sequence (which is
; o2/net.inc's), over mailbox.inc. The rules it carries forward, learned
; the hard way on the Odyssey 2 and Intellivision:
;
;  * The CLOSE belongs at the START of the next request, never after the
;    READ: every transaction repaints the whole reply window, a CLOSE
;    reply is empty, and every screen renders straight out of the window
;    between polls.
;  * FNRXLL/FNRXLH belong to the most recent transaction, so the reply
;    length is captured immediately after the READ.
;  * The ESP32 reports STATUS as soon as SOME of the response has
;    arrived; poll until two consecutive readings agree (bounded) rather
;    than guessing a delay.
;  * The 4th STATUS reply byte is nDevStatus_t and checking it is not
;    optional: the HTTP GET is deferred until the first STATUS, and an
;    HTTP error page still has a readable body.
;  * Validate the reply length against what playerCount implies before
;    believing a byte of it.
;
; All routines return R0 = 0 on success, nonzero on failure; the load
; sets CC, so callers branch on the return directly. MCOMMIT clobbers
; every register, so loop counters live in page-0 RAM cells reached
; through the pointer words.

; NCLOSE: close the previous request's connection. A first call has
; nothing to close and earns a NAK; both results are ignored.
; Calls one down. Clobbers R0-R3.
NCLOSE: BSTA,UN MSTART
        LODI,R1 FRCMD
        LODI,R2 NCCLOSE
        BSTA,UN MREGWR
        LODI,R1 FRNPARM
        LODI,R2 0
        BSTA,UN MREGWR
        BSTA,UN MCOMMIT
        EORZ    R0
        RETC,UN

; NOPEN: OPEN the devicespec as HTTP GET, no translation.
; Calls two down (BLDURL -> FNTXSTR). Clobbers R0-R3, FSTRP.
NOPEN:  BSTA,UN MSTART
        LODI,R1 FRCMD
        LODI,R2 NCOPEN
        BSTA,UN MREGWR
        LODI,R1 FRNPARM
        LODI,R2 2
        BSTA,UN MREGWR
        LODI,R2 NMHTTPG
        BSTA,UN MPARB
        LODI,R2 NTRNONE
        BSTA,UN MPARB
        BSTA,UN BLDURL
        BSTA,UN MCOMMIT
        BCFR,EQ NOBAD
        LODA,R0 FNREPLY
        COMI,R0 FCACK
        BCFR,EQ NOBAD
        EORZ    R0
        RETC,UN
NOBAD:  LODI,R0 $FF
        RETC,UN

; NSTATUS: one STATUS poll. On success AVAIL = bytes waiting.
; Calls one down. Clobbers R0-R3.
NSTATUS: BSTA,UN MSTART
        LODI,R1 FRCMD
        LODI,R2 NCSTAT
        BSTA,UN MREGWR
        LODI,R1 FRNPARM
        LODI,R2 2
        BSTA,UN MREGWR
        LODI,R2 0
        BSTA,UN MPARB
        LODI,R2 0
        BSTA,UN MPARB
        BSTA,UN MCOMMIT
        BCFR,EQ NSBAD
        LODA,R0 FNREPLY
        COMI,R0 FCACK
        BCFR,EQ NSBAD
        LODA,R0 FNRDATA+NSDEVST
        COMI,R0 1               ; SUCCESS -- anything else is an HTTP error
```

```
         BCFR,EQ NSBAD
         LODA,R0 FNRDATA+NSAVLO
         STRA,R0 *PAVA0
         LODA,R0 FNRDATA+NSAVHI
         STRA,R0 *PAVA1
         EORZ    R0
         RETC,UN
NSBAD:   LODI,R0 $FF
         RETC,UN

; NREAD: READ min(AVAIL, GMAXLEN) bytes into the reply window, then
; capture the reply length and re-home the slice cache.
; Calls one down. Clobbers R0-R3.
NREAD:   BSTA,UN MSTART
         LODI,R1 FRCMD
         LODI,R2 NCREAD
         BSTA,UN MREGWR
         LODI,R1 FRNPARM
         LODI,R2 1
         BSTA,UN MREGWR
         LODI,R1 2               ; one 2-byte parameter: the length, LE
         LODA,R0 FNDATA,R1
         LODA,R0 *PAVA1
         COMI,R0 GMAXLEN>>8
         BCTR,GT NRCLMP
         BCFR,EQ NRTAKE          ; hi < clamp hi: take avail
         LODA,R0 *PAVA0
         COMI,R0 GMAXLEN&$FF
         BCTR,GT NRCLMP
NRTAKE:  LODA,R1 *PAVA0
         LODA,R0 FNDATA,R1
         LODA,R1 *PAVA1
         LODA,R0 FNDATA,R1
         BCTR,UN NRGO
NRCLMP:  LODI,R1 GMAXLEN&$FF
         LODA,R0 FNDATA,R1
         LODI,R1 GMAXLEN>>8
         LODA,R0 FNDATA,R1
NRGO:    BSTA,UN MCOMMIT
         BCFR,EQ NRBAD
         LODA,R0 FNREPLY
         COMI,R0 FCACK
         BCFR,EQ NRBAD
         LODA,R0 FNRXLL          ; capture NOW: the next transaction's
         STRA,R0 *PRXL0          ; reply length would replace it
         LODA,R0 FNRXLH
         STRA,R0 *PRXL1
         EORZ    R0
         STRA,R0 *PCURSL         ; the cart republished slice 0
         RETC,UN
NRBAD:   LODI,R0 $FF
         RETC,UN

; ------------------------------------------------------------------------
; APICALL: R0 = request (0 tables, 1 state, 2 move, 3 leave). One full
; round trip; on success (R0 = 0) the reply is in the window with the
; slice cache homed, and stays valid until the next APICALL.
; Calls three down (NOPEN -> BLDURL -> FNTXSTR). Clobbers R0-R3 and the
; net RAM cells.
APICALL: STRA,R0 *PVURL
         BSTA,UN NCLOSE          ; the PREVIOUS request's connection
         BSTA,UN NOPEN
         BCFA,EQ APRET           ; open failed: report it (already closed)
         EORZ    R0
         STRA,R0 *PPRA0
         STRA,R0 *PPRA1
         LODI,R0 20              ; settle-loop bound
         STRA,R0 *PVTMP
APSET:   BSTA,UN NSTATUS
         BCFA,EQ APFAIL
         LODA,R0 *PAVA0
         BCFR,EQ APNZ
         LODA,R0 *PAVA1
         BCTR,EQ APAGN           ; nothing has arrived at all yet
APNZ:    LODA,R0 *PAVA0
         COMA,R0 *PPRA0
         BCFR,EQ APAGN
         LODA,R0 *PAVA1
         COMA,R0 *PPRA1
         BCTR,EQ APRD            ; two readings agree: it has landed
APAGN:   LODA,R0 *PAVA0
         STRA,R0 *PPRA0
         LODA,R0 *PAVA1
         STRA,R0 *PPRA1
         LODI,R3 3               ; ~3 frames between polls
APDLY:   BSTA,UN WAITVB
         BDRR,R3 APDLY
         LODA,R0 *PVTMP
         SUBI,R0 1
         STRA,R0 *PVTMP
         BCFA,EQ APSET
APFAIL:  BSTA,UN NCLOSE          ; failed before the READ: nothing to keep
         LODI,R0 $FF
APRET:   RETC,UN
APRD:    BSTA,UN NREAD
         BCFA,EQ APFAIL
; Validation failure after the READ returns nonzero WITHOUT a close --
```

```
; the next APICALL's leading NCLOSE retires the connection, and closing
; here would wipe the window a caller may still be rendering the last
; good reply from. Falls through into VALID8.

; VALID8: sanity-check RXLEN for the request in V_URL. Tables: nonempty.
; Game: exactly 154 + 33*playerCount, playerCount < 9 (table-driven --
; pc*33 overflows 8 bits, the 9-entry DWBE table does not).
VALID8: LODA,R0 *PVURL
        BCFR,EQ VALGM
        LODA,R0 *PRXL0          ; tables: any nonzero length; the count
        BCFR,EQ VALOK           ; byte bounds the walk
        LODA,R0 *PRXL1
        BCFR,EQ VALOK
        BCTR,UN VALBAD
VALGM:  LODA,R0 FNRDATA+GOPLCNT
        COMI,R0 9
        BCFR,LT VALBAD          ; more seats than the server has
        RRL,R0                  ; pc*2: table index
        STRZ    R1
        LODA,R0 EXPTB,R1
        COMA,R0 *PRXL1
        BCFR,EQ VALBAD
        LODA,R0 EXPTB+1,R1
        COMA,R0 *PRXL0
        BCFR,EQ VALBAD
VALOK:  EORZ    R0
        RETC,UN
VALBAD: LODI,R0 $FF
        RETC,UN

EXPTB:  DWBE    GMINLEN+0*PLSTRID
        DWBE    GMINLEN+1*PLSTRID
        DWBE    GMINLEN+2*PLSTRID
        DWBE    GMINLEN+3*PLSTRID
        DWBE    GMINLEN+4*PLSTRID
        DWBE    GMINLEN+5*PLSTRID
        DWBE    GMINLEN+6*PLSTRID
        DWBE    GMINLEN+7*PLSTRID
        DWBE    GMINLEN+8*PLSTRID

```

CONFIG/MAILBOX.INC config/mailbox.inc

```
; mailbox.inc -- the cartridge mailbox, spoken from page 1 (block 2).
;
; Mirrors fujinet-firmware/pico/arcadia/firmware/include/fuji_mailbox.h,
; the single source of truth; every equate is a CONSOLE address (image
; offset + $1000 for image $1000 and up). The connector has no write
; strobe, so both directions ride reads:
;   - a register write is a REGSEL read of FNRSEL+reg and then, back to
;     back, a REGDATA read of FNRDAT+value (the pairing is the cart's
;     stray-read defense);
;   - a TX byte is a read of FNDATA+value;
;   - the reply window and status page are ROM bytes the cart repaints,
;     only on a commit or a slice select -- which is what lets screens
;     draw, and stream TX, straight out of the window.
; Block 2 shares page 1 with all of it, so every access here is a direct
; (indexed) load. Page-0 code reaches the window through FRDP and
; friends at the bottom of txlib.inc.
;
; SEQ is always the cart's own persisted ACKSEQ + 1 (255 wraps to 1, 0
; reserved), never a local counter: console RESET restarts CONFIG but
; not the cart.

; ---- reply bytes (repainted ROM) ----------------------------------------
FNRDATA EQU     $2B00           ; reply slice 0 (every CONFIG reply fits)
FNACKSQ EQU     $2C00           ; echoes SEQ once the reply is ready
FNSTATS EQU     $2C01           ; b0 link up
FNERR   EQU     $2C02           ; fb_status of the last transaction
FNREPLY EQU     $2C03           ; $06 ACK / $15 NAK
FNBSTAT EQU     $2C06           ; boot state
FNBPCT  EQU     $2C07           ; boot progress 0-100
FNBERR  EQU     $2C08           ; boot error
FNMAGF  EQU     $2C09           ; 'F'
FNMAGN  EQU     $2C0A           ; 'N'

; ---- hotspot pages ------------------------------------------------------
FNRSEL  EQU     $2D00           ; read +reg: arm register
FNRDAT  EQU     $2E00           ; read +val: armed register = val
FNDATA  EQU     $2F00           ; read +val: append val to the TX stream
FNSWAP  EQU     $2DFE           ; armed only: serve the staged image

; ---- registers and values -----------------------------------------------
FRDEVIC EQU     $00
FRCMD   EQU     $01
FRNPARM EQU     $02
FRDATRS EQU     $05
FRSEQ   EQU     $10
FRBOOTL EQU     $11             ; BOOTLOCK: FNBLMAG arms the swap
FNBLMAG EQU     $B5
FNDFUJI EQU     $70             ; FujiBus id of the FujiNet itself
FCACK   EQU     $06
FCNAK   EQU     $15
FETIMO  EQU     $FF             ; helper-local: the commit gave up
FBIDLE  EQU     0               ; FNBSTAT values
```

```
FBXFER  EQU     1
FBREADY EQU     2
FBFAIL  EQU     $80

; ---- Fuji commands -------------------------------------------------------
FCACFGX EQU     $C4             ; GET_ADAPTERCONFIG_EXTENDED
FCCOPY  EQU     $D8             ; COPY_FILE ("src|dst", 1-based hosts)
FCSDFP  EQU     $E2             ; SET_DEVICE_FULLPATH
FCSDPS  EQU     $E4             ; SET_DIRECTORY_POSITION
FCWEN   EQU     $EA             ; GET_WIFI_ENABLED
FCWHSL  EQU     $F3             ; WRITE_HOST_SLOTS
FCRHSL  EQU     $F4             ; READ_HOST_SLOTS
FCCDIR  EQU     $F5             ; CLOSE_DIRECTORY
FCRDIR  EQU     $F6             ; READ_DIR_ENTRY
FCODIR  EQU     $F7             ; OPEN_DIRECTORY
FCMIMG  EQU     $F8             ; MOUNT_IMAGE
FCMHOST EQU     $F9             ; MOUNT_HOST
FCWSTAT EQU     $FA             ; GET_WIFISTATUS (3 = connected)
FCSSID  EQU     $FB             ; SET_SSID (ssid[33] + pass[64])
FCSCRES EQU     $FC             ; GET_SCAN_RESULT (ssid[33] + rssi)
FCSCAN  EQU     $FD             ; SCAN_NETWORKS
FCGSSID EQU     $FE             ; GET_SSID

; ---- routines -----------------------------------------------------------

; MREGWR: mailbox register R1 = value R2. Leaf. Clobbers R0.
MREGWR: LODA,R0 FNRSEL,R1
        LODA,R0 FNRDAT,R2
        RETC,UN

; MSTART: begin a Fuji-device transaction: TX rewound, device $70,
; command R3, R2 parameters to follow. Leaf. Clobbers R0 only.
MSTART: LODA,R0 FNRSEL+FRDATRS
        LODA,R0 FNRDAT
        LODA,R0 FNRSEL+FRDEVIC
        LODA,R0 FNRDAT+FNDFUJI
        LODA,R0 FNRSEL+FRCMD
        LODA,R0 FNRDAT,R3
        LODA,R0 FNRSEL+FRNPARM
        LODA,R0 FNRDAT,R2
        RETC,UN

; MPARB: append a 1-byte parameter, value R1. Leaf. Clobbers R0.
MPARB:  LODA,R0 FNDATA+1
        LODA,R0 FNDATA,R1
        RETC,UN

; MSEQ: write SEQ = ACKSEQ + 1. R0 = the SEQ written, i.e. the ACKSEQ
; that will mean "done". Leaf. Clobbers R0,R2.
MSEQ:   LODA,R0 FNACKSQ
        ADDI,R0 1
        BCFR,EQ MSQ1
        LODI,R0 1               ; 255 wraps past the reserved 0
MSQ1:   STRZ    R2
        LODA,R0 FNRSEL+FRSEQ
        LODA,R0 FNRDAT,R2
        LODZ    R2
        RETC,UN

; MCOMMIT: commit and poll for the ACKSEQ echo. R0 = FNERR (0 = ok) or
; FETIMO after ~12 s: 64K polls of ~180 us, past the cart's worst case
; for an ordinary command (3 s link wait + 5 s transact) -- a client
; that gave up early would reuse the SEQ and read the late reply as its
; own. Calls one down. Clobbers R0-R3.
MCOMMIT: BSTR,UN MSEQ
        LODI,R3 0               ; outer x256
MCMO:   LODI,R1 0               ; inner x256
MCMI:   LODI,R0 8               ; pad the poll (~12 NOPs' worth)
MCMD:   BDRR,R0 MCMD
        LODA,R0 FNACKSQ
        COMZ    R2
        BCTR,EQ MCMOK
        BDRR,R1 MCMI
        BDRR,R3 MCMO
        LODI,R0 FETIMO
        RETC,UN
MCMOK:  LODA,R0 FNERR
        RETC,UN

; CGO: commit and validate. R0 = 0 with CC EQ on an ACK; otherwise R0 is
; FNERR, FETIMO or the reply byte, with CC not EQ. CGOCHK validates a
; commit someone else waited out (R0 = FNERR). Calls two down.
; Clobbers R0-R3.
CGO:    BSTR,UN MCOMMIT
CGOCHK: COMI,R0 0
        BCFR,EQ CGO9
        LODA,R0 FNREPLY
        COMI,R0 FCACK
        BCFR,EQ CGO9
        EORZ    R0
CGO9:   RETC,UN

; CDO0: command R3, no parameters. CDO1: command R3, 1-byte param R1.
; Results as CGO. Calls two down. Clobbers R0-R3.
CDO0:   LODI,R2 0
        ZBSR    *ZMSTART
        BCTR,UN CGO
CDO1:   LODI,R2 1
```

```
151         ZBSR    *ZMSTART
152         ZBSR    *ZMPARB
153         BCTR,UN CGO
154
155 ; CDENT: READ_DIR_ENTRY with maxlen R1 (never below 36: the firmware
156 ; only appends a directory's '/' when it fits). R0 = 0 / CC EQ with the
157 ; name in the window; else an error code, or $7F at the end of the
158 ; directory (the reply is two $7F bytes). Calls two down.
159 CDENT:  LODI,R3 FCRDIR
160         LODI,R2 2
161         ZBSR    *ZMSTART
162         ZBSR    *ZMPARB           ; maxlen
163         LODI,R1 0               ; no extra metadata (keeps icons out)
164         ZBSR    *ZMPARB
165         BSTR,UN CGO
166         BCFR,EQ CDE9
167         LODA,R0 FNRDATA
168         COMI,R0 $7F
169         BCFR,EQ CDEOK
170         LODA,R0 FNRDATA+1
171         COMI,R0 $7F
172         BCTR,EQ CDE9            ; R0 = $7F, CC EQ against $7F...
173 CDEOK:  EORZ    R0
174         RETC,UN
175 CDE9:   LODI,R0 $7F             ; ...so reload: CC must be not-EQ here
176         RETC,UN
177
178 ; MCHECK: R0 = 0 / CC EQ when the 'F','N' cart magic is present. Leaf.
179 MCHECK: LODA,R0 FNMAGF
180         COMI,R0 'F'
181         BCFR,EQ MCKNO
182         LODA,R0 FNMAGN
183         COMI,R0 'N'
184         BCFR,EQ MCKNO
185         EORZ    R0
186         RETC,UN
187 MCKNO:  LODI,R0 $FF
188         RETC,UN
189
190 ; ---- pointer words: the page-0 cells page-1 code touches ------------------
191 ; Indirect operands carry 15 bits, so STRA,R0 *PVCUR is page-1 code's
192 ; three-byte "STRA,R0 V_CUR"; an indexed indirect (*PSTUBM,R3) reaches a
193 ; whole page-0 buffer through one word.
194 PDSTRH: DWBE    DSTRP
195 PDSTRL: DWBE    DSTRP+1
196 PDCOL:  DWBE    DCOLOR
197 PFSTRH: DWBE    FSTRP
198 PFSTRL: DWBE    FSTRP+1
199 PVCMOD: DWBE    V_CMODE
200 PVNNET: DWBE    V_NNET
201 PVCUR:  DWBE    V_CUR
202 PVCNT:  DWBE    V_CNT
203 PVHLW:  DWBE    V_HLW
204 PVHOST: DWBE    V_HOST
205 PVSSID: DWBE    V_SSID
206 PVPASS: DWBE    V_PASS
207 PVCTRY: DWBE    V_CTRY
208 PVBSRC: DWBE    V_BSRC
209 PVTXN:  DWBE    V_TXN
210 PVT2:   DWBE    V_T2
211 PVT3:   DWBE    V_T3
212 PVT4:   DWBE    V_T4
213 PVT5:   DWBE    V_T5
214 PSTUBM: DWBE    STUB-1
215
```

CONFIG/BOOT.INC `config/boot.inc`

```
 1 ; boot.inc -- SET_DEVICE_FULLPATH + MOUNT_IMAGE with a progress bar,
 2 ; then the swap (block 2: page-0 RAM through mailbox.inc's pointer
 3 ; words, the status page direct).
 4 ;
 5 ; MOUNT_IMAGE makes the ESP32 pull the file and push it to the cart
 6 ; over the DBC side-channel BEFORE it ACKs, so the commit is done by
 7 ; hand (BCOMMIT): SEQ written, then -- while ACKSEQ is still pending --
 8 ; only the status page is read (boot state / percent, never a hotspot,
 9 ; which would queue a transaction) and the bar repainted. The cart's
10 ; own deadline for a mount is 60 s; ours is 75 s with no progress.
11 ; Under MAME the push runs inside the SEQ read itself, so the bar can
12 ; only jump to 100%.
13 ;
14 ; This cart's firmware does not reset the boot registers when a mount
15 ; starts, so the verdict is read AFTER the ACK, and a FAILED before it
16 ; is ignored. ACK + still IDLE means the server mounted the file as a
17 ; plain disk and pushed nothing (not a cartridge image). A READY left
18 ; over from a RESET between READY and the swap reads as fresh; nothing
19 ; on this protocol version can tell the two apart.
20 ;
21 ; On READY: BOOTLOCK, the 11-byte stub into RAM at $18E0, and a jump to
22 ; it. The cart serves the staged image from the stub's read of the swap
23 ; hotspot on; the stub's BCTA to $0000 lands in it. CONFIG_BOOT ($D9)
24 ; is never sent: CONFIG lives in the cart's flash.
25
26 STUB    EQU     $18E0
27 BTIMO   EQU     75*60           ; frames with no progress before giving up
```

```
28
29 ; BOOTSEL: boot the browser's cursor row from V_HOST.
30 BOOTSEL: ZBSR   *ZSFULL
31         BCFA,EQ SFERR
32         EORZ    R0
33         STRA,R0 *PVBSRC
34         BSTA,UN BTSCR
35         SETST1  FNRDATA         ; the name, before any commit repaints it
36         BSTA,UN BTNAME
37         BSTA,UN BTSDFP          ; path = V_PATH + the window name
38         SETFS1  V_PATH
39         ZBSR    *ZCTXSTR
40         SETFS1  FNRDATA
41         ZBSR    *ZCTXSTR
42         BSTA,UN BTSDF2
43         BSTA,UN CCDIR
44         BCTR,UN BOOTMI
45
46 ; BOOTLB: boot the lobby image from V_HOST (already mounted).
47 BOOTLB: LODI,R0 1
48         STRA,R0 *PVBSRC
49         BSTA,UN BTSCR
50         SETST1  TLOBP
51         BSTA,UN BTNAME
52         BSTA,UN BTSDFP
53         SETFS1  TLOBP
54         ZBSR    *ZCTXSTR
55         BSTA,UN BTSDF2
56
57 BOOTMI: EORZ    R0              ; the empty bar
58         ZBSR    *ZBARPCT
59         LODI,R3 FCMIMG
60         LODI,R2 2
61         ZBSR    *ZMSTART
62         LODI,R1 0               ; device slot 0
63         ZBSR    *ZMPARB
64         LODI,R1 1               ; read-only
65         ZBSR    *ZMPARB
66         BSTA,UN BCOMMIT
67         BCTR,EQ BTVERD
68         STRZ    R2              ; a NAK or timeout: the cart's verdict,
69         LODA,R0 FNBSTAT         ;   if it failed the push, names the
70         COMI,R0 FBFAIL          ;   cause (it refuses an image > 8K at
71         BCTA,EQ BTBERR          ;   the stream open, and the server
72         LODZ    R2              ;   then NAKs the mount)
73         BCTA,UN BTMERR
74 BTVERD: LODA,R0 FNBSTAT         ; the verdict, now that it is fresh
75         COMI,R0 FBREADY
76         BCTA,EQ BREADY
77         COMI,R0 FBFAIL
78         BCTA,EQ BTBERR
79         COMI,R0 FBIDLE
80         BCTA,EQ BTNCRT
81         LODI,R3 0               ; still XFER: up to ~30 s more
82 BTW1:   LODI,R2 7
83 BTW2:   ZBSR    *ZWAITVB
84         BDRR,R2 BTW2
85         LODA,R0 FNBSTAT
86         COMI,R0 FBREADY
87         BCTR,EQ BREADY
88         COMI,R0 FBFAIL
89         BCTA,EQ BTBERR
90         BDRR,R3 BTW1
91         LODI,R0 FETIMO
92         BCTA,UN BTMERR
93
94 BREADY: LODI,R0 100
95         ZBSR    *ZBARPCT
96         BSTA,UN SNDFUJI
97         EORZ    R0              ; a silent machine for the next image
98         STRA,R0 *PPITCH         ;   (this also leaves multicolor)
99         STRA,R0 *PVOLUM
100        LODI,R1 FRBOOTL
101        LODI,R2 FNBLMAG
102        BSTA,UN MREGWR
103        LODI,R3 STUBLEN
104 STCPY:  LODA,R0 STUBSRC-1,R3
105        STRA,R0 *PSTUBM,R3
106        BDRR,R3 STCPY
107        BCTA,UN STUB
108
109 ; The stub, as bytes: it runs at $18E0, and AS would refuse its
110 ; page-0 operand assembled here in page 1.
111 ;       EORZ R0 / LPSU / LPSL       PSU/PSL as at power-on
112 ;       LODA,R0 *$18E9              the swap read: the cart flips here
113 ;       BCTA,UN $0000               into the staged image
114 ;       DWBE FNSWAP                 (the pointer word at $18E9)
115 STUBSRC: DB     $20,$92,$93,$0C,$98,$E9,$1F,$00,$00
116        DWBE    FNSWAP
117 STUBLEN EQU     $-STUBSRC
118
119 ; the failures: a message and back where the boot came from.
120 BTBERR: LODA,R0 FNBERR
121        STRZ    R2
122        COMI,R0 1
123        BCFR,EQ BTBE2
124        LODI,R0 EBTOOB>>8
```

```
         LODI,R1 EBTOOB&$FF
         BCTR,UN BTFAIL
BTBE2:   COMI,R0 2
         BCFR,EQ BTBE3
         LODI,R0 EBTRNC>>8
         LODI,R1 EBTRNC&$FF
         BCTR,UN BTFAIL
BTBE3:   LODI,R0 ESTAGE>>8
         LODI,R1 ESTAGE&$FF
         BCTR,UN BTFAIL
BTNCRT:  STRZ    R2
         LODI,R0 ENCART>>8
         LODI,R1 ENCART&$FF
         BCTR,UN BTFAIL
BTPERR:  STRZ    R2
         LODI,R0 EPATH>>8
         LODI,R1 EPATH&$FF
         BCTR,UN BTFAIL
BTMERR:  STRZ    R2
         LODI,R0 EMOUNT>>8
         LODI,R1 EMOUNT&$FF
BTFAIL:  ZBSR    *ZUFAIL
         LODA,R0 *PVBSRC
         BCTA,EQ SFDRAW
         ZBRR    *ZHOSTPG

; BTSCR: the boot screen frame. Calls three down.
BTSCR:   LODI,R0 TBOOT>>8
         LODI,R1 TBOOT&$FF
         ZBSR    *ZUTITLE
         PRINT   TNOPOW
         RETC,UN

; BTNAME: the tail of the string at (DSTRP) on row 2. Calls two down.
BTNAME:  DCOL1   AACC
         DAT     2,0
         LODI,R1 16
         ZBRR    *ZDTAIL

; BTSDFP: begin SET_DEVICE_FULLPATH(slot 0, V_HOST, read); the caller
; streams the path, then BTSDF2 pads and commits it (a failure does not
; return: it reports and leaves). Calls two down.
BTSDFP:  LODI,R3 FCSDFP
         LODI,R2 3
         ZBSR    *ZMSTART
         LODI,R1 0
         ZBSR    *ZMPARB
         LODA,R1 *PVHOST
         ZBSR    *ZMPARB
         LODI,R1 1
         ZBSR    *ZMPARB
         EORZ    R0
         STRA,R0 *PVTXN
         RETC,UN
BTSDF2:  BSTA,UN CPAD
         ZBSR    *ZCGO
         BCFA,EQ BTPERR
         RETC,UN

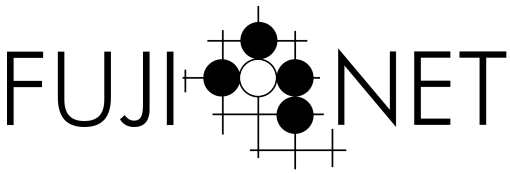
; BCOMMIT: commit the staged MOUNT_IMAGE and repaint the bar from the
; status page while it is outstanding. Results as CGO. Calls three down.
; Clobbers R0-R3, V_T2-V_T5 (V_T2 doubles as BARPCT's own copy of the
; percent, which is the same value).
BCOMMIT: LODI,R0 $FF
         STRA,R0 *PVT2           ; the percent last drawn
         EORZ    R0
         STRA,R0 *PVT4           ; frames without progress, HI,LO
         STRA,R0 *PVT5
         BSTA,UN MSEQ
         STRA,R0 *PVT3           ; the ACKSEQ that means "done"
BCL:     LODA,R0 FNACKSQ
         COMA,R0 *PVT3
         BCTA,EQ BCACK
         LODA,R0 FNBPCT
         COMA,R0 *PVT2
         BCTR,EQ BCL1
         STRA,R0 *PVT2           ; progress: repaint, restart the clock
         ZBSR    *ZBARPCT
         EORZ    R0
         STRA,R0 *PVT4
         STRA,R0 *PVT5
BCL1:    ZBSR    *ZWAITVB
         LODA,R0 *PVT5
         ADDI,R0 1
         STRA,R0 *PVT5
         BCFR,EQ BCL2
         LODA,R0 *PVT4
         ADDI,R0 1
         STRA,R0 *PVT4
BCL2:    LODA,R0 *PVT4
         COMI,R0 BTIMO>>8
         BCFR,EQ BCL
         LODA,R0 *PVT5
         COMI,R0 BTIMO&$FF
         BCFR,EQ BCL
         LODI,R0 FETIMO
         RETC,UN
```

```
222 BCACK:  LODA,R0 FNERR
223         BCTA,UN CGOCHK
224
```

)

ARCADIA-2001





A FujiNet Project publication

FN2001-PH REV. A

Typeset in Helvetica